

Física & JSPlotly

Índice

1. Introdução.....	2
2. Mecânica: Por que as coisas se movem ?.....	2
2.1 Movimento.....	5
Como descrever.....	5
Por que muda.....	5
Interações rápidas: impulso.....	5
2.2 Comparando a dinâmica de 2 corpos.....	8
3. Eletricidade e Magnetismo: forças sem contato que moldam o mundo.....	12
3.1 Vendo o campo magnético no espaço.....	17
3.2 Regra da mão direita.....	19
4. Ondas: do movimento ao som e à luz.....	21
4.1 Amplitude.....	24
4.2 Comprimento de onda.....	24
4.3 Frequência.....	24
4.4 Período.....	24
4.5 Tempo.....	25
4.6 Fase.....	25
4.6.1 Amplitudes.....	29
4.6.2 Frequências.....	29
4.6.3 Comprimentos de onda.....	29
4.6.4 Tempo.....	29
4.6.5 Índices dos meios.....	30
4.7 Fenômenos de interação.....	30
4.7.1 Interferência.....	30
4.7.2 Reflexão.....	30
4.7.3 Refração.....	31
4.8 Lentes delgadas.....	32
4.8.1 O que cada parâmetro faz.....	33
5. Gravitação e Fluidos — quando a gravidade molda a matéria.....	38
6. Transformações Invisíveis: Termodinâmica, Gases e Soluções.....	52

Uma animação para gases reais.....	62
6.1 Estrutura de JS.....	68
7. Radioatividade, Energia Nuclear e Impactos.....	69
Agite antes de usar.....	71
Separação de comandos, inserção de comentários, e boas práticas.....	84
Declaração de variáveis.....	84
“Botando a mão na massa”.....	84
Tipos de dados.....	86
Constantes.....	86
Funções.....	87
8. Relatividade e Quântica — Quando espaço, tempo, energia e matéria deixam de ser intuitivos.....	89
8.0.1 Relatividade: espaço e tempo em movimento.....	94
8.0.2 Massa e energia: duas formas de representar a mesma realidade.....	94
8.0.3 Quântica: energia, luz e matéria em pacotes.....	95
8.0.4 Dualidade onda-partícula.....	95
Agite antes de usar.....	97
Aritmética.....	105
Arrays.....	106
Uma palavrinha sobre métodos.....	107
JavaScript, uma linguagem orientada a objetos.....	108

1. Introdução

Introdução

2. Mecânica: Por que as coisas se movem ?

O MUNDO TE CHAMA



Um carro arranca lentamente, ganha velocidade e depois freia até parar. Uma bola parada passa a se mover quando é chutada. Um aviõzinho de papel é atirado de uma janela. Em uma colisão, um corpo pode mudar drasticamente seu movimento em um intervalo de tempo muito curto. Essas e tantas outras situações fazem parte do cotidiano e, embora sejam bastante distintas, compartilham o que faz um corpo se mover ou mudar de movimento na Mecânica. Neste capítulo será abordado como diferentes formas de descrever o movimento — posição, velocidade e aceleração — se conectam com suas causas físicas, como a força e o impulso.

Depois de explorar, você consegue:

- interpretar o movimento a partir de gráficos (1a. e 2a. Leis de Newton);
- relacionar velocidade e aceleração com mudanças no movimento;
- compreender o papel da força resultante;
- perceber como interações rápidas produzem efeitos significativos;
- usar visualizações interativas para aprendizagem de movimento.

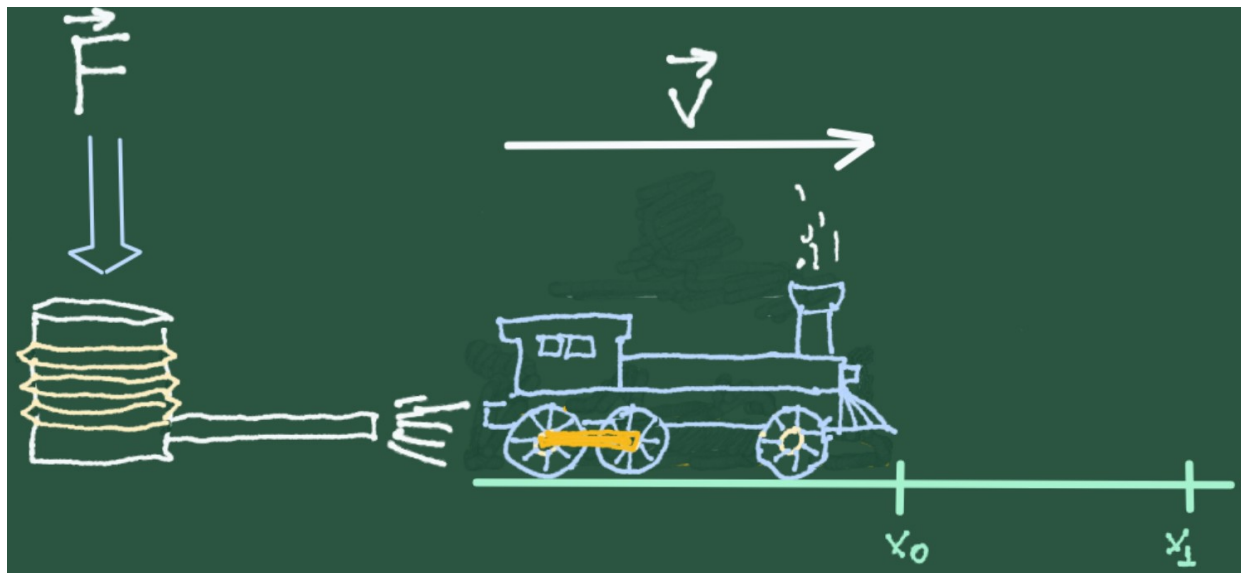


Figura 2.1: Quando a força aplicada a um objeto promove sua aceleração e consequente mudança de posição.

MEXA ANTES DE ENTENDER



Antes de qualquer definição formal, experimente o objeto interativo a seguir. Sem pressa. Sem fórmulas.

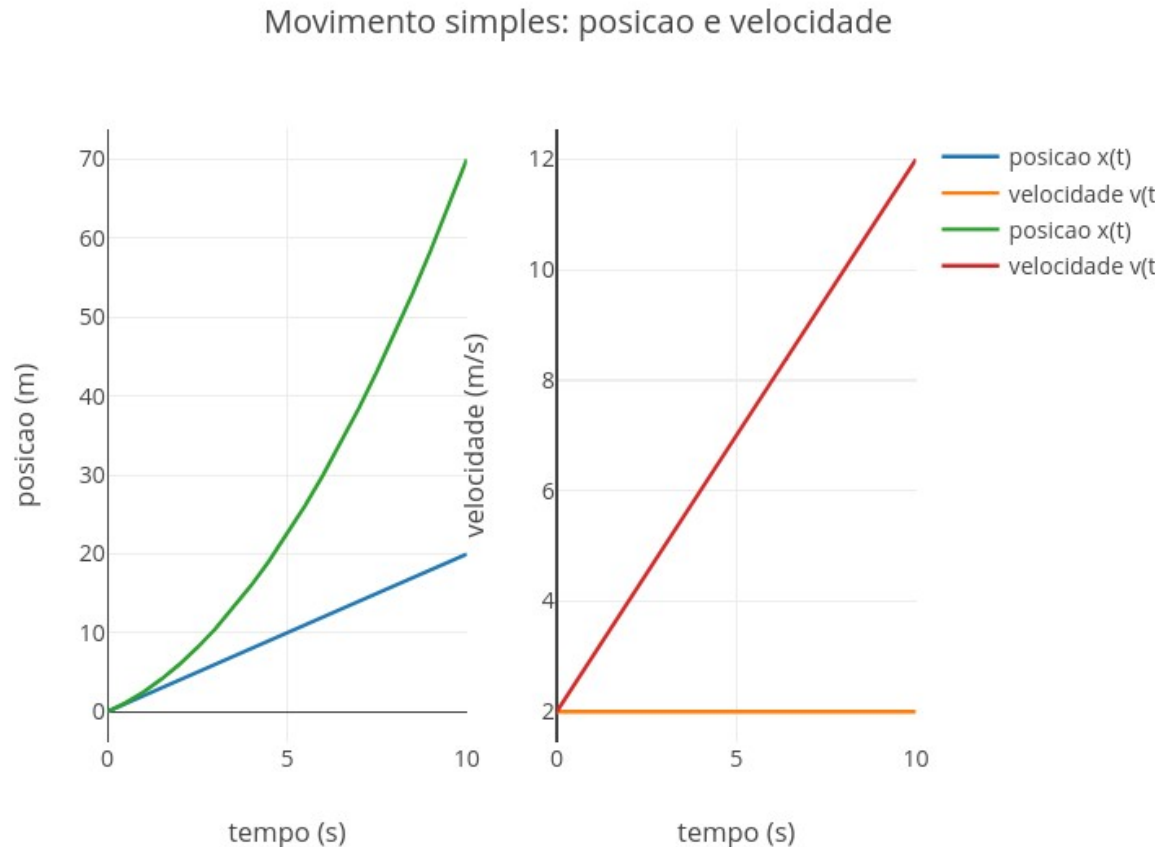


Figura 2.2: Aplicativo de JSPlotly para explorar o movimento. Clique com o botão direito do mouse neste [LINK](#) e abra-o numa nova aba. Ou use a tela capacitiva de seu dispositivo móvel (celular, tablet).

Observe como a posição muda linearmente (função afim) sob uma velocidade fixa para o objeto. Dobre o valor da velocidade inicial v_0 e clique em *add* novamente, visualizando como fica o gráfico:

```
const v0 = 4;
```

Essa forma linear em que o gráfico de posição é apresentado dá-se em função do MRV, ou movimento retilíneo uniforme. Veja que o editor de códigos apresenta a opção de MRV em seu início. Experimente passar essa opção para MUV, movimento retilíneo uniformemente variado. Pra isso, clique em *clean*, apenas para limpar a área dos gráficos, altere o termo dentro das aspas, e depois clique em *add*, como segue:

```
const tipo = "MUV"; // "MRU" ou "MUV"
```

Perceba agora que a velocidade em MUV cresce linearmente, enquanto a distância cresce como uma curva. Isso ocorre porque o objeto está sofrendo uma aceleração. Experimente dobrar agora o valor da aceleração, como você fez anteriormente para a velocidade inicial, e veja o que acontece. Mexendo um pouco naqueles parâmetros do editor, tente

observar também quando o gráfico da posição é uma reta, quando ele se curva, quando a velocidade se mantém constante, e quando ela cresce ou diminui.

O *app* suscita algumas questões curiosas sobre posição e velocidade durante o movimento. Você saberia responder se é possível estar em movimento sem mudar a velocidade ? Ou o que muda primeiro, se a posição ou a velocidade ? Saberia como reconhecer aceleração apenas olhando um gráfico ? E por que mudar o valor de a não altera a forma do gráfico quando o movimento é MRU ?



FAZENDO APARECER

2.1 Movimento

O primeiro passo é descrever o movimento, para depois entender por que ele muda.

Como descrever

A *posição* indica onde um corpo está. A *velocidade* indica como essa posição muda com o tempo. Já a *aceleração* revela como a velocidade varia. Quando a velocidade é constante, a posição cresce de forma linear. Quando há aceleração, a velocidade deixa de ser constante, e a posição passa a crescer de forma cada vez mais rápida ou mais lenta.

Por que muda

Para entender por que o movimento muda, entra em cena a ideia de *força*. A força resultante sobre um corpo está relacionada à sua aceleração. Isso significa que forças não produzem diretamente velocidade, mas sim mudanças na velocidade.

Interações rápidas: impulso

Nem toda mudança de movimento acontece lentamente. Um chute, uma batida ou uma rebatida são exemplos de interações rápidas. Nesses casos, uma força atua por um intervalo de tempo muito curto, mas ainda assim produz uma mudança significativa no movimento. Esse efeito é descrito pelo *impulso*, que representa a variação da quantidade de movimento de um corpo. Assim, o movimento pode ser entendido como:

$$\text{Força} \rightarrow \text{Aceleração} \rightarrow \text{Velocidade} \rightarrow \text{Posição}$$

Ou seja, forças produzem aceleração, a aceleração altera a velocidade, e a velocidade modifica a posição.

VOLTA PRO MUNDO



Volte às situações iniciais. Quando um carro mantém velocidade constante, seu movimento continua, mas não muda. Quando acelera, sua velocidade cresce. Quando freia, diminui. Em uma colisão, essa mudança pode ocorrer de forma abrupta. O que diferencia esses casos não é apenas o movimento em si, mas como ele evolui ao longo do tempo. E essa evolução depende da aceleração.

Experimente alterar a aceleração para um valor negativo no aplicativo JSPlotly da [Figura 2.2](#), bem como para um valor nulo. Observe que:

- quando $a > 0 \rightarrow$ reta inclinada pra cima
- quando $a < 0 \rightarrow$ reta inclinada pra baixo
- quando $a = 0 \rightarrow$ reta horizontal

Então...

Qual grandeza revela melhor o que está acontecendo em cada momento do movimento?

Assim como para diversos domínios da Física, o MRV e MUV são descritos por funções matemáticas específicas para a posição e velocidade. Para MRU:

$$x(t) = x_0 + v_0 t$$

$$v(t) = v_0$$

E para MUV:

$$x(t) = x_0 + v_0 t + \frac{1}{2} a t^2$$

$$v(t) = v_0 + a t$$

E SE...



Agora é hora de ir além. Juntar num “*mesmo saco*” posição, velocidade, aceleração, força (2ª Lei de Newton), e impulso. Para isso experimente as simulações do aplicativo abaixo para JSPlotly.

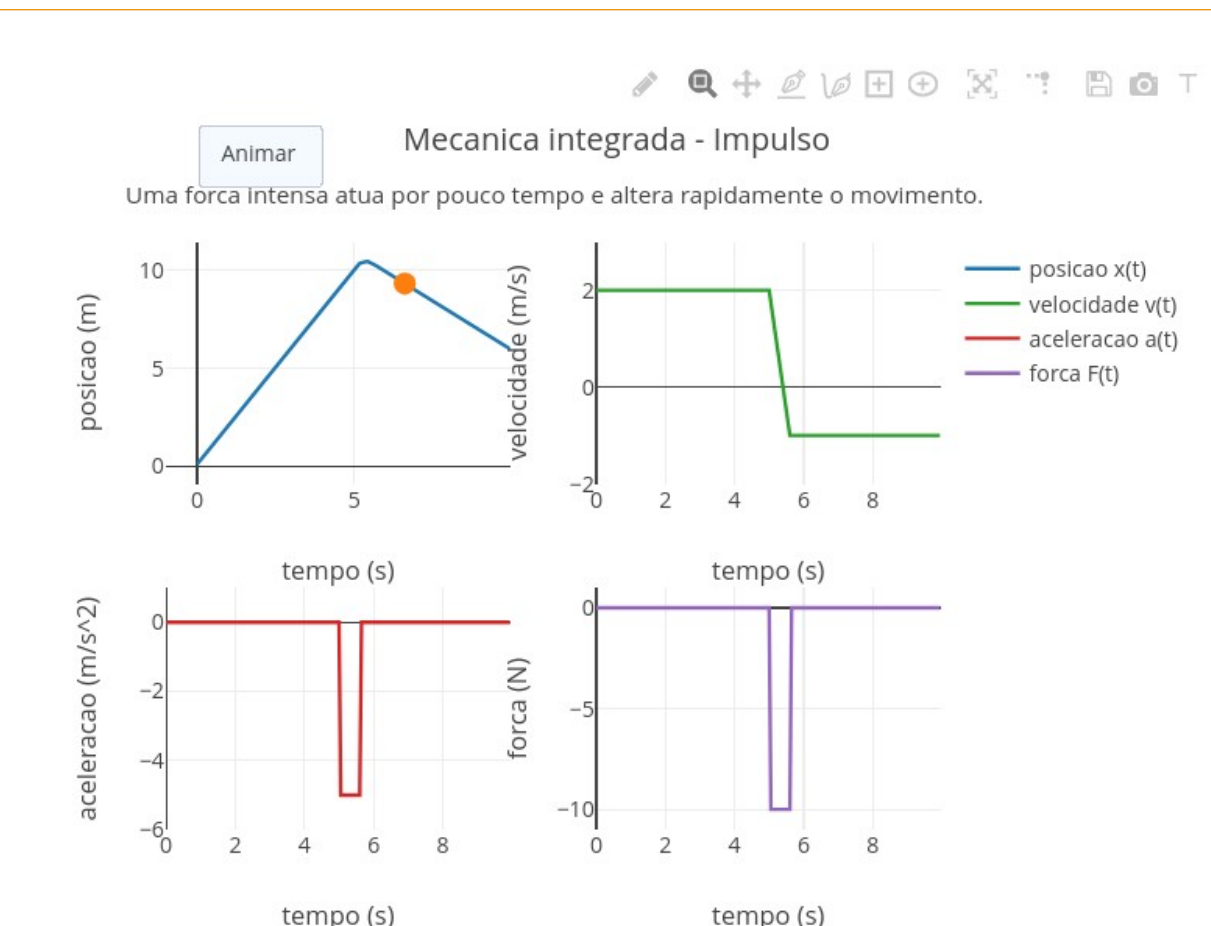


Figura 2.3: Aplicativo JSPlotly para explorar posição, velocidade, aceleração e força. Clique com o botão direito do mouse neste [LINK](#) e abra-o numa nova aba. Ou use a tela capacitiva de seu dispositivo móvel (celular, tablet).

Neste segundo objeto interativo, várias grandezas aparecem ao mesmo tempo (força, aceleração, massa, e impulso). Sendo um pouco mais descritivo, o *app* busca integrar MRU, MUV, 2ª Lei de Newton e impulso, conforme as relações abaixo.

$$\text{MRU: } \underset{\text{posição}}{x(t)} = \underset{\text{velocidade}}{x_0} + v_0 t \quad \underset{\text{aceleração}}{x(t)} = x_0 + v_0 t + \frac{1}{2} a t^2 \quad \underset{\text{força}}{F(t)} = m a(t) \quad \text{Impulso: } J = \Sigma [F(t) * t]$$

Na última equação acima, “ Σ ” é o símbolo para *somatória* e, no caso, do produto da força por cada pequeno intervalo de tempo.

Agite antes de usar

Observe como posição, velocidade, aceleração e força se conectam. Neste segundo momento, o objeto interativo apresenta a descrição do movimento, mas agora incorporando sua interpretação física mais integrada. Nessa, posição, velocidade, aceleração e força permitem que se observe como a mudança em uma grandeza repercute nas demais. Para utilizar esse novo *app* de JSPlotly para MRU, MUV, força, e impulso, experimente seguir esse pequeno tutorial:

1. Altere const cenário para escolher entre "MRU", "MUV", "Força constante", "Freada" e "Impulso";

2. Em "MRU", observe movimento com velocidade constante e força nula;
3. Em "MUV", altere a_{Const} e veja como a aceleração muda a velocidade;
4. Em "Força constante", varie m e F_{Const} para observar a relação entre força, massa e aceleração;
5. Em "Freada", aumente a_{Const} para simular uma desaceleração mais intensa;
6. Em "Impulso", modifique t_{Imp} , Δt_{Imp} e J para controlar quando, por quanto tempo e com que intensidade a força atua;
7. Use o botão *Animar* para acompanhar o corpo no gráfico de posição;
8. Compare os gráficos de posição, velocidade, aceleração e força, para perceber como uma mudança física se propaga pelo movimento.

Tendo experimentado as variações acima, você estará mais do que apto para aventurar-se como um *mergulhador abissal* da Mecânica do movimento, não apenas observando-o, mas compreendendo-o. E poderá trabalhar com cenários diversos para melhor compreensão das relações de movimento, tais como:

1. MRU - como um corpo pode continuar se movendo mesmo sem força resultante (ou...por que em MRU a força é zero e a velocidade não) ?
2. MUV - como a aceleração constante aparece ao mesmo tempo em $v(t)$ e $x(t)$ (ou...o que muda primeiro quando aplicamos força) ?
3. Força constante - por que aumentar a massa enfraquece o efeito da mesma força ?
4. Freada - o que significa uma aceleração negativa ?
5. Impulso - como uma força breve consegue alterar tanto o movimento ?
6. Como o impulso aparece no gráfico da força ?
7. Por que a aceleração depende da massa ?

Experimente também modificar diversos parâmetros simultaneamente, como segue.

- mantenha a força igual a zero;
- inverta o sentido da aceleração;
- aumente a massa;
- aplique uma força intensa por pouco tempo.

E pergunte-se:

- E se não há força resultante ?
- E se a aceleração for negativa...como a velocidade responde ?
- E se aumentar a massa ?
- E se houver uma ação rápida, qual o tamanho do efeito ?

2.2 Comparando a dinâmica de 2 corpos

Agora experimente um último aplicativo JSPlotly, esse voltado para o estudo de posição, velocidade e aceleração de dois corpos submetidos a uma mesma força.

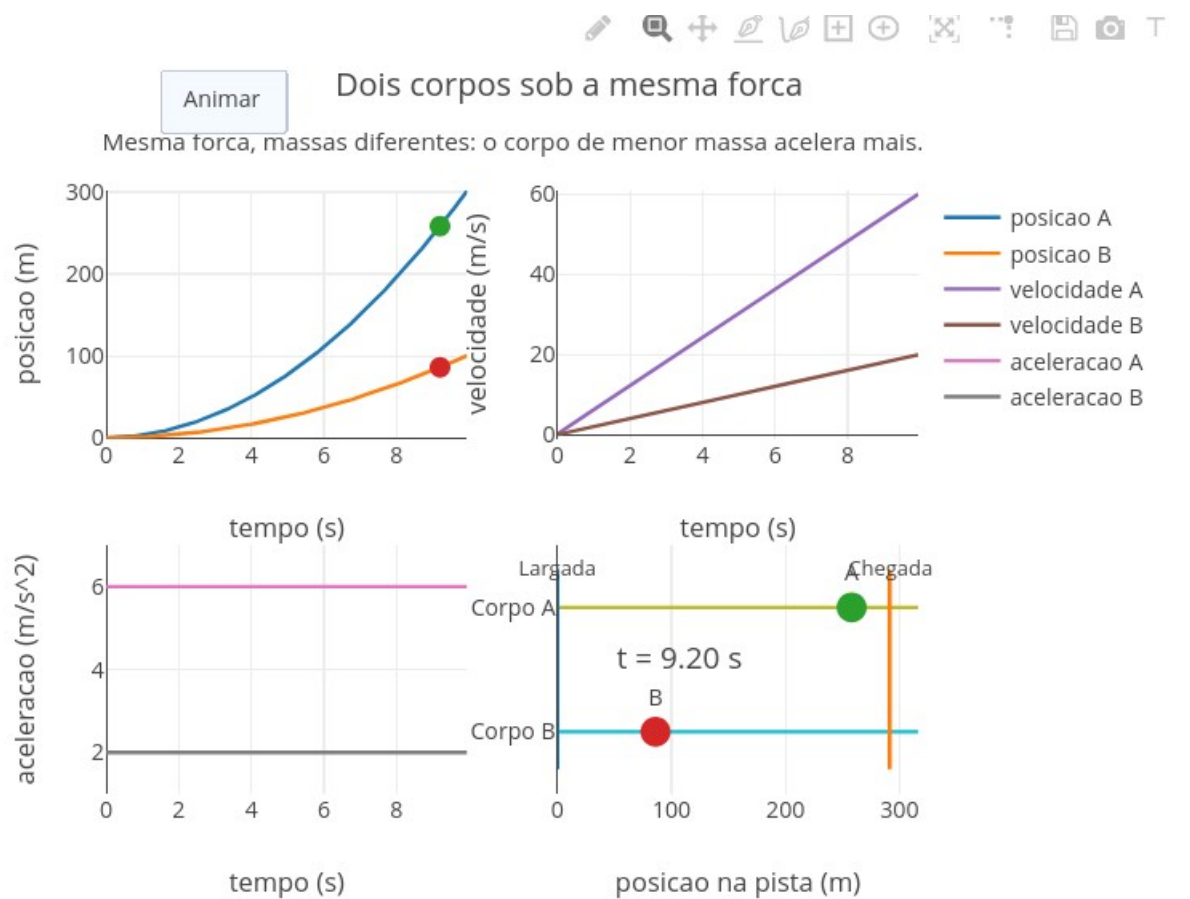


Figura 2.4: JSPlotly para visualizar as relações de posição, velocidade e aceleração de 2 corpos submetidos a uma mesma força. Clique [AQUI](#) para acessar o aplicativo.

Nesta nova versão, a comparação entre os dois corpos ganha um apoio visual adicional. A pista explicita a ideia de percurso, enquanto o contador de tempo reforça que a diferença entre os corpos não depende apenas da posição final, mas de como o movimento evolui ao longo do processo.

Agite antes de usar

Este app compara dois corpos sob a mesma força, destacando o papel da massa na aceleração.

1. Observe o cenário padrão. Dois corpos (A e B) partem da mesma posição e com velocidade inicial nula:

```
const x0A = 0.0;
const x0B = 0.0;
```

```
const v0A = 0.0;
const v0B = 0.0;
```

2. Compare as massas.

```
const mA = 1.0;  
const mB = 3.0;
```

O corpo A é mais leve que o corpo B.

3. Mantenha a mesma força para ambos

```
const Fconst = 6.0;
```

A força resultante é igual para os dois corpos. A ideia básica desse experimento é que a aceleração é calculada por:

$$a = \frac{F}{m}$$

Portanto, quanto menor a massa, maior a aceleração.

4. Preveja antes de rodar. Qual corpo chegará primeiro, o mais leve (A) ou o mais pesado (B)?
5. Clique em “Animar” e observe o gráfico de posição $x(t)$, o gráfico de velocidade $v(t)$, o de aceleração $a(t)$, e o de movimento na pista (largada e chegada).
6. Interprete os gráficos comparando a inclinação das curvas de posição, o crescimento da velocidade e o valor constante da aceleração.
7. Faça experimentos simples:

- Troque as massas:

```
const mA = 3.0;  
const mB = 1.0;
```

- Aumente a força:

```
const Fconst = 10.0;
```

- Dê velocidade inicial:

```
const v0A = 2.0;  
const v0B = 2.0;
```

Ufa !!! Se chegou até aqui, já deve ter percebido que *uma mesma força não significa um mesmo movimento; a massa é quem controla como o movimento responde à força*. Quer mais umas perguntinhas a responder com o app da [Figura 2.4](#) ? * Por que o corpo mais leve acelera mais ? * Se a força é a mesma, então por que os movimentos são diferentes ? * O que muda primeiro: aceleração, velocidade ou posição?



A mesma lógica apresentada neste capítulo de *Mecânica* também surge em diversos contextos. Por exemplo, em veículos acelerando e freando, objetos em queda, bolas sendo lançadas ou chutadas, colisões entre corpos, e sistemas mecânicos simples. A partir daqui, portanto, novos caminhos podem ser explorados no tema, incluindo queda livre e gravidade, distância de frenagem, análise de colisões, e movimentos mais complexos, cada qual nascente da mesma estrutura apresentada no capítulo. Em todos eles, a força gera aceleração, a aceleração muda a velocidade, e a velocidade muda a posição. Isso é movimento.

POR DENTRO DA FERRAMENTA



Esse espaço é devotado a aplicar um pouco do **pensamento computacional** diretamente na **lógica de programação** por trás dos objetos interativos dos conteúdos do capítulo. Nesse sentido, e selecionando o primeiro objeto para tal, define-se primeiro o cenário físico e as condições iniciais. Em seguida, o tempo é dividido em pequenos intervalos. Para cada instante, calculam-se as grandezas relevantes e atualiza-se o estado do sistema. Em resumo, as ações do código envolvem:

1. definir condições iniciais;
2. escolher o tipo de movimento;
3. dividir o tempo em pequenos intervalos;
4. para cada instante:
 - a. calcular a aceleração;
 - b. atualizar a velocidade;
 - c. atualizar a posição;
5. armazenar os resultados;
6. construir os gráficos

3. Eletricidade e Magnetismo: forças sem contato que moldam o mundo



O MUNDO TE CHAMA

A eletricidade está presente em praticamente todas as atividades humanas. Ela alimenta lâmpadas, celulares, computadores, eletrodomésticos, sistemas de comunicação e redes inteiras de transmissão de energia. Já o magnetismo aparece em motores, transformadores, alto-falantes, cartões magnéticos e em muitos dispositivos que usamos sem perceber. E é claro, um (quase) não vive sem o outro.

Mas o que realmente acontece quando ligamos um aparelho ? Como as cargas elétricas se movimentam em um circuito ? E por que um fenômeno aparentemente elétrico pode dar origem a efeitos magnéticos ? Isso pode nos levar a uma unificação curiosa da Física, a ideia de que eletricidade e magnetismo não são temas isolados, mas “duas faces de uma mesma moeda”.

Como algo invisível pode empurrar, puxar e mover objetos sem encostar neles ?

Depois de explorar, você consegue:

- compreender a relação entre tensão, corrente elétrica e resistência;
- interpretar graficamente situações simples de circuitos elétricos;
- reconhecer como correntes elétricas produzem campos magnéticos e como isso se conecta a tecnologias do cotidiano.
- relacionar tensão, corrente elétrica e resistência em situações simples;
- interpretar representações gráficas e espaciais de fenômenos eletromagnéticos;
- reconhecer aplicações da eletricidade e do magnetismo em tecnologias do cotidiano.
- interpretar fenômenos de eletricidade e magnetismo com auxílio de simuladores interativos.

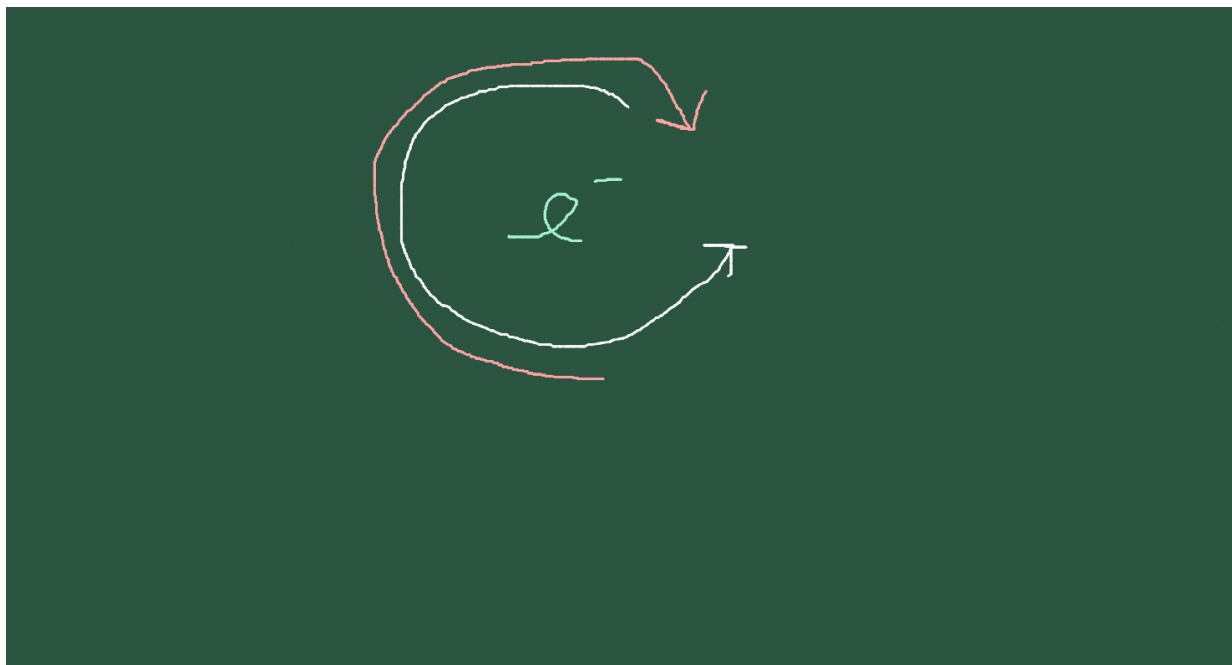


Figura 3.1: Corrente elétrica, resistência e campo magnético são ramos integrados.

MEXA ANTES DE ENTENDER



Antes de mergulhar nas explicações formais, experimente o JSPlotly abaixo para alterar os valores no código e observar os efeitos produzidos no gráfico.

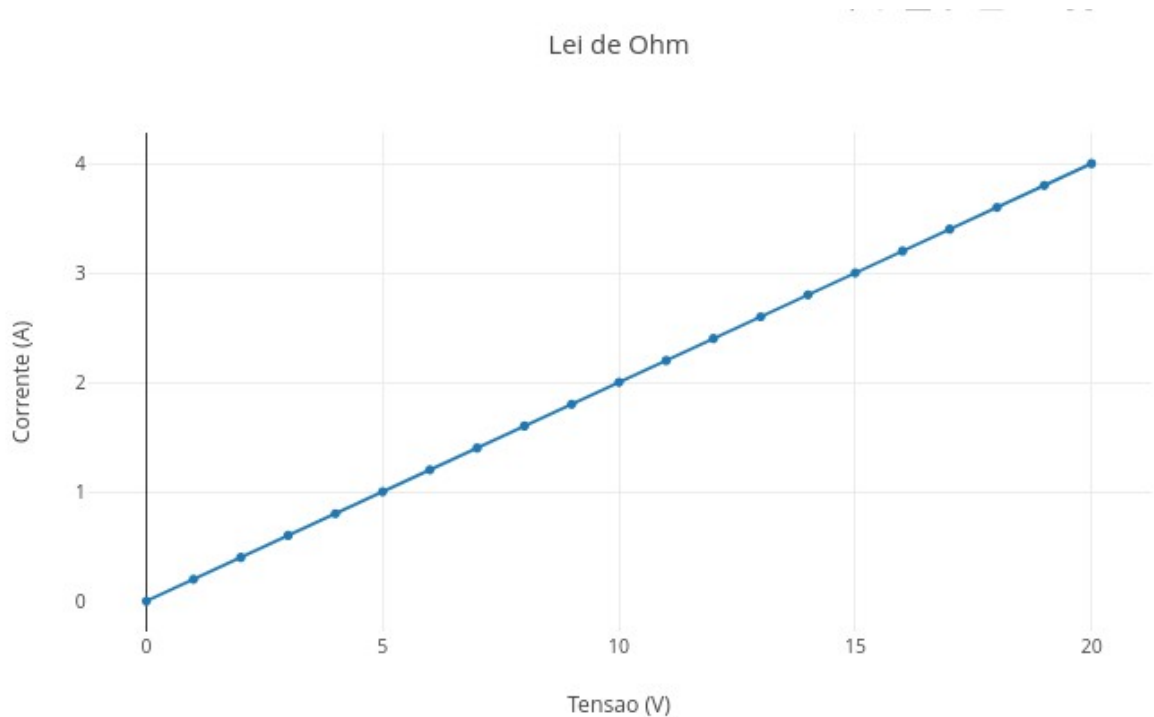


Figura 3.2: Lei de Ohm: quando tensão, corrente e resistência dependem um do outro. Clique neste [LINK](#) para carregar o app.

Agite antes de usar

O script mostra a relação entre *tensão elétrica* (ou potencial elétrico, V), *corrente elétrica* (I) e *resistência* (R). Para isso ele usa a 1ª. lei de Ohm:

$$V = R \cdot I \quad (3.1)$$

ou, reorganizando:

$$I = \frac{V}{R} \quad (3.2)$$

Onde: V é dado em Volts (V), I em Âmpere (A), e R em ohm (Ω).

Veja pelas [Equação 3.1](#) e [Equação 3.2](#) que o comportamento é puramente linear, e dependente da *resistência* definida no código. Ex:

```
const R = 7;
```

Observe que quando a tensão aumenta, a corrente também aumenta. Isso acontece porque a resistência permanece constante. Parece até meio óbvio, mas essa é a *alma* de um circuito elétrico *resistivo*. E que pode aparecer desde um simples LED de celular até uma *biocélula a combustível* ! Ou seja, a “Lei de Ohm” mostra que, em um resistor ôhmico:

$$I \propto V$$

Isso significa que tensão e corrente são diretamente proporcionais. A inclinação da reta informa como a corrente responde à tensão. Quanto maior a resistência, menor a inclinação. Quanto menor a resistência, maior a inclinação. Assim, se a tensão dobra, a corrente também dobra. Experimente comparar situações com baixa e alta resistência, e tente prever o resultado antes de alterar o código. Em qual situação a corrente cresce mais “lentamente” ?

E compare também o que você experimentou acima com uma cena do cotidiano. Imagine uma lâmpada, um chuveiro ou um carregador. Todos dependem da relação entre tensão, corrente, e resistência. Agora, se a corrente for do tipo que “sai” da tomada de casa, bom, aí a coisa muda de figura, pois entra em ação o *magnetismo*. E muda de figura também a pergunta central deste capítulo:

Como descrever, prever e visualizar a relação entre tensão, corrente elétrica, resistência e efeitos magnéticos em situações simples, mas fisicamente significativas ?

Em outras palavras, queremos entender como uma diferença de potencial elétrico (tensão) pode colocar cargas em movimento (corrente), como esse movimento encontra oposição no circuito (resistência), e como, a partir daí, surgem efeitos magnéticos observáveis.

VOLTA PRO MUNDO



Agora saímos da observação inicial e voltamos ao mundo real com mais ferramentas de interpretação: um simulador JSPlotly mais completo.

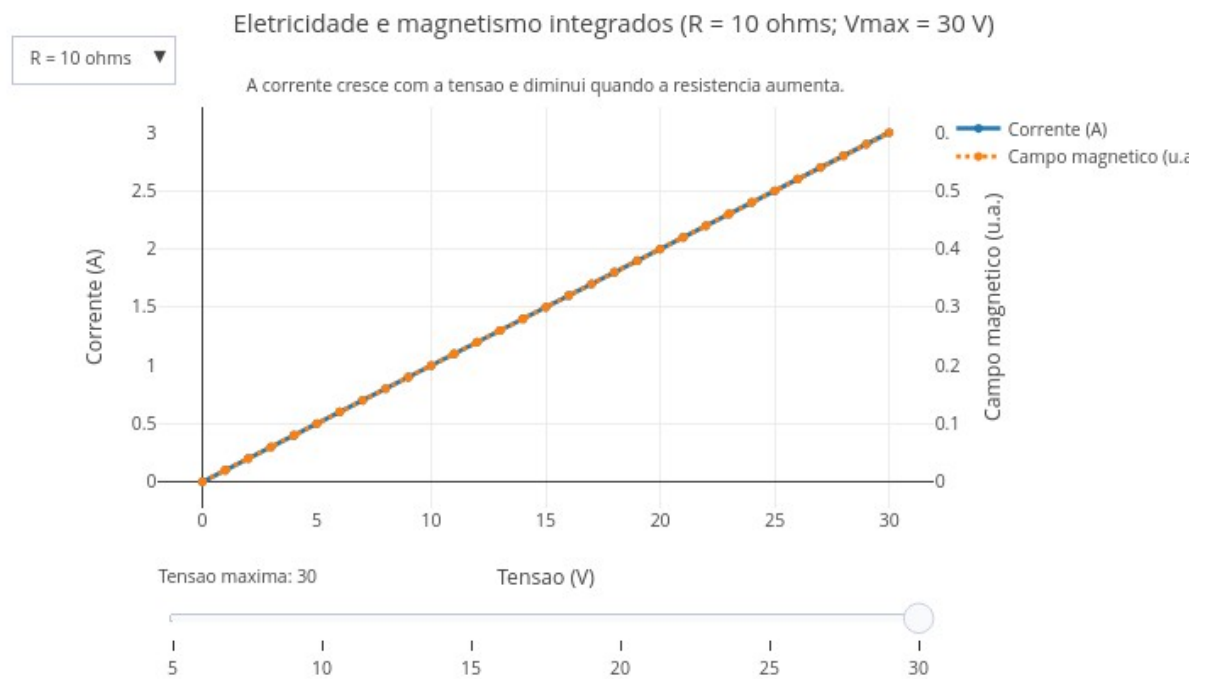


Figura 3.3: Integrando eletricidade e campo magnético. Tensão produz corrente, e corrente gera magnetismo. Clique no [LINK](#) para rodar o aplicativo.

No simulador você poderá explorar a relação entre tensão, corrente e resistência, a variação da intensidade da corrente em diferentes condições, a associação entre corrente elétrica e campo magnético, e a leitura integrada de mais de uma dessas grandezas ao mesmo tempo.

Agite antes de usar

O *script* apresenta uma curva de corrente, uma curva de campo magnético relativo, um menu suspenso (*dropdown*) para resistência, e um controle deslizante (*slider*) de tensão máxima. O menu permite escolher o valor para resistência (2, 5, ou 10 ohms), enquanto o *slider* um valor máximo para tensão (5 a 30 V). Quando você muda isso, a corrente muda, e a curva do campo magnético muda junto.

1. *Iniciando o aplicativo.* Inicie com $R = 5 \text{ ohms}$ e $V_{\text{max}} = 10 \text{ V}$.
 - O que acontece com a corrente quando a tensão cresce ?
 - O gráfico parece linear ?
 - Por que a curva do campo acompanha a da corrente ?
2. *Mexendo na resistência.* Troque no menu de $R = 5$ para $R = 10$.
 - Para a mesma tensão, a corrente aumentou ou diminuiu ?
 - O campo magnético ficou mais intenso ou menos intenso ?
 - O que isso sugere sobre a relação entre resistência e fluxo de cargas ?
3. *Mexendo no slider.* Agora arraste o slider para $V_{\text{max}} = 20$, e depois $V_{\text{max}} = 30$.
 - O que mudou no alcance do gráfico ?
 - A forma da curva mudou ou apenas sua extensão ?
 - O modelo continua obedecendo à mesma relação física ?

Você deve ter percebido que o campo magnético é proporcional à corrente. No modelo simplificado do *script*, o campo magnético B é dado por:

$$B = k \cdot I$$

Onde: $k = 0.2$.

Substituindo $I = \frac{V}{R}$ em " $B = kI$ " :

$$B = k \cdot \frac{V}{R}$$

Essa é a ponte para se compreender como funciona um eletroímã (fio retilíneo com corrente gerando campo magnético em torno de um condutor). Muitos objetos do cotidiano seguem essas mesmas relações matemáticas, como as *redes elétricas*, nas quais tensão, corrente e resistência precisam ser equilibradas para transmissão eficiente, os *motores elétricos*, que convertem energia elétrica em movimento com base em interações eletromagnéticas, os *transformadores*, que operam pela íntima relação entre correntes e campos magnéticos variáveis, e os *dispositivos eletrônicos*, que dependem de controle preciso do fluxo de cargas.

Agora tente responder às questões que seguem, *mexendo* no aplicativo:

- Se a resistência de um circuito aumenta muito, o que acontece com a corrente elétrica para uma mesma tensão (experimente mudar R de 2 pra 10 ohms) ?
- Como isso aparece no gráfico ?
- Se a corrente diminui, o campo magnético relativo aumenta ou diminui ?
- Ao aumentar a tensão máxima no slider, a relação física muda ou apenas vemos mais pontos do mesmo comportamento ?
- Por que a curva de corrente é linear neste modelo ?

Resumindo, tensão, corrente, resistência e campo magnético guardam relações importantes que se expressam no cotidiano. Tensão elétrica está relacionada à "força de empurrão" sobre as cargas, enquanto que resistência representa a oposição à passagem da corrente. Por sua vez, correntes elétricas produzem campos magnéticos ao seu redor. E a integração dessas ideias permite compreender como operam dispositivos reais.



E SE...

Refletindo mais um pouco sobre as relações acima, é possível divagar para "outros mundos" da eletricidade e magnetismo. Por exemplo:

- E se a resistência nula ? Experimente “ $R = 1e-20$ ” no *app* da [Figura 3.2](#).
- E se não houver passagem de corrente, qual seria o valor da resistência ?
- E se aumentarmos a tensão máxima mantendo a resistência fixa ? Use o *app* da [Figura 3.3](#) para auxiliá-lo.
- E se aumentarmos a resistência mantendo a mesma tensão ?
- E se a resistência for pequena: por que a corrente cresce mais rapidamente ?
- E se duas situações tiverem a mesma tensão, mas resistências diferentes ?
- E se quisermos gerar um campo magnético maior: devemos aumentar a tensão ou reduzir a resistência ?
- E se a corrente for zero: ainda existe campo magnético nesse modelo ?
- E se a corrente fosse zero; ainda existiria campo magnético ?
- E se aumentarmos muito a corrente; o campo cresce indefinidamente ?
- E se nos afastarmos muito do fio; o campo desaparece completamente ?
- E se invertermos o sentido várias vezes; o campo acompanha instantaneamente ?
- E se existirem dois fios próximos; como os campos interagem ?
- E se o fio não for retilíneo; as linhas de campo continuam circulares ?
- E se o campo fosse representado por vetores em vez de círculos ?
- E se aproximarmos uma bússola do fio; O que aconteceria ?

3.1 Vendo o campo magnético no espaço

Nos gráficos anteriores, a relação entre tensão, corrente e resistência foi observada principalmente em duas dimensões. Agora, podemos avançar para uma representação espacial do campo magnético em torno de um fio retilíneo percorrido por corrente, como segue.

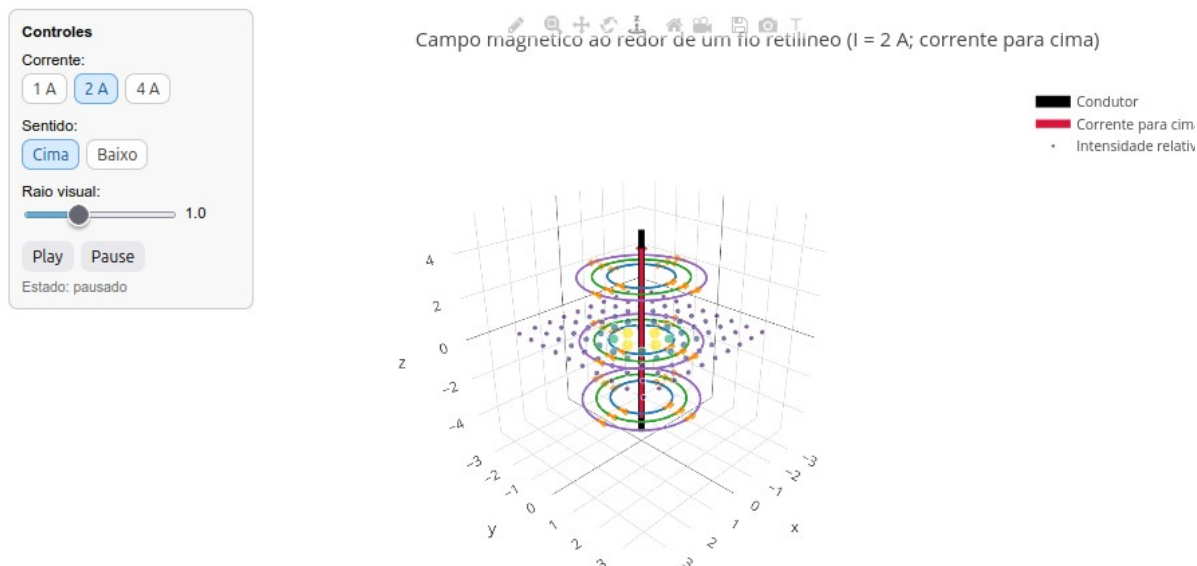


Figura 3.4: Visualização em 3D do campo magnético criado por uma corrente elétrica ao redor de um fio. Clique no [LINK](#) para rodá-lo.

Nessa visualização, o fio condutor aparece no centro, enquanto curvas ao seu redor sugerem as linhas de campo magnético. Ao variar a corrente, a representação da intensidade do campo também se altera.

Agite antes de usar

O app apresenta-se com:

1. Um fio retilíneo no centro;
2. Setas tangenciais ao redor do fio mostrando o sentido local do campo magnético;
3. Várias circunferências ao redor do fio representando as linhas de campo;
4. Uma seta vermelha indicando o sentido da corrente;
5. Pontos no plano indicando a ideia de que a intensidade do campo depende da distância ao fio.

Esse objeto interativo mostra como uma corrente elétrica em um fio gera um campo magnético ao seu redor. Esse campo que forma círculos ao redor do fio depende da corrente e da distância ao fio. Matematicamente, e de forma simplificada:

$$B \propto \frac{I}{r}$$

Assim, você pode experimentar:

1. Use o painel lateral para selecionar a corrente (1, 2 ou 4 A). Observe como muda a intensidade das linhas;
2. Ative o movimento do campo com Play/Pause, e observe o sentido de circulação da corrente;

3. Altere o sentido das linhas de campo para cima ou para para baixo, e observe que o campo inverte o sentido da corrente;
4. O raio visual controla a posição das linhas de campo. Observe que o campo existe em todo o espaço, mas enfraquece com a distância;

3.2 Regra da mão direita

Essa é uma convenção física para facilitar a compreensão de como o sentido da corrente e do campo magnético estão interligados. Nessa representação o polegar aponta para o sentido da corrente, enquanto que os dedos com a mão fechada apontam para o sentido do campo. E isso aparece visualmente no *app* da [Figura 3.4](#).

Alguns cenários permitem explorar o aplicativo. Veja:

1. *Intensidade do campo*. Aumente a corrente e veja como o campo fica mais intenso.
2. *Inversão do campo*. Mude o sentido da corrente e observe o campo gira ao contrário.
3. *Distância ao fio*. Observe os pontos do plano, e perceba que o campo diminui com a distância.
4. *Estrutura tridimensional*. Olhe o gráfico de diferentes ângulos, clicando no menu de barras superior para ícones de rotação e translação. Observe então a tridimensionalidade do campo magnético.

MESMO PADRÃO, OUTROS MUNDOS



A mesma lógica estudada aqui reaparece em muitos outros contextos, e mesmo introduz as ideias de *eletromagnetismo*. Veja, por exemplo, num chuveiro elétrico, em que a resistência influencia diretamente o aquecimento. Em carregadores e fontes, que controlam tensão e corrente para alimentar dispositivos. Na estrutura dos eletroímãs, em que a passagem de corrente gera campo magnético utilizável. Em motores e ventiladores, cujo funcionamento depende da interação entre corrente e magnetismo. Em alto-falantes, que convertem sinais elétricos em vibração mecânica com base em forças magnéticas. E em muitas outras situações do cotidiano.

POR DENTRO DA FERRAMENTA



Os modelos desse capítulo utilizam relações fundamentais da Física, especialmente:

- a 1a. Lei de Ohm, que relaciona tensão, corrente e resistência;
- a ideia de que uma corrente elétrica gera campo magnético ao redor de um condutor;
- relações de proporcionalidade que podem ser representadas numericamente e visualizadas graficamente.

Do ponto de vista computacional, o código cria sequências de valores, aplica uma relação matemática a cada elemento e devolve essas informações na forma de gráficos interativos. Assim, você pode manipular parâmetros e perceber como pequenas mudanças numéricas produzem efeitos físicos visíveis.

Agora uma síntese da lógica utilizada para o script da [Figura 3.2](#).

A resistência é definida aqui:

```
const R = 7;
```

Os valores de tensão são gerados de 0 até 30 volts:

```
for (let V = 0; V <= 30; V += 1)
```

Para cada valor de tensão, o script calcula a corrente:

```
I_values.push(V / R);
```

Depois, o gráfico é construído com:

```
x: V_values,  
y: I_values
```

4. Ondas: do movimento ao som e à luz

O MUNDO TE CHAMA



Ondas estão por toda parte. Elas aparecem na vibração de uma corda, no som da voz, na música, na propagação da luz, nas cores observadas em um prisma, na pedra atirada na água, e nos dispositivos que usamos todos os dias para comunicação. Embora muitas vezes sejam estudadas em capítulos separados, as ondas mecânicas, sonoras e ópticas compartilham ideias estruturantes comuns: periodicidade, propagação, frequência, comprimento, amplitude, energia, velocidade, percepção e interação com meios materiais. Neste capítulo, você é convidado a explorar a ondulatória para descrever fenômenos físicos distintos.

Depois de explorar, você consegue:

- reconhecer características fundamentais de fenômenos ondulatórios;
- relacionar frequência, período, comprimento de onda e velocidade de propagação;
- diferenciar ondas mecânicas, sonoras e eletromagnéticas;
- interpretar amplitude, intensidade e energia em diferentes contextos;
- compreender qualitativamente fenômenos como interferência, reflexão, refração e difração;
- entender como a imagem é formada em lentes ópticas;
- usar simulações visuais interativas para explorar ondulatória

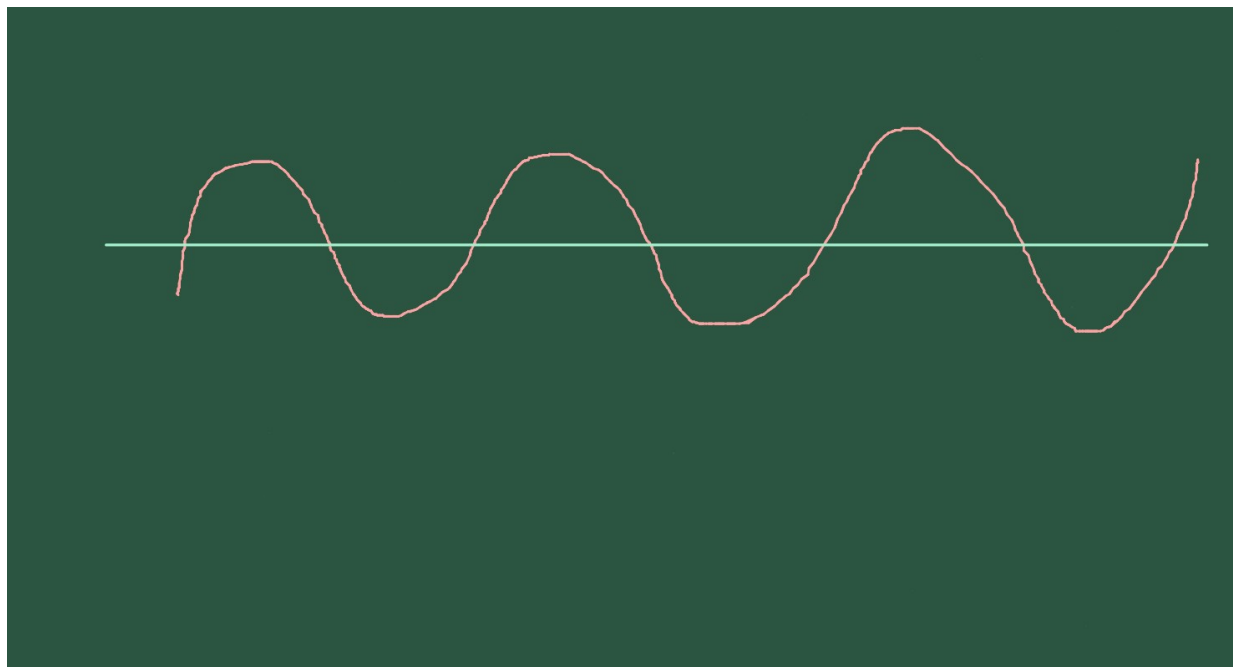


Figura 4.1: A Física que se propaga em cordas, som e luz.

MEXA ANTES DE ENTENDER



A melhor forma de começar a estudar ondulatória é observar como pequenas alterações em parâmetros simples mudam bastante o comportamento de uma onda. Para isso, experimente o objeto interativo de JSPlotly da figura abaixo.



Figura 4.2: Uma onda para começar. Clique com o botão direito neste [LINK](#) para abrir o aplicativo em nova aba.

Neste primeiro experimento, você vai observar uma *onda senoidal*, uma das formas mais importantes e comuns para representar fenômenos periódicos. Ela aparece em muitos contextos, como na vibração de uma corda, na água, na luz e radiações eletromagnéticas, bem como nos sinais elétricos e na comunicação. Basicamente, *uma onda é uma perturbação que se repete e se propaga no espaço e no tempo*.

Agite antes de usar

Sem se preocupar com o que está ocorrendo no gráfico, você pode alterar alguns parâmetros do código para ver como a onda se comporta. Por exemplo, pode alterar A , f , ou λ , onde:

O script mostra como a onda senoidal varia ao longo da posição. Para usar o *app*, basta clicar em add e testar algumas alterações no início do *script*, modificando os valores para os parâmetros amplitude, frequência, λ_{onda} , fase, e tempo. Seguem seus significados.

- A é a amplitude da onda;
- f é a frequência da onda;

- λ é o comprimento de onda (a distância entre 2 vales ou picos consecutivos);
- t é o tempo onde a onda se propaga.

Depois de alterar um valor, execute novamente o código e observe como a forma da onda muda. Você pode alternar ondas pelos botões `clean` depois `add`, ou compará-las, bastando adicionar uma nova onda simplesmente com `add`.

Como para os outros *apps* desse livro, explore alguns cenários.

1. *Altere apenas a amplitude.* Coloque `const amplitude = 2.0;`, e observe que a onda fica mais alta, embora a distância entre os picos não mude.
2. *Onda mais comprimida.* Reduza o comprimento de onda para `const lambda_onda = 1.5;`. Veja que agora aparecem mais oscilações no mesmo intervalo de posição.
3. *Onda mais esticada.* Aumente o comprimento de onda para `const lambda_onda = 8.0;`, e verifique que a onda fica mais espaçada, com menos ciclos visíveis no gráfico.
4. *Propagação no tempo.* Agora mude apenas o tempo para `*const tempo = 0.3`, e observe que a onda parece caminhar pelo espaço.
5. *Mudança de fase.* Altere a fase inicial para `const fase = Math.PI / 2;`. Esse é um truquezinho para trazer o valor π à metade. Veja que a onda muda de posição, mas mantém a mesma forma geral.

Se quiser aventurar-se um pouco mais...

1. Experimente aumentar a amplitude sem alterar a frequência, observando o que muda e o que permanece;
2. Compare ondas de mesmo comprimento de onda, mas com frequências diferentes;
3. Use valores de fase distintos para perceber deslocamentos horizontais na forma da onda;
4. Tente identificar visualmente quando uma onda parece “mais comprimida” ou “mais espaçada”;
5. Relacione o gráfico obtido com movimentos reais, como uma corda sendo agitada ou uma mola oscilando.
6. Entenda o que realmente se desloca na propagação de uma onda: matéria ou energia;
7. Procure saber o que muda quando alteramos o meio;
8. Aprenda como distinguir uma onda mais intensa de uma onda mais rápida.

Assim, utilize esse *script* de onda senoidal para explorar alguns parâmetros e compreender na próxima seção o seu significado na ondulatória: amplitude, frequência, comprimento de onda, período, velocidade, fase, e propagação.



FAZENDO APARECER

A função matemática que a define onda senoidal da [Figura 4.2](#) não é lá assim agradável. Mas você não precisa preocupar-se com ela, e sim com o efeito que ela produz no objeto interativo. A forma matemática usada é:

$$y(x, t) = A \cdot \sin \left[2\pi \left(\frac{x}{\lambda} - f t \right) + \phi \right] \quad (4.1)$$

Nessa equação:

- A é a amplitude da onda;
- f é a frequência;
- λ é o comprimento de onda (a distância entre 2 vales ou picos consecutivos);
- t é o tempo;
- ϕ é a fase inicial.

Os componentes principais são explicados abaixo.

4.1 Amplitude

A amplitude controla a altura máxima da onda. Quando a amplitude aumenta, os picos e vales ficam mais afastados da linha central.

Fisicamente, isso pode representar uma onda mais intensa, como um som mais forte, ou uma vibração mais forte numa corda.

4.2 Comprimento de onda

O comprimento de onda, representado por λ , indica a distância entre dois pontos equivalentes da onda, como dois picos ou dois vales consecutivos. Assim, quando λ aumenta, a onda fica mais “esticada”, e quando λ diminui, a onda fica mais “comprimada”.

4.3 Frequência

A frequência indica quantas oscilações ocorrem por unidade de tempo. Mesmo que o gráfico mostre o perfil espacial, a frequência aparece no cálculo da velocidade como:

$$v = \lambda f \quad (4.2)$$

Assim, para uma mesma frequência, aumentar o comprimento de onda também aumenta v , a sua velocidade de propagação.

4.4 Período

O período é o tempo necessário para uma oscilação completa. Ele é calculado por:

$$T = \frac{1}{f}(4.3)$$

Dessa forma, quanto maior a frequência, menor o período. Isso equivale dizer que ondas mais frequentes completam cada ciclo em menos tempo.

4.5 Tempo

A variável *tempo* permite observar a onda em diferentes instantes. Ao modificar esse valor, a curva parece se deslocar horizontalmente. Isso representa a propagação da onda.

4.6 Fase

A fase define o ponto onde a onda começa, podendo estar atrasada ou adiantada em relação a uma referência. Assim, a fase inicial desloca a onda sem alterar sua forma, funcionando como um “ajuste de posição” da oscilação. Nesse caso, duas ondas podem ter mesma amplitude, frequência e comprimento, mas estarem *fora de fase*. E essa é a “alma” dos circuitos elétricos nos equipamentos de tomada !

VOLTA PRO MUNDO



A importância de se estudar ondas não reside na compreensão de um conjunto de curvas, mas sim de utilizá-la como uma ferramenta para interpretação do mundo. O som de uma nota musical, o eco em um ambiente, a formação de sombras, as cores da luz e o funcionamento de instrumentos ópticos e acústicos podem ser observados a partir dos mesmos conceitos acima. Não obstante as ideias sejam as mesmas, as ondas se apresentam de formas distintas na natureza. Para compreender um pouco mais da diferença entre ondas mecânicas, sonoras e ópticas, experimente o JSPlotly a seguir.

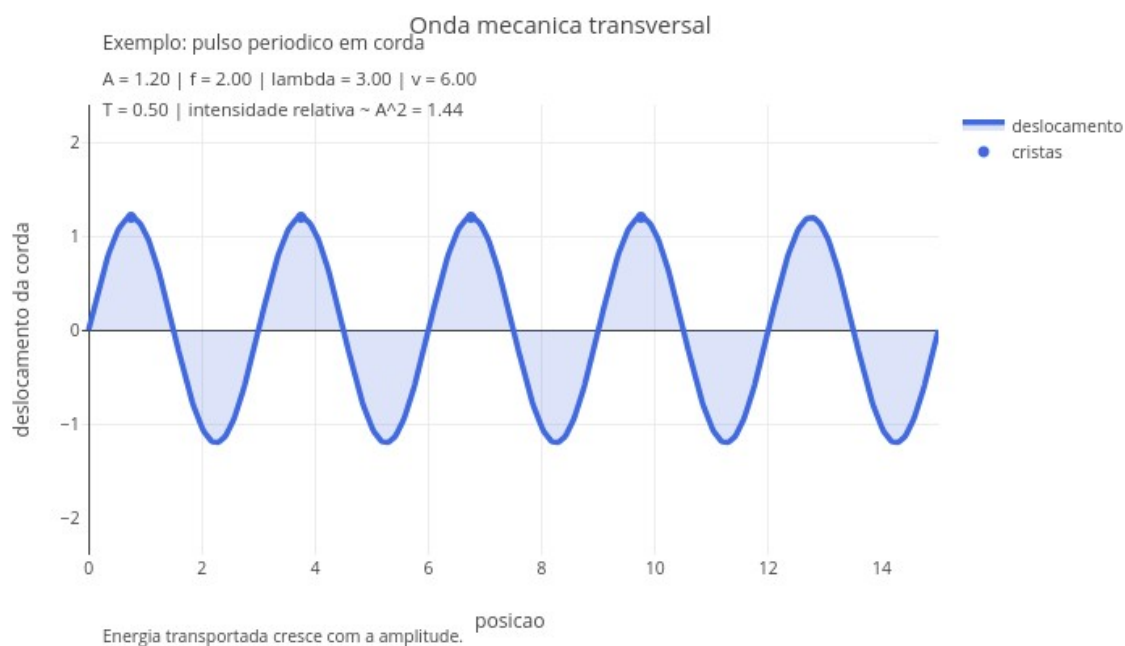


Figura 4.3: Onda mecânica, sonora e óptica: mesma linguagem, mas diferentes interpretações. Clique neste [LINK](#) para abrir o app.

O *script* é um comparador para as ondas mecânica, sonora e óptica, cada qual com sua forma de expressão. Enquanto a onda mecânica pode ser lida como deslocamento transversal, a onda sonora pode ser representada como variação de pressão, e a onda óptica associada a uma cor aproximada e intensidade luminosa.

Agite antes de usar

Comece escolhendo o tipo de onda no menu suspenso: mecânica, sonora, óptica. Em seguida, mantenha a amplitude fixa e varie apenas a frequência. Observe como a forma matemática continua reconhecível, mas o significado físico muda de acordo com o fenômeno representado. Depois, mantenha a frequência fixa e altere apenas a amplitude, discutindo como isso afeta a intensidade relativa da onda.

Após “brincar” um pouco com o *app*, tente responder às questões que seguem:

1. *Frequência e percepção.* Quando aumentamos a frequência, o que muda na interpretação física do fenômeno para o som em relação ao da luz?
2. *Intensidade e energia.* Se a amplitude dobra, por que a intensidade cresce mais do que intuitivamente poderíamos imaginar?
3. *Amplitude e frequência.* O que dizer de duas ondas distintas na amplitude embora com a mesma frequência?
4. *Luz e som.* Por que a luz pode se propagar no vácuo, mas o som não?
5. *Amplitude e energia.* Por que aumentar a amplitude não significa apenas “esticar” o gráfico, mas também alterar a energia transportada?

Perceba então que a interpretação física para o tipo de onda muda, à despeito das características comuns exibidas por seus tipos. O som, por exemplo, pode ser mais grave ou mais agudo (frequência), e possuir um volume mais alto ou mais baixo (amplitude). Já a cor, por sua vez, pode se deslocar entre o amarelo e o vermelho, por exemplo. E que tal um batimento cardíaco ? Esse manifesta-se mais rápido ou mais lento (frequência cardíaca), mais forte ou mais fraco (amplitude de pulso).

As ondas mecânicas precisam de um meio material para se propagar. Já as ondas sonoras, embora também sejam mecânicas, possuem características específicas ligadas a compressões e rarefações em meios materiais, como o ar ou a água. A luz, por sua vez, é uma onda eletromagnética e não depende de um meio material para sua propagação no vácuo. Ainda assim, todas elas podem ser analisadas por meio de grandezas como frequência, comprimento de onda, velocidade e amplitude.



E SE...

Como o que você observou acima, agora dá pra “pegar uma onda” em situações do tema.

- E se a amplitude dobrar ?
- E se aumentarmos a frequência, mas mantivermos a velocidade do meio constante ?
- E se a velocidade dobrar, mas a frequência permanecer constante ?
- E se duas ondas tiverem a mesma frequência, mas fases diferentes ?

Agora, leia essas outras aqui embaixo, e veja se consegue imaginar uma resposta à altura !

- E se duas ondas iguais se encontrarem em fase ?
- E se elas se encontrarem em oposição de fase ?
- E se duas ondas se encontrarem ?
- E se a luz passar de um meio para outro ?
- E se a frequência aumentar, mas a velocidade do meio permanecer constante ?
- E se o som encontrar uma parede distante ?
- E se a luz atravessar uma fenda estreita ?

Esses são assuntos discutidos em situações mais realistas, como interferência entre ondas, batimentos, ondas estacionárias, som, luz, espectro eletromagnético, óptica ondulatória, e que fazem parte de *fenômenos ondulatórios integrados*. Para uma abordagem mais eficiente do tema, experimente o script de JSPlotly abaixo.

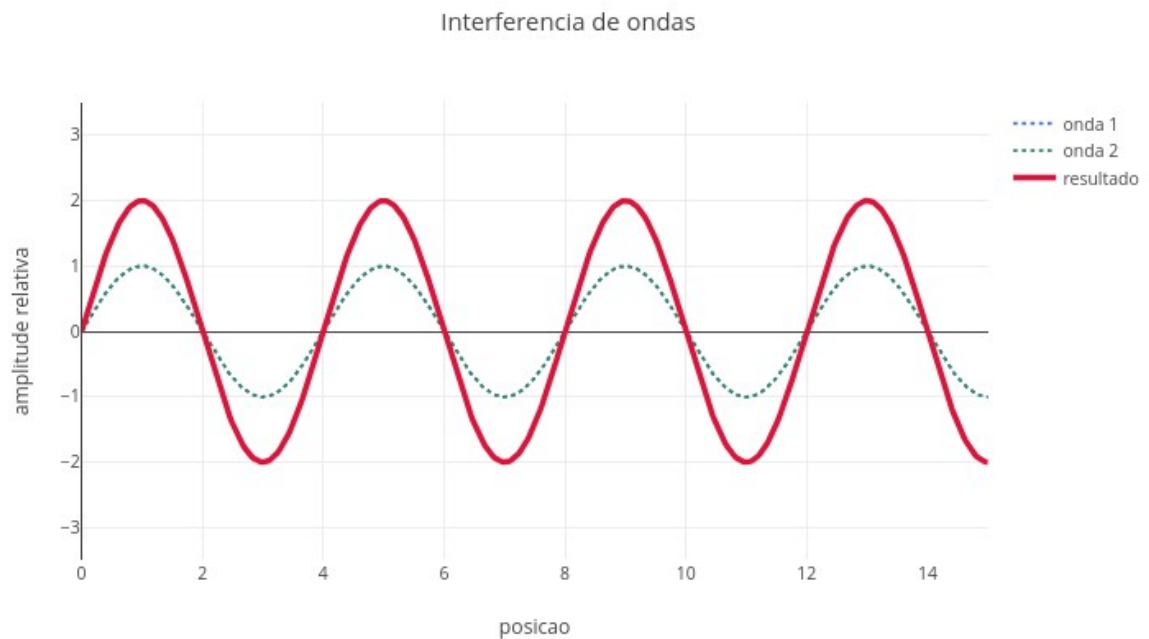


Figura 4.4: Ondas que interagem: interferência, refração, e reflexão. Clique neste [LINK](#) para abrir o app.

Agite antes de usar

No gráfico resultante, a linha azul pontilhada representa a primeira onda. A linha verde pontilhada representa a segunda onda ou a parte transmitida. E a linha vermelha representa o resultado observado, essa bem importante, porque mostra o fenômeno físico final.

Comece selecionando o fenômeno de interesse no menu suspenso (const tipoFenomeno =). Em interferência, procure comparar situações em fase e em oposição de fase:

- *interferência*, quando duas ondas se sobrepõem;
- *reflexão*, quando uma onda retorna ao encontrar uma barreira;
- *refração*, quando uma onda muda de meio e altera seu comprimento de onda.

Como o script simula a interação entre 2 ondas, seu controle é dado também por parâmetros para as duas ondas, como:

```
input_amplitude1
input_amplitude2
...
input_frequencia1
input_frequencia2
...
input_lambda1
input_lambda2
...
input_indice1
input_indice2
```

Apenas o parâmetro tempo é comum às duas ondas (`input_tempo`). Alterar o tempo desloca as ondas e mostra que o gráfico representa um instante de um processo dinâmico. Já o parâmetro `indice` refere-se ao *índice de refração*, e representa o meio atravessado pela onda. Quando a onda passa para um meio com índice maior, a velocidade relativa diminui e o comprimento de onda fica menor.

Em reflexão, observe a superposição entre a onda incidente e a onda refletida. Em refração, acompanhe a mudança do perfil ondulatório ao atravessar a fronteira entre meios. A interferência pode ser associada a pulsos em cordas, fontes sonoras ou franjas luminosas. Já a reflexão costuma servir para eco, extremidade fixa/livre, espelhos e reflexão de ondas em fronteiras. E a refração está bem envolvida com mudanças na velocidade no meio.

4.6.1 Amplitudes

```
input_amplitude1
input_amplitude2
```

Amplitudes controlam a altura das ondas. Em interferência, amplitudes maiores produzem uma soma mais intensa. Em reflexão, a amplitude da onda refletida indica quão forte é o retorno da onda. E em refração, a amplitude principal representa a intensidade da onda ao atravessar o meio.

4.6.2 Frequências

```
input_frequencia1
input_frequencia2
```

As frequências controlam o ritmo de oscilação das ondas. Na interferência, quando as frequências são iguais, as ondas mantêm uma relação mais estável. Quando as frequências são diferentes, o padrão resultante muda ao longo do tempo. Já na refração, a frequência permanece constante ao se mudar o meio, pois, fisicamente, ela é determinada exclusivamente pela fonte emissora na passagem entre os meios.

4.6.3 Comprimentos de onda

```
input_lambda1
input_lambda2
```

Os comprimentos de onda controlam o espaçamento entre cristas consecutivas. Quanto maior o comprimento de onda, mais espaçada fica a curva. Quanto menor o comprimento de onda, mais comprimida ela aparece.

4.6.4 Tempo

```
input_tempo
```

O tempo permite observar a evolução do fenômeno. Alterar o tempo desloca as ondas e mostra que o gráfico representa um instante de um processo dinâmico.

4.6.5 Índices dos meios

```
input_indice1
input_indice2
```

São usados no modo de refração. Eles representam os meios atravessados pela onda. Quando a onda passa para um meio com índice maior, a velocidade relativa diminui e o comprimento de onda fica menor, como ocorre na água em comparação com o ar.

4.7 Fenômenos de interação

Agora é possível testar alguns cenários no código para cada um dos fenômenos.

4.7.1 Interferência

Na interferência, duas ondas ocupam a mesma região do espaço ao mesmo tempo. O script calcula:

```
soma = onda1 + onda2;
```

Isso representa o princípio da superposição, quando *o deslocamento resultante é a soma dos deslocamentos das ondas individuais*. Quando as ondas estão em fase, ocorre *interferência construtiva*, e quando estão em oposição, ocorre *interferência destrutiva*.

No código, selecione *interferencia* e escolha os parâmetro a seguir.

```
input_amplitude1 = 1.0
input_amplitude2 = 1.0
input_frequencia1 = 1.2
input_frequencia2 = 1.2
input_lambda1 = 4.0
input_lambda2 = 4.0
input_tempo = 0.0
```

Com isso, espera-se que ondas se somem de forma regular. Depois altere apenas:

```
input_lambda2 = 3.0
```

Agora o padrão de soma muda, pois as cristas deixam de coincidir perfeitamente.

4.7.2 Reflexão

Na reflexão, uma onda encontra uma barreira ou mudança de meio e retorna. No *script*, isso aparece nesta linha:

```
onda2 = amplitude2 * Math.sin(2 * Math.PI * (-xi / lambda2 -
frequencia2 * tempo));
```

O sinal negativo em $-xi / \lambda_{2}$ indica que a segunda onda se propaga no sentido oposto. A soma das duas ondas pode formar regiões de *reforço*

e cancelamento, aproximando a ideia de *onda estacionária*. Assim, selecione `reflexao` e use:

```
input_amplitude1 = 1.0
input_amplitude2 = 1.0
input_frequencia1 = 1.2
input_frequencia2 = 1.2
input_lambda1 = 4.0
input_lambda2 = 4.0
input_tempo = 0.0
```

E depois altere:

```
input_tempo = 0.5
```

Observe como a superposição muda com o tempo.

4.7.3 Refração

Na refração, a onda passa de um meio para outro. O script define uma “fronteira”:

```
const fronteira = 7.5;
```

Antes da fronteira, a onda usa o comprimento de onda original. Depois da fronteira, o comprimento de onda é recalculado:

```
const velocidadeRelativa = indice1 / indice2;
const lambdaRefratada = lambda1 * velocidadeRelativa;
```

Se `indice2` for maior que `indice1`, a onda entra em um meio mais refringente (menos transparente). Nesse caso, tanto sua velocidade diminui como seu comprimento de onda diminuem.

Já a frequência é mantida constante por:

```
const frequenciaRefratada = frequencia1;
```

Isso é importante porque, na mudança de meio, a frequência da onda não muda.

Para testar o fenômeno, escolha `refracao` e faça:

```
input_indice1 = 1.0
input_indice2 = 1.5
input_lambda1 = 4.0
input_frequencia1 = 1.2
input_tempo = 0.0
```

Observe que depois da fronteira a onda fica mais comprimida. Depois teste:

```
input_indice2 = 2.0
```

E observe que a compressão aumenta.

Bom, essa seção do capítulo é para reflexão do tipo “E se...”, lembra-se? Então, que tal complementá-la com mais “ecês”... ?

- E se as ondas tiverem a mesma frequência, mas comprimentos de onda diferentes ?
- E se a onda refletida tiver menor amplitude que a onda incidente ?
- E se a onda passar para um meio de índice menor ?
- E se a frequência for mantida, mas o comprimento de onda mudar ?

Resumindo, esse terceiro *script* apresenta conceitos de interação de ondas que vão além da descrição de suas formas periódicas. Ondas podem se somar, se refletir, mudar de velocidade e reorganizar sua forma em situações específicas. Essa mesma ideia matemática também aparece em cordas vibrantes, ondas estacionárias, instrumentos musicais, eco, refração da luz, fibras ópticas, interferência em filmes finos, difração e padrões de onda, ondas em água, em sinais tecnológicos e lentes ópticas.

E por falar em *lentes ópticas*, que tal um último script que as explore só um pouquinho ?

4.8 Lentes delgadas

A ideia do JSPlotly que segue é mostrar, de forma integrada, alguns conceitos que envolvem a óptica geométrica para lentes. O script usa a equação da lente para calcular a posição da imagem e, em seguida, usa o aumento linear para determinar sua altura e orientação. Dessa forma, são abordados o tipo de lente (convergente ou divergente), bem como alguns parâmetros do objeto (posição, altura), da imagem (posição, tamanho, orientação), bem como a distância focal e raios principais.

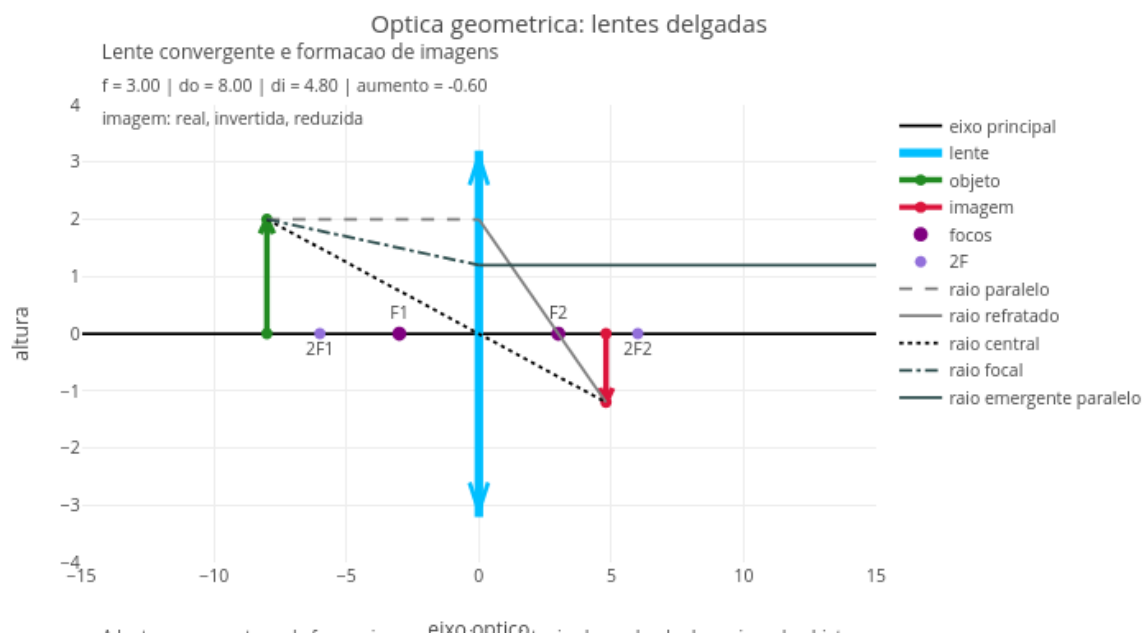


Figura 4.5: A posição da imagem é uma consequência da geometria dos raios. Clique neste [LINK](#) para abrir em nova aba.

Agite antes de usar

No script é possível alterar o tipo de lente, a distância focal, a posição do objeto e sua altura. A cada modificação, o gráfico atualiza simultaneamente a posição da imagem, sua orientação, seu tamanho relativo e os principais raios geométricos usados na construção. Uma boa estratégia é começar pela lente convergente e deslocar gradualmente o objeto ao longo do eixo óptico. Com isso, você perceberá mudanças na natureza da imagem. Em seguida, pode-se repetir a atividade com a lente divergente, observando que o padrão obtido é bastante distinto, ajudando a consolidar a interpretação física da equação das lentes.

Para tanto, segue uma exploração guiada para o script da [Figura 4.5](#).

A formação da imagem segue a relação:

$$\frac{1}{f} = \frac{1}{d_o} + \frac{1}{d_i} \quad (4.4)$$

Onde:

- f é o foco da lente;
- d_o é a distância do objeto;
- d_i é a distância da imagem.

O script então calcula automaticamente d_i , o aumento, e classifica a imagem. As demais funções abaixo completam a equação da lente.

$$\frac{1}{f} = \frac{1}{d_o} + \frac{1}{d_i} \Rightarrow d_i = \frac{1}{\frac{1}{f} - \frac{1}{d_o}} \Rightarrow m = \frac{h_i}{h_o} = -\frac{d_i}{d_o} \Rightarrow h_i = m \cdot h_o \quad (4.5)$$

Os parâmetros são explicados abaixo.

foco (f), distância do objeto (d_o), distância da imagem (d_i), aumento (m), altura do objeto (h_o), altura da imagem (h_i).

Para uso direto do *app*, edite este bloco no topo do código e execute o script (add):

```
const tipo_lente = "convergente";
const foco_valor = 3.0;
const pos_objeto = 8.0;
const altura_objeto = 2.0;
const mostrar_raios = true;
```

4.8.1 O que cada parâmetro faz

1. *Tipo de lente*. A lente pode ser convergente (forma imagem real ou virtual) ou divergente (forma somente imagem virtual).
2. *Foco (f)*. O foco controla do “poder” da lente. Um foco pequeno sugere uma lente forte, enquanto um foco grande sugere uma lente fraca.

```
const foco_valor = 3.0;
```

3. *Posição do objeto* (d_o). Variável que controla o tipo de imagem, sua posição e orientação.

```
const pos_objeto = 8.0;
```

4. *Altura do objeto*. Afeta o tamanho da imagem, mas não sua natureza.

```
const altura_objeto = 2.0;
```

5. *Raios principais*. Aqui há uma lógica chamada *booleana* em computação, e que direciona o algoritmo quando uma condição é satisfeita (*verdadeiro*) ou não (*falso*). Nesse caso, *true* mostra a construção geométrica, enquanto que *false* deixa só o objeto e imagem.

```
const mostrar_raios = true;
```

6. *Leitura do gráfico*. No script, você interpreta quem é quem pela cor das linhas e pontos no gráfico, como segue:

- *linha verde* → objeto
- *linha vermelha* → imagem
- *linha azul/laranja* → lente
- *pontos roxos* → focos (F e 2F)
- *linhas cinza* → raios principais

Com essa gama de opções, agora dá pra pensar em alguns cenários para uso do *app*:

1. *Fora do foco (imagem real)*. Veja como a imagem parece real e invertida.

```
tipo_lente = "convergente"
pos_objeto = 8.0
```

2. *No foco*. A imagem “vai para o infinito”.

```
pos_objeto = foco_valor
```

3. *Dentro do foco*. Imagem virtual e direita.

```
pos_objeto = 2.0
```

4. *Lente divergente*. Imagem sempre virtual, direita e reduzida.

```
tipo_lente = "divergente"
```

5. Verifique o que ocorre quando o objeto estiver exatamente em 2F.
6. Observe o que acontece aproximarmos o objeto lentamente do foco.
Ou se aumentarmos muito o foco (lente quase plana).

Veja que mover o objeto muda tudo. E observe também que a lente não “cria” a imagem, mas reorganiza os raios. A imagem real aparece do outro lado do objeto, enquanto a imagem virtual surge do mesmo lado do objeto. Ou seja, a lente não “faz mágica”, mas redireciona a trajetória dos raios, permitindo que a imagem surja onde os raios se encontram. E esse script também não “faz mágica”, embora apresente uma clara conexão entre geometria (construção de raios), álgebra (equação da lente), e física (formação de imagens).

MESMO PADRÃO, OUTROS MUNDOS



Essa linguagem da física ondulatória está presente em inúmeros fenômenos do cotidiano, parte deles já descritos anteriormente. Só pra não “deixar cair a peteca” das comparações, lá vai uma enxurrada de outras situações práticas que envolvem ondas.

Em uma corda de violão, o comprimento vibrante interfere diretamente na frequência produzida. Em ambientes fechados, a reflexão do som ajuda a explicar eco, reverberação e qualidade acústica, além de vibração de uma mola. Em instrumentos ópticos, como lentes e prismas, a mudança de meio altera a propagação da luz e permite formação de imagens e decomposição de cores. Em comunicações modernas, a transmissão de sinais por ondas eletromagnéticas interliga a ondulatória a tecnologias sofisticadas. Além disso, inúmeras investigações em ondulatória abrangem também o estudo e aplicação de ultrassom, sonar, fibras ópticas, cancelamento de ruído, cores estruturais e interferência em películas finas, entre muitas outras.

POR DENTRO DA FERRAMENTA



Nesse capítulo foram vistos diversos códigos para JSPlotly, cada qual para uma aresta de estudo de ondulatória. Assim, enquanto o *script* no. 1 tratou de propriedades básicas de uma onda, o *script* no. 2 abordou a comparação entre os tipos de ondas. Já o *script* no. 3 apresentou os fenômenos de interação entre ondas, enquanto que o *script* no. 4, a óptica geométrica que envolve as lentes. Como não cabe explorar todo mundo nessa seção, aqui propomos uma continuidade da explicação e pseudocódigo sugeridos para o script da [Figura 4.4](#).

Inicialmente, o script cria três listas principais:

```
const y1 = [];
const y2 = [];
const ys = [];
```

Elas guardam:

- y1: a primeira onda;
- y2: a segunda onda ou onda transmitida;
- ys: o resultado final.

O laço principal percorre várias posições:

```
for (let i = 0; i <= 500; i++) {
```

```
const xi = i * 0.03;
}
```

Para cada posição, o script calcula o valor da onda naquele ponto. Ou seja, para cada ponto do espaço, o código decide como a onda se comporta dependendo do fenômeno físico. Segue o pseudocódigo:

INÍCIO

```
DEFINIR tipoFenomeno
(interferencia, reflexao ou refracao)
```

```
DEFINIR parâmetros das ondas:
amplitude1, amplitude2
frecuencial1, frecuencia2
lambda1, lambda2
tempo
indice1, indice2
```

```
CRIAR listas:
x → posições
y1 → onda 1
y2 → onda 2
ys → resultado final
```

```
PARA cada posição xi no espaço:
```

```
  CALCULAR onda1
```

```
  SE tipoFenomeno = interferencia:
    CALCULAR onda2 normalmente
    soma = onda1 + onda2
```

```
  SENÃO SE tipoFenomeno = reflexao:
    CALCULAR onda2 invertida (propagação oposta)
    soma = onda1 + onda2
```

```
  SENÃO SE tipoFenomeno = refracao:
```

```
    DEFINIR posição da fronteira
```

```
    SE xi está antes da fronteira:
      soma = onda1
```

```
    SENÃO:
      CALCULAR nova velocidade relativa
      CALCULAR novo comprimento de onda
      MANTER frequência constante
      CALCULAR onda no novo meio
```

```
  DEFINIR onda1 = 0
  DEFINIR onda2 = soma
```

ARMAZENAR valores em y1, y2 e ys

FIM DO LOOP

DEFINIR título de acordo com o fenômeno

SE fenômeno = refração:
 ADICIONAR anotação da fronteira

MONTAR gráfico com:
 onda 1
 onda 2
 resultado final

FIM

5. Gravitação e Fluidos — quando a gravidade molda a matéria



O MUNDO TE CHAMA

Por que um astronauta parece flutuar no espaço, mesmo sob ação da gravidade ? Por que a pressão no fundo do oceano é gigantesca, a ponto de esmagar estruturas metálicas rígidas ? Por que planetas como Júpiter têm atmosferas tão extensas, e outros não ? Por que nosso ouvido parece fechar quando descemos uma serra próxima ao mar ?

A resposta que unifica essas e mais um sem-número de outras relações já está ao final da primeira pergunta aí de cima: a gravidade, que puxa objetos e organiza fluidos. A gravitação descreve como massas se atraem, e também como se comportam o ar da atmosfera, a água dos oceanos e o interior dos planetas. Este capítulo busca explorar um pouco como a gravidade atua sobre objetos isolados e também em sistemas contínuos, tais como líquidos e gases, revelando padrões que se repetem desde o fundo dos oceanos até as atmosferas planetárias.

Ao final, você será capaz de:

- compreender a gravidade como uma interação capaz de organizar movimentos e estruturas no Universo;
- relacionar gravidade e pressão em líquidos e gases;
- interpretar a pressão hidrostática como resultado do peso de um fluido;
- compreender como atmosferas se formam e variam em diferentes mundos;
- investigar como densidade e gravidade influenciam flutuação e empuxo;
- comparar fenômenos físicos em diferentes ambientes planetários;
- explorar visualizações interativas por códigos de programação.

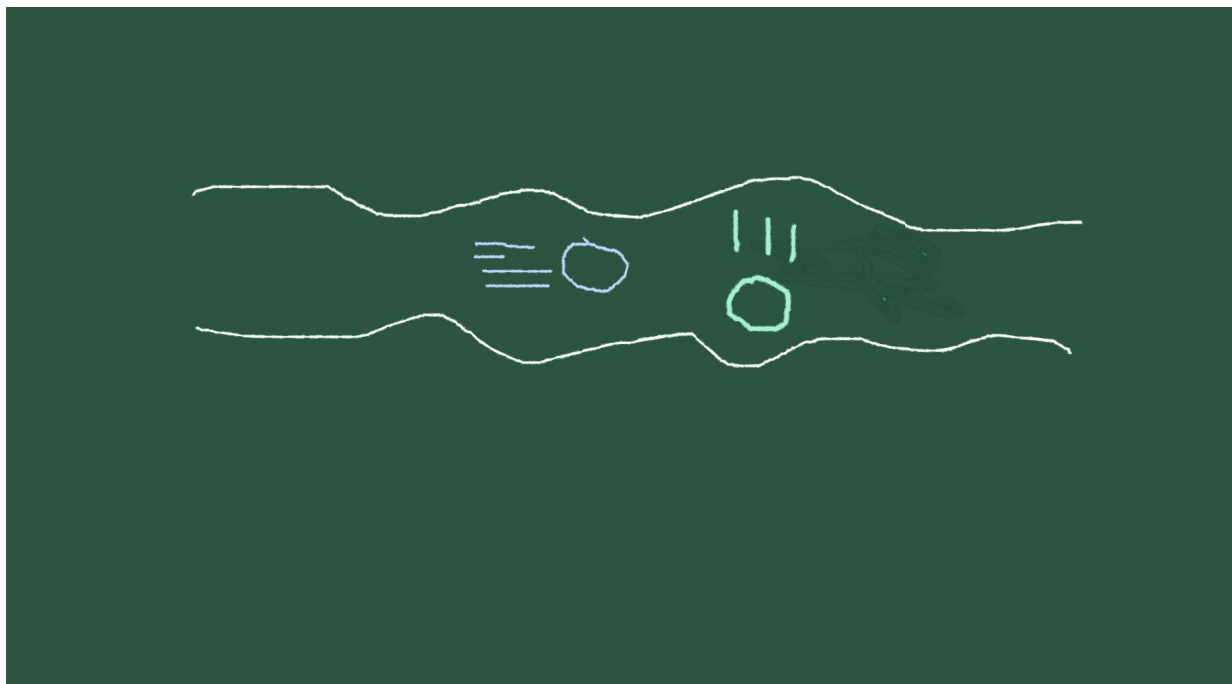


Figura 5.1: A gravidade descreve massas que se atraem e determina a distribuição de pressão e densidade nos fluidos.

MEXA ANTES DE ENTENDER



Antes dos conceitos de praxe, um JSPlotly inicial para o capítulo, como já é de praxe !

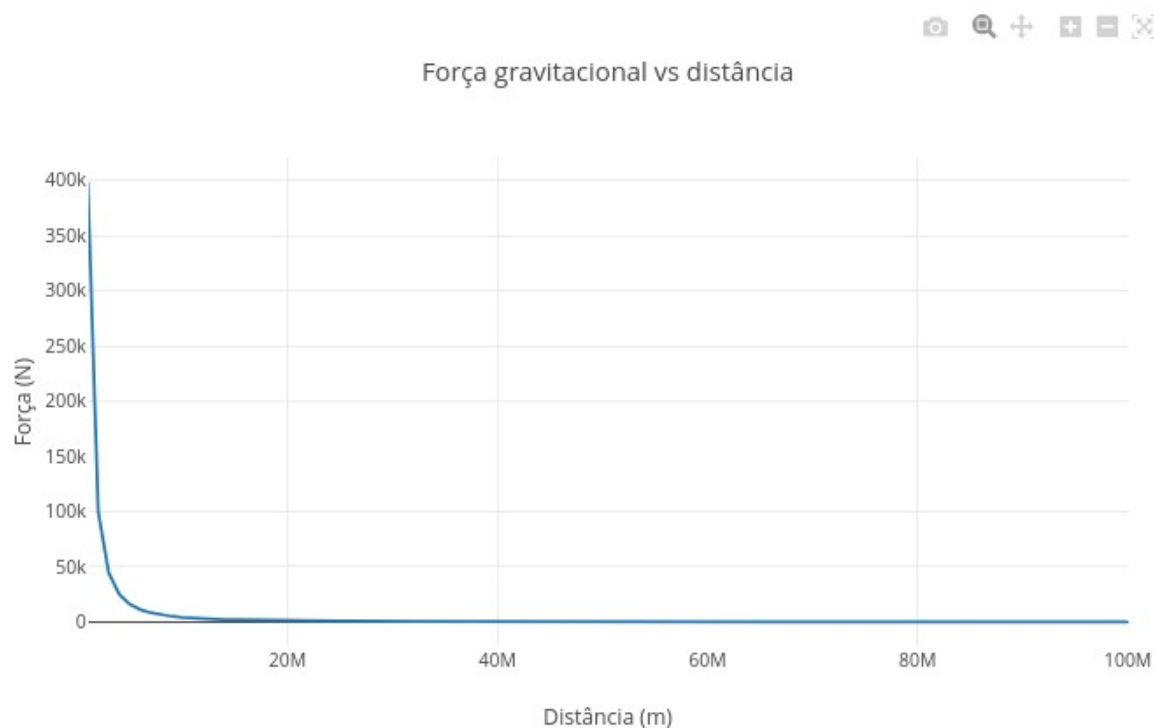


Figura 5.2: A gravidade depende de massa e distância. Clique com o botão direito neste [LINK](#) e

abra o app em nova aba.

Esse *app* mostra como a força gravitacional entre dois corpos diminui quando a distância entre eles aumenta. A ideia central é simples: as massas se atraem, mas essa atração se enfraquece rapidamente com a distância.

A equação usada é:

$$F = G \frac{m_1 m_2}{r^2} \quad (5.1)$$

Essa é a chamada Lei da Gravitação Universal. Nela, m_1 e m_2 são as massas dos corpos, r é a distância entre eles e G é a constante gravitacional.

Agite antes de usar

Após carregar o aplicativo, observe o gráfico da força gravitacional em função da distância. O trecho do código que determina essa força é:

```
F.push(G * m1 * m2 / (dist * dist));
```

Ele calcula a força para várias distâncias, gerando a curva mostrada no gráfico. Os parâmetros pra você mexer estão aqui:

```
let m1 = 5.97e24; // massa da Terra
let m2 = 1000;    // massa do objeto
let escalaDist = 1e6;
```

Alguns cenários se mostram interessantes para você explorar o aplicativo. Como em outros scripts, você pode apagar um cenário com `clean` antes do outro, ou não, e nesse caso sobrepondo curvas para comparação entre cenários.

1. Objeto próximo da Terra.

```
let m1 = 5.97e24;
let m2 = 1000;
let escalaDist = 1e6;
```

Observe que a força é maior nas menores distâncias e cai rapidamente à medida que r aumenta. Isso ocorre porque a gravidade não desaparece de repente, mas diminui continuamente com a distância.

2. *Objeto mais massivo.* Aumente em 10 vezes a massa do objeto. Veja que a força aumenta em relação ao cenário anterior. Isso acontece por que, se você dobrar ou multiplicar a massa, a força gravitacional aumenta na mesma proporção.

3. *Planeta menos massivo* Que tal simular a força de atração em Marte ?!

```
let m1 = 6.4e23; // massa aproximada de Marte, em kg
let m2 = 1000;
```

Perceba que a força fica menor do que no caso da Terra. Ou seja, mundos com menor massa exercem menor atração gravitacional sobre o mesmo objeto.

1. *Distâncias maiores.* Agora altere a distância entre os objetos:

```
let escalaDist = 5e6;
```

Veja que a curva fica mais baixa, porque as distâncias consideradas são maiores. Ou seja, a distância tem efeito muito forte, pois aparece ao quadrado no denominador ([Equação 5.1](#)).

Esse cenários mostram que a gravidade depende de duas coisas: massa e distância. Massas maiores aumentam a atração, e distâncias maiores reduzem a atração — e reduzem muito, porque a relação é inversamente proporcional ao quadrado da distância. Em outras palavras, aumentar a massa fortalece a gravidade, mas aumentar a distância a enfraquece. E a distância pesa muito nessa conta.

Até aqui, percebe-se que a gravidade depende da massas de um corpo, o que se reflete em seu peso, ou força peso:

$$P = m * g \quad (5.2)$$

Ocorre que, se o peso for de um fluido, como a água, essa força acaba por gerar pressão. E a pressão explica fenômenos como atmosfera, hidrostática, empuxo e peso aparente. Temos então a gravitação sendo tratada também dentro da água, do ar e dos corpos que flutuam, e não ficando restrita somente ao espaço.

Isso é assunto pro próximo *app* de JSPlotly, pressão em fluidos.

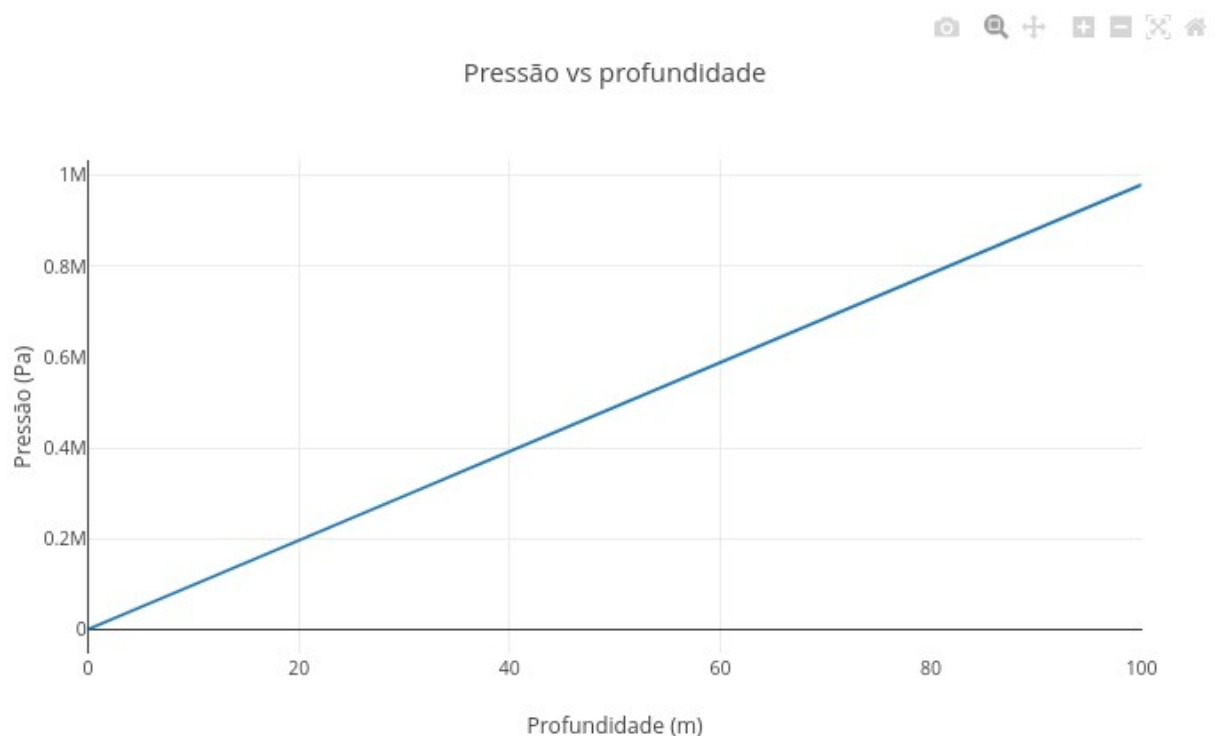


Figura 5.3: A pressão hidrostática surge do peso do fluido. Clique com o botão direito neste [LINK](#) e abra o aplicativo em nova aba.



FAZENDO APARECER

O script da [Figura 5.3](#) mostra como a pressão aumenta à medida que mergulhamos mais profundamente em um fluido. A ideia é que cada camada de líquido exerce peso sobre as camadas abaixo.

A equação utilizada é:

$$P = \rho g h \quad (5.3)$$

Onde ρ é a densidade do fluido, g é a gravidade, e h é a profundidade.

Agite antes de usar:

Você pode editar principalmente esses valores no código:

```
let rho = 1000;
let g = 9.8;
let hmax = 100;
```

Depois clique em add e observe como a pressão varia com a profundidade. O cálculo principal aparece neste trecho:

```
P.push(rho * g * depth);
```

O script então calcula a pressão em diferentes profundidades e gera a curva correspondente. “Facinho”, não ? Alguns cenários pra você explorar.

1. *Água na Terra.*

```
let rho = 1000;
let g = 9.8;
```

Observe que a pressão cresce linearmente com a profundidade, ou seja: quanto mais fundo, maior o peso da coluna de líquido acima.

2. *Fluido mais denso.* Aumente para “ $\rho = 13600$ ”, um valor próximo ao do mercúrio. Veja que a curva cresce muito mais rapidamente. Isso ocorre porque líquidos mais densos exercem maior pressão para uma mesma profundidade.

3. *Gravidade menor.* Agora altere a gravidade para um valor próximo ao da Lua.

```
let g = 1.6;
```

Observe que pressão aumenta mais lentamente, porque a pressão hidrostática depende diretamente da gravidade e, nesse caso, do mundo considerado.

4. *Grandes profundidades.* Coloque a altura em:

```
let hmax = 500;
```

O gráfico agora se estende para profundidades maiores, porque, mesmo mantendo o mesmo líquido, as pressões são enormes.

No script, a profundidade é gerada com espaçamentos iguais:

```
let depth = i * (hmax / 100);
```

Ele então calcula a pressão correspondente para cada ponto, o que resulta num gráfico praticamente linear, porque:

$$P \propto h$$

Ou seja, dobrar a profundidade dobra a pressão. Além disso, quanto maior a densidade do líquido, ou maior a gravidade, maior será também a pressão exercida. A gravidade estudada no script não atua apenas em planetas e órbitas, mas também nos líquidos, comprimindo as camadas inferiores, e também na atmosfera como um todo.

Desse modo, algumas coisas passam a fazer mais sentido, embora não sejam nada conectadas. Por exemplo, mergulhadores sofrem maior pressão quanto mais fundo descem, planetas com maior gravidade comprimem suas atmosferas, e estrelas são mantidas estáveis por equilíbrio entre gravidade e pressão. Resumindo o “babado”, todos os fluidos pesam, gerando uma pressão que aumenta com a profundidade. Inclusive o ar, pelas camadas da atmosfera.

Que tal compreender um pouco mais sobre a influência da gravidade nessa última, a atmosfera ? Para isso, veja se o *app* da [Figura 5.4](#) sobre pressão atmosférica pode te ajudar.

VOLTA PRO MUNDO



O ar também possui massa. A atmosfera terrestre forma uma enorme coluna de gás ao redor do planeta, comprimida pela gravidade. Desse jeito, quanto maior a altitude, menor a quantidade de ar acima de nós, e menor a pressão atmosférica. Isso pode ser evidenciado com o script da [Figura 5.4](#). O código mostra como a pressão atmosférica diminui com a altitude. Diferentemente da pressão hidrostática simples, que cresce linearmente, a atmosfera apresenta uma queda aproximadamente exponencial.

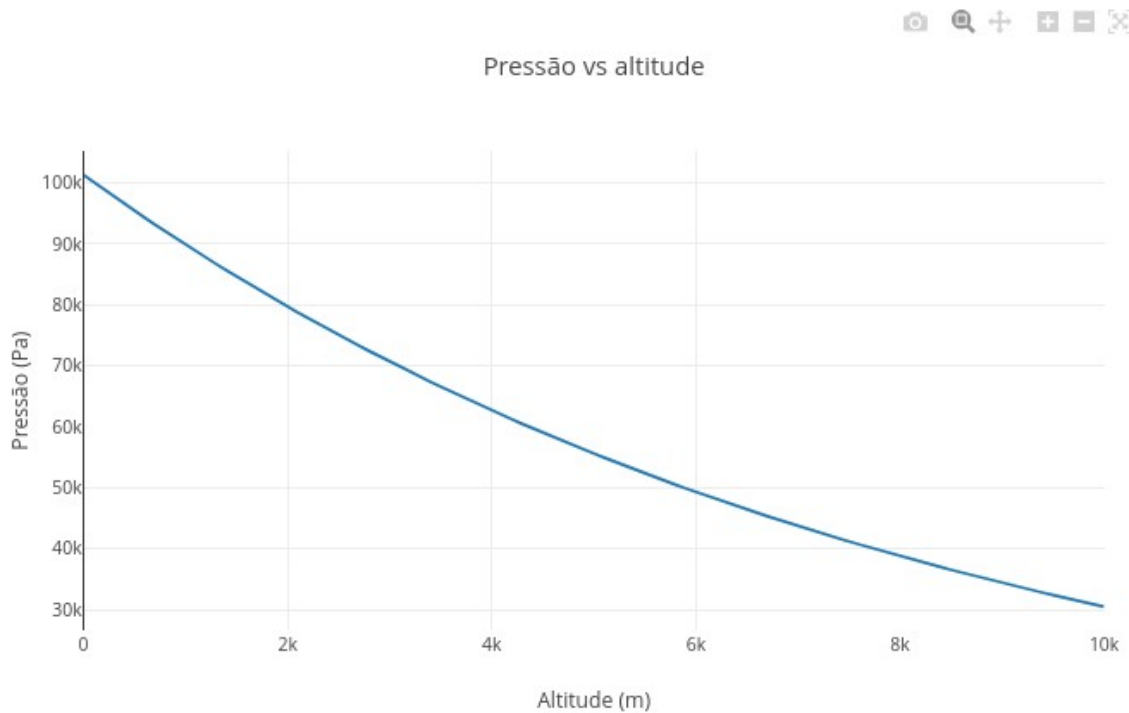


Figura 5.4: A pressão atmosférica existe porque o ar possui massa e sofre ação da gravidade. Clique neste [LINK](#) para abrir o app.

A equação utilizada no código é:

$$P = P_0 e^{-kh} \quad (5.4)$$

Nessa equação:

- P_0 é a pressão ao nível do solo;
- h é a altitude;
- k controla a rapidez da queda da pressão.

Agite antes de usar

Você pode editar o trecho abaixo, clicar em add, e observar como a pressão varia com a altitude.

```
const P0 = 101325;
const k = 0.00012;
```

O cálculo principal para isso aparece neste trecho:

```
P.push(P0 * Math.exp(-k * alt));
```

Essa função exponencial (`Math.exp`) faz com que a pressão diminua rapidamente nas primeiras altitudes, e mais lentamente em grandes alturas. Isso dá pra bolar alguns cenários interessantes, veja:

1. *Atmosfera terrestre padrão.*

```
const P0 = 101325;
const k = 0.00012;
```

O gráfico resultante mostra que a pressão cai gradualmente com a altitude, porque há menos ar acima de nós exercendo peso.

2. *Atmosfera mais densa.* Agora a pressão dobra ! E é isso o que ocorre com outros planetas, onde atmosferas mais densas podem apresentar pressões muito maiores ao nível do solo.

```
const P0 = 202650;
```

3. *Queda mais rápida da pressão.*

```
const k = 0.00025;
```

Veja que a curva despenca mais rapidamente. Isso se dá porque valores maiores de k representam atmosferas que perdem pressão mais rapidamente com a altitude.

4. *Atmosfera mais extensa.* Aqui você deve perceber que a pressão diminui mais lentamente, já que atmosferas mais extensas conseguem manter pressão significativa mesmo em grandes altitudes.

```
const k = 0.00005;
```

O script calcula a pressão em diferentes altitudes e depois aplica o modelo exponencial para estimar a pressão atmosférica correspondente. A diferença em relação ao que vimos no script da [Figura 5.3](#) é simples: na água, a densidade varia pouco, mas na atmosfera, o próprio ar vai ficando menos denso com a altitude. E é por isso que o perfil vai ficando curvilíneo.

Experimente “brincar” um pouco mais com esse *app*, para verificar diferenças sobrepondo os gráficos com o botão add. Compare, por exemplo, atmosferas densas com rarefeitas, quedas rápidas com quedas lentas, bem como diferentes condições planetárias.

Em síntese, o ar também pesa, comprimido pela gravidade, embora subir signifique atravessar camadas da atmosfera cada vez menos densas. Juntando o que você já experimentou nas seções anteriores do capítulo, perceba agora que a gravidade aparece em três escalas diferentes: entre corpos celestes, em líquidos e na atmosfera.

Imagine poder juntar tudo isso, ou melhor, corpos celestes e líquidos ! De repente, um mesmo oceano se comportando de formas muito diferentes dependendo do mundo em que se está. Em planetas ou luas com gravidade maior, a pressão aumenta mais rapidamente com a profundidade. Em mundos com gravidade menor, esse aumento acontece de forma mais suave. E adivinha ?! Que tal exercitar isso num próximo script de JSPlotly ?

E SE...



Essa seção tem sido dedicada a levantar hipóteses, realistas ou não, e em contextos variados. Mas antes que nos defrontemos com as divagações de praxe, vamos explorar um pouco a ideia mencionada acima, a de que gravidade de diferentes mundos altera a pressão hidrostática, junto ao app da [Figura 5.5](#).

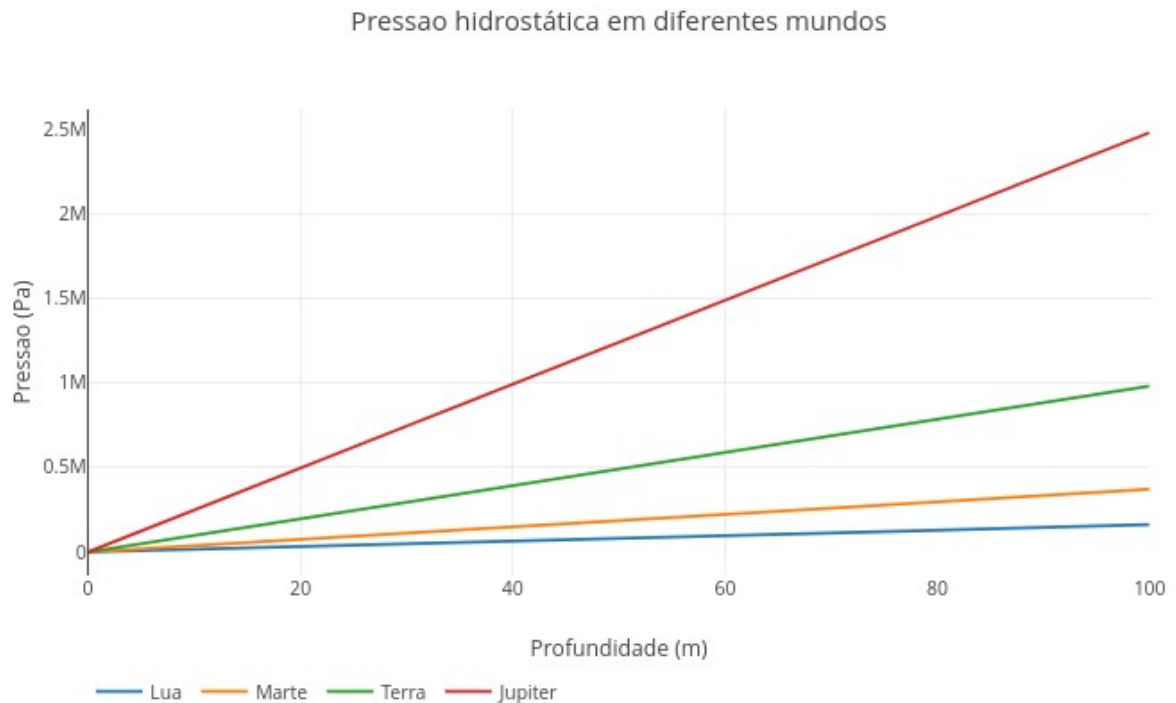


Figura 5.5: A pressão hidrostática depende não apenas do fluido, mas também do planeta ou lua em que o fluido se encontra. Clique neste [LINK](#) para abrir o aplicativo.

Esse script compara como a pressão hidrostática varia em diferentes corpos celestes. Embora o fluido possa ser o mesmo, a gravidade de cada mundo altera significativamente a pressão exercida em grandes profundidades. A equação utilizada continua sendo:

$$P = \rho g h$$

Mas agora o valor de g muda para cada mundo. Assim, edite principalmente o início do código:

```
const rho = 1000;
const hmax = 100;
```

O script então compara automaticamente a pressão prevista na Lua, em Marte, na Terra, e em Júpiter. E cada curva mostra como a pressão cresce com a profundidade naquele mundo. As gravidades usadas aparecem neste trecho:

```
const mundos = {
  "Lua": 1.62,
  "Marte": 3.71,
```

```
"Terra": 9.81,  
"Júpiter": 24.79  
};
```

Veja que todas as curvas aumentam linearmente, mas com inclinações diferentes. A curva de Júpiter cresce muito mais rapidamente do que a da Terra, enquanto a da Lua cresce lentamente. Isso acontece porque:

$$P \propto g$$

Ou seja, a gravidade controla o quanto o fluido “pesa”. Vamos experimentar alguns cenários ?

1. *Água em diferentes mundos.*

```
const rho = 1000;
```

A comparação padrão utiliza um fluido semelhante à água. Perceba que, mesmo usando o mesmo líquido, a pressão muda drasticamente entre os mundos.

2. *Fluido mais denso.* Altere a densidade acima para um valor próximo ao do mercúrio (“const rho = 13600”). Agora, todas as curvas ficam mais inclinadas, porque a densidade e gravidade atuam juntas no aumento da pressão.
3. *Grandes profundidades.* Agora mude:

```
const hmax = 500;
```

Veja que as diferenças entre os mundos ficam ainda mais evidentes. De fato, em grandes profundidades, pequenas diferenças de gravidade geram enormes diferenças de pressão (acho que o Tio Ben, da saga Homem-Aranha, disse algo parecido uma vez).

4. *Comparando Lua e Júpiter.* Note que, na Lua, o crescimento é lento, enquanto que em Júpiter o crescimento é muito rápido. Traduzindo... mergulhar em um oceano hipotético de Júpiter seria uma experiência realmente “esmagadora”.
5. *Comparando com qualquer planeta ou satélite natural.* Aqui é pura diversão. Basta você substituir um planeta e o seu valor no código por outro diferente, como Saturno ou Vênus, comparando-os em seguida com os demais.

Enfim, a ideia até aqui é mostrar que líquidos possuem um peso que depende da gravidade local, o que faz com que mundos diferentes gerem pressões também diferentes.

E isso ocorre de forma análoga em condições de atmosfera variável. Então, que tal agora, de um jeito mais breve, verificar o efeito da gravidade em atmosferas comparadas a partir de um menu de opções ?

Para isso, teste o *app* da [Figura 5.6](#). Diferente dos demais até então, agora você não precisará editar nada, apenas observar o que acontece quando seleciona uma opção diferente. O script compara modelos atmosféricos

usando um menu interativo do próprio gráfico, e cada qual apresentando uma velocidade diferente de queda da pressão com a altitude.

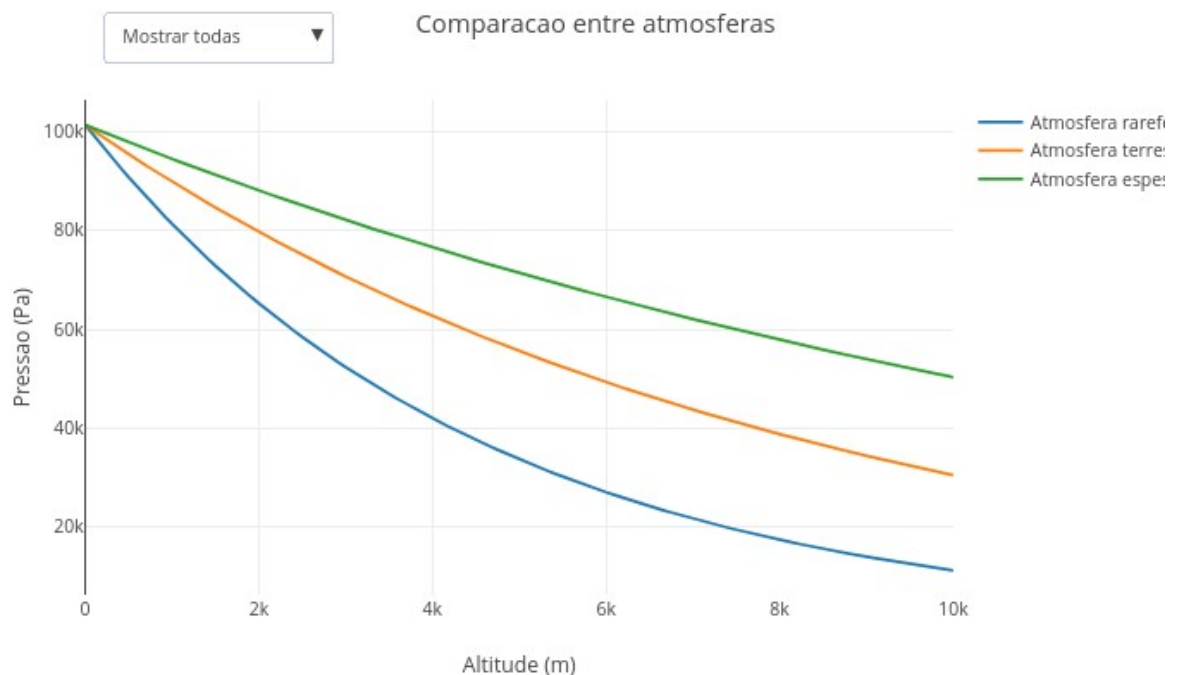


Figura 5.6: A gravidade e a estrutura da atmosfera determinam como a pressão varia com a altitude. Clique neste [LINK](#) para abrir o app .

Agite antes de usar

Nesse app, basta você utilizar o menu suspenso no topo do gráfico para alternar entre atmosfera rarefeita, atmosfera terrestre, ou atmosfera espessa. Depois, utilize a opção *Mostrar todas*, para comparar simultaneamente os diferentes comportamentos. O resultado é apresentado como um gráfico com *decaimento exponencial*, como mostra a [Figura 5.6](#). Esse decaimento decorre do modelo da [Equação 5.5](#) aplicado ao código:

$$P = P_0 e^{-kh} \quad (5.5)$$

Observe que as curvas apresentam quedas diferentes de pressão com a altitude. Assim, em atmosferas rarefeitas, ocorre uma perda rápida de pressão, pois atmosferas pouco densas oferecem menor proteção e menor retenção de gases. Por outro lado, em atmosferas espessas, essa queda é mais lenta, permanecendo a pressão elevada mesmo em grandes altitudes. Isso ocorre porque atmosferas densas conseguem sustentar pressão significativa por longas distâncias.

O comportamento intermediário, por sua vez, pode ser observado junto à atmosfera terrestre. Se você clicar em “Mostrar todas”, verá que pequenas mudanças no comportamento atmosférico geram diferenças planetárias enormes. Portanto, diferentes mundos podem apresentar

atmosferas extremamente distintas, mesmo obedecendo a princípios físicos semelhantes. Diferentes condições produzem atmosferas também diferentes. E por consequência, a ação da gravidade sobre essas atmosferas de massas distintas faz com que se mantenham próximas aos planetas.

Agora sim, a gente tá apto a “viajar na maionese” como se falava nos saudosos anos 80’.

- E se estivéssemos na Lua ? Como líquidos e atmosferas se comportariam em uma gravidade tão baixa ?
- E se a gravidade da Terra fosse o dobro da atual ? Nosso corpo suportaria facilmente a nova pressão ?
- E se o fluido fosse muito menos denso que a água ? A pressão aumentaria lentamente ou rapidamente com a profundidade ?
- E se mergulhássemos em um oceano de mercúrio ? O que aconteceria com a pressão mesmo em pequenas profundidades ?
- E se estivéssemos em Marte ? Como seriam a atmosfera, a pressão e a flutuação nesse ambiente ?
- E se Júpiter tivesse uma superfície líquida navegável ? O que aconteceria com um mergulhador em uma gravidade tão intensa ?
- E se a atmosfera terrestre fosse muito mais espessa ? Como mudariam respiração, clima e pressão atmosférica ?
- E se a Terra perdesse parte de sua atmosfera ? O que aconteceria com a pressão ao nível do solo ?
- E se um planeta tivesse gravidade muito baixa, mas atmosfera extremamente densa ?
- E se fosse possível construir submarinos para oceanos extraterrestres ? Quais mundos seriam mais perigosos para explorar ?
- E se a gravidade variasse continuamente enquanto mergulhamos ? Como ficaria a pressão hidrostática ?
- E se comparássemos um oceano terrestre, um oceano marciano e um oceano hipotético em Europa, a lua de Júpiter ?



Bom, já estamos navegando em “*outros mundos*” desde o início do capítulo. Mas com o pé no chão, digo, na Terra, a gente pode também elencar diversas situações em que as *relações de gravidade, fluido e atmosfera* convivem conosco cotidianamente. E não são poucas ! A percepção do aumento de pressão com a profundidade, por exemplo, é clássico pra qualquer um que já mergulhou em piscinas ou oceanos. Aquele negócio do ouvido “*tampar*” em mergulhos e viagens devido à variações da pressão externa, também. Quem já observou a diferença de pressão em fluidos em seringas e canudos ? Uma simples panela de pressão também nos revela uma alteração da temperatura de ebulição pela pressão.

Além disso, navios flutuam graças ao empuxo em fluidos, enquanto submarinos controlam internamente densidade e flutuação. As marés oceânicas decorrem da interação gravitacional entre Terra, Lua e Sol. Indo para o céu, o voo de aviões decorre da interação entre atmosfera, pressão e sustentação. A sensação de cansaço ao escalar montanhas também resulta da menor pressão atmosférica e da menor disponibilidade de oxigênio. Indo para o chão, a chuva “*nada mais é*” do que o movimento de massas de ar sob ação da gravidade (circulação atmosférica). Já as barragens exibem pressão hidrostática crescente com a profundidade. E indo para nós mesmos, a circulação sanguínea resulta de um equilíbrio entre pressão e movimento de fluidos no corpo humano. E a simples sensação de “*peso*” ao carregar objetos nada mais é do que a ação gravitacional da Terra sobre os corpos.

POR DENTRO DA FERRAMENTA



A seção desse capítulo poderia ter vindo num robusto plural, já que percorremos os conceitos com auxílio de 6 aplicativos de JSPlotly:

Script 1 - gravitação básica; Script 2 - pressão hidrostática; Script 3 - atmosfera simplificada; Script 4 - comparação de planetas; Script 5 - atmosferas múltiplas; Script 6 - empuxo, flutuação e “peso aparente”

Junte tudo isso no mesmo caldeirão e você terá: gravidade gera peso que gera pressão, por sua vez moldando atmosferas que alteram flutuação, e essa modificando o peso aparente.

Então “*sapequemos*” apenas um pseudocódigo, o de pressão hidrostática em diferentes mundos !

INICIAR parâmetros principais

```
definir densidade do fluido
definir profundidade máxima
```

```
criar tabela de mundos:
```

```
Lua →  $g = 1,62$ 
```

```
Marte → g = 3,71  
Terra → g = 9,81  
Júpiter → g = 24,79
```

```
gerar lista de profundidades
```

```
    PARA vários valores de profundidade  
        calcular profundidade atual  
        armazenar valor  
FIM
```

```
criar conjunto de curvas
```

```
PARA cada mundo
```

```
    obter gravidade do mundo
```

```
    criar lista de pressões
```

```
    PARA cada profundidade
```

```
        calcular pressão hidrostática:
```

$$P = \rho \times g \times h$$

```
        armazenar pressão
```

```
    FIM
```

```
    criar curva do mundo:  
        profundidade × pressão
```

```
    adicionar curva ao gráfico
```

```
FIM
```

```
configurar gráfico:
```

```
    título  
    eixo da profundidade  
    eixo da pressão  
    legenda comparativa
```

```
mostrar resultado final
```

6. Transformações Invisíveis: Termodinâmica, Gases e Soluções



O MUNDO TE CHAMA

Você já percebeu que uma bomba de bicicleta esquenta quando o ar é comprimido, ou que um spray aerossol fica gelado enquanto é usado, e também que algumas substâncias parecem “desaparecer” na água, alterando a temperatura da mistura ?

Mágica ?! Não, Termodinâmica. Um conceito que se traduz como energia se transformando o tempo todo. A Termodinâmica estuda como calor, pressão, volume, temperatura e matéria se relacionam em sistemas físicos e químicos. Ela aparece em motores, geladeiras, pneus, atmosferas planetárias, metabolismo celular, cozimento de alimentos, mudanças climáticas e até no funcionamento do próprio corpo humano.

Neste capítulo, você irá explorar esses mundos invisíveis. Como a pressão e volume se relacionam em gases, como partículas reais diferem de modelos ideais, como diferentes condições alteram cenários termodinâmicos, e como processos físicos podem ser visualizados em movimento. E isso lhe permitirá enxergar a matéria como um sistema dinâmico de partículas, colisões, energia e transformações.

Depois de explorar, você consegue:

- reconhecer como pressão, volume e temperatura se relacionam em sistemas gasosos;
- interpretar a equação dos gases ideais como uma forma condensada de várias regularidades experimentais;
- distinguir calor, temperatura, energia interna e trabalho sem reduzi-los a sinônimos;
- perceber que um sistema pode trocar energia com o meio de modos diferentes;
- compreender que soluções envolvem interações entre partículas e, por isso, nem sempre seguem comportamentos ideais;
- comparar o comportamento mais livre de um gás com o comportamento mais interativo de uma solução;
- interpretar qualitativamente por que algumas dissoluções aquecem o meio, enquanto outras o resfriam;
- utilizar simulações computacionais para visualização interativa dos conceitos abordados.

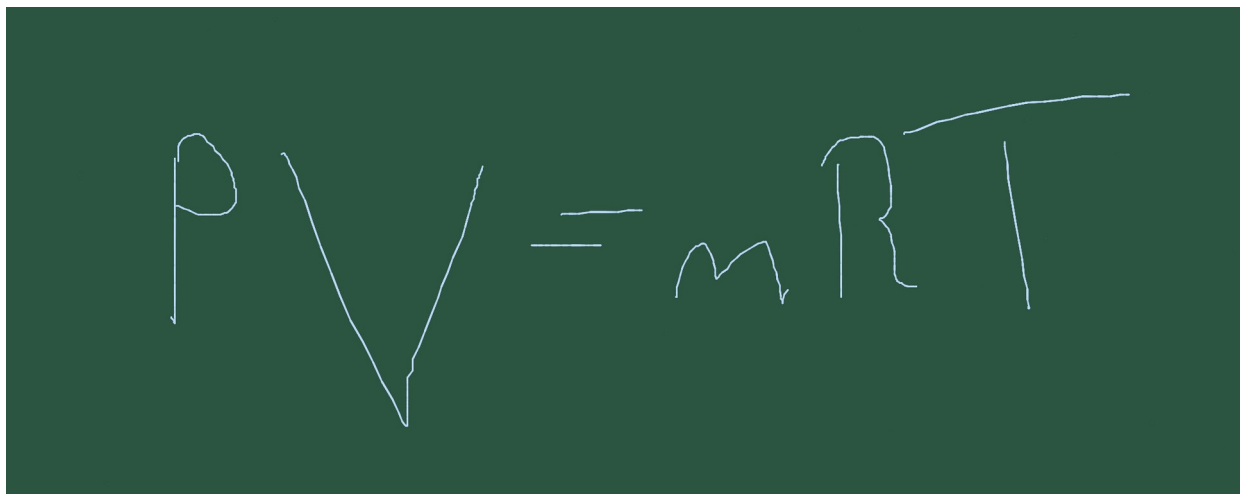


Figura 6.1: Energia, matéria e movimento estão constantemente se transformando.

MEXA ANTES DE ENTENDER



O *app* de JSPlotly abaixo mostra a relação entre pressão e volume de um gás, uma das mais importantes da Termodinâmica.

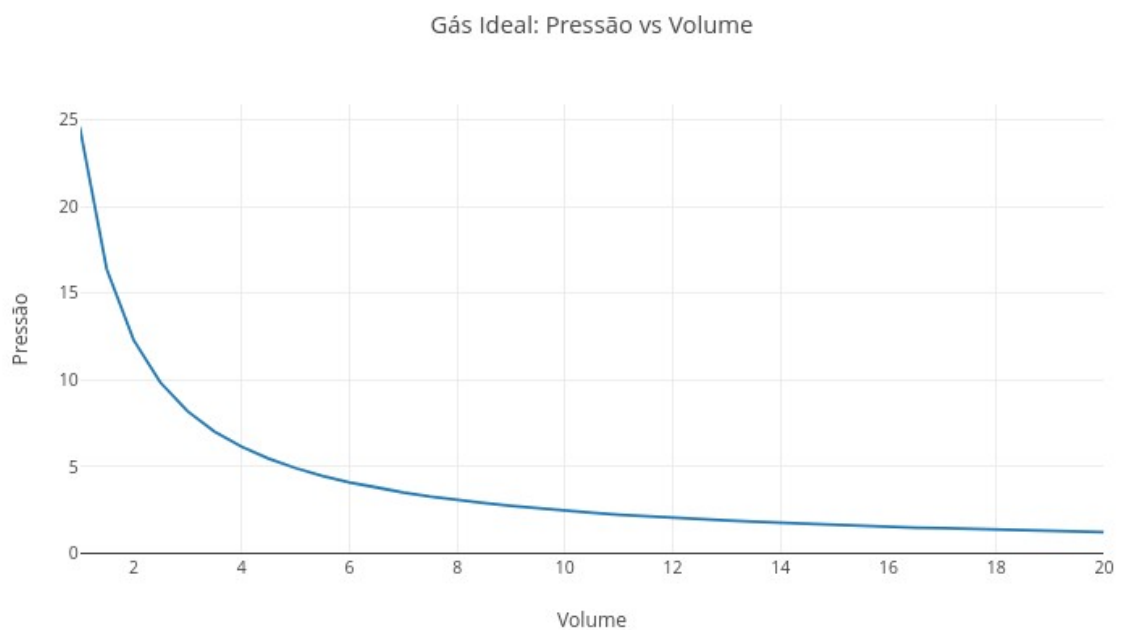


Figura 6.2: Gás ideal: quando apertar o espaço muda a pressão. Clique neste [LINK](#) para abrir o aplicativo em nova aba.

Ao alterar o espaço disponível para as partículas, a pressão muda automaticamente, o que revela no gráfico que pressões maiores levam a volumes menores. Essa é a famosa relação inversa entre pressão e volume descrita pela Lei de Boyle:

$$P_1 * V_1 = P_2 * V_2 \quad (6.1)$$

Agite antes de usar

Execute o script original e depois o altere para diferentes temperaturas T (200, 500, 800, por ex). Observe as diferenças.

```
const T = 300;
const n = 1;
```

Altere agora a quantidade de matéria n e veja como a pressão responde (0.5, 2, por ex).

O script usa a Equação Geral dos Gases Ideais, um conceito teórico aplicado a vários temas e áreas, desde a Física clássica até o metabolismo celular:

$$PV = nRT \quad (6.2)$$

Para calcular a pressão, basta reorganizar a equação:

$$P = \frac{nRT}{V} \quad (6.3)$$

O gráfico resultante forma uma hipérbole, mostrando uma relação inversa entre as variáveis. Isso significa que a pressão aumenta quando o volume diminui, e vice-versa. E também informa que a temperatura e a quantidade de gás influenciam positivamente a pressão.

Para o *app*, o script percorre diferentes valores de volume:

```
for (let v = 1; v <= 20; v += 0.5)
```

Para cada valor de volume, calcula-se a pressão:

```
P.push((n * R * T) / v);
```

Os valores são armazenados em listas:

```
V.push(v);
P.push(...)
```

A lógica de programação envolve a escolha de um volume, o cálculo da pressão correspondente, o armazenamento do resultado, um laço de repetição (*loop*) para criar novos pontos do gráfico, e o desenho final da curva.

Você pode seguir esse pequeno tutorial para utilizar o aplicativo.

1. Rode o script com os valores originais;
2. Observe o formato da curva;
3. Diminua a temperatura;
4. Veja como a curva “abaixa”;
5. Aumente a temperatura;
6. Observe o aumento da pressão;
7. Altere a quantidade de matéria (n);

8. Compare diferentes cenários;
9. Observe especialmente o comportamento em pequenos volumes.

Exercitando-se acima, você compreenderá rapidamente que comprimir um gás aumenta muito sua pressão. A seguir, alguns cenários para você se divertir... “sem pressão”.

1. *Compressão intensa.* Na condição térmica abaixo, observe os menores volumes do gráfico. Por que pequenas reduções de volume aumentam tanto a pressão ?

const T = 300;

2. *Gás quente.* Suba a temperatura para 700. Por que gases mais quentes exercem maior pressão ?

const T = 700;

3. *Poucas partículas.* Menos partículas significam menos colisões ?

const n = 0.3;

4. *Muito gás em pouco espaço.* Por que recipientes fechados podem se tornar perigosos quando aquecidos ?

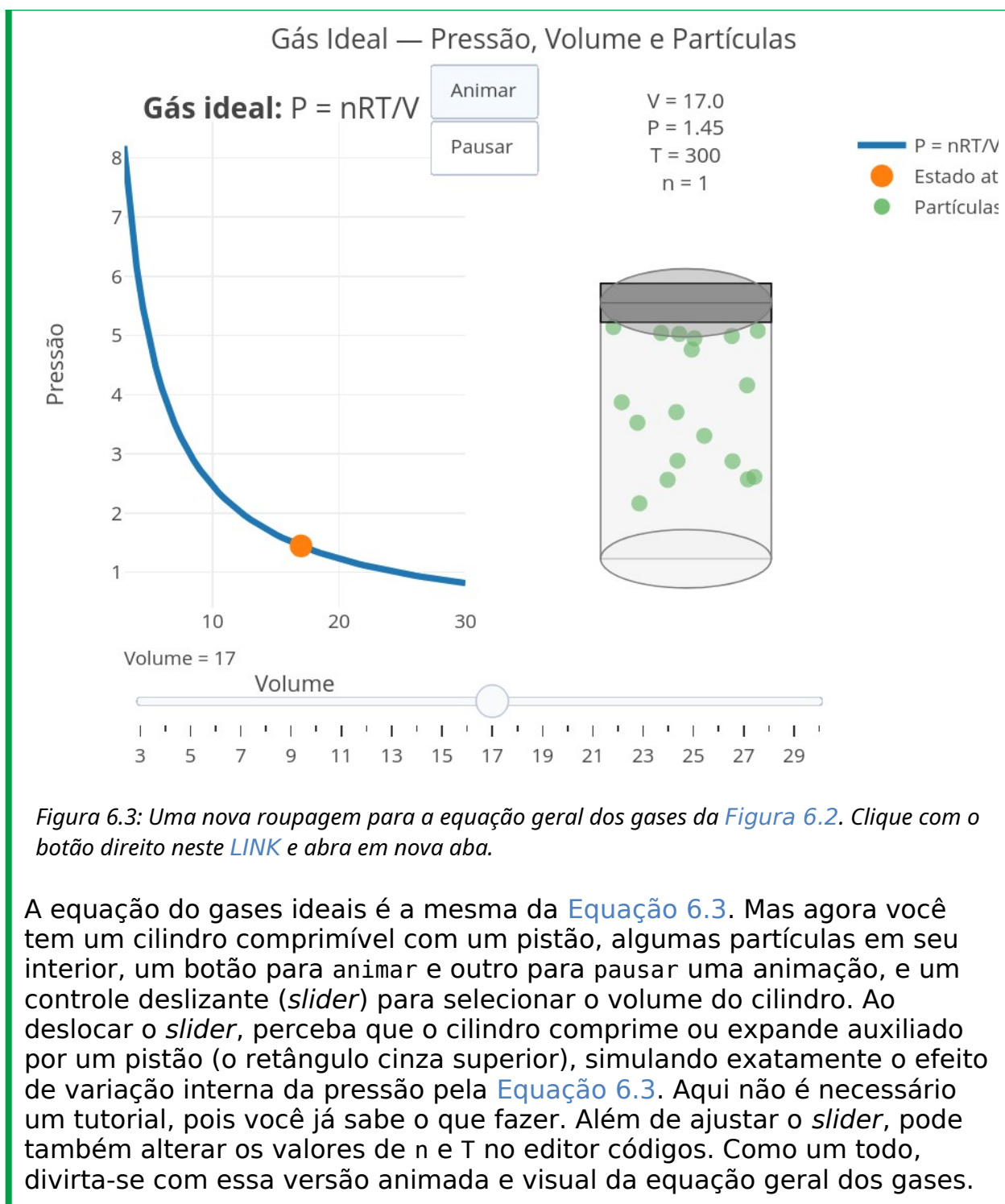
const n = 3;

const T = 500;

Ainda que o mecanismo seja invisível, o gráfico simula colisões microscópicas que ocorrem quando há variação de volume. Quando esse diminui, as partículas ficam mais próximas, as colisões aumentam, e a pressão cresce. Já quando a temperatura aumenta, as partículas se movem mais rapidamente, as colisões ficam mais intensas, e a pressão aumenta ainda mais. Assim, a curva traduz o comportamento coletivo de bilhões de partículas invisíveis.

Ou até bem mais do que isso, já que o mol, a unidade do Sistema Internacional para quantidade, representa $6,02 \times 10^{23}$ moléculas. Ou a bagatela de *seiscentos e dois sextilhões* de moléculas...ou ainda *602.000.000.000.000.000.000.000 moléculas* !! Ou ainda...e essa poucos sabem, praticamente o mesmo valor do peso de Marte, em kilogramas (útil isso, não ?!).

O que acha de dar uma incrementada na representação da [Figura 6.2](#) ? Não para acrescentar conceitos teóricos, mas dar um “*charme*” à visualização, além de *usabilidade*, e estilo ! Veja na [Figura 6.3](#) como uma mesma ideia pode ser vista de formas distintas usando linguagem de programação.



A equação dos gases ideais da [Figura 6.2](#) propõe uma situação no mínimo muito curiosa, e já traduzida pelo próprio nome: gases ideais. Um *gás ideal* é um conceito físico e não material. Ou seja, ele não existe no mundo real, pois a condição *ideal* pressupõe limitações impossíveis de acontecer. Isso porque um gás ideal é formado por partículas tão pequenas que não possuem massa, são muito afastadas uma das outras e não interagem entre si.

Por conseguinte, um gás ideal é um modelo teórico simplificado para explicar as relações de pressão, volume, temperatura e quantidade num gás “perfeito” ([Equação 6.2](#)). Mas o mundo costuma ser menos perfeito. Quando se assume que o gás possui partículas reais capazes de interagir entre si, o conceito deixa de ser ideal e cai no mundo real... ou do gás real. Uma maneira de compreender como o *gás real* se comporta em relação ao *gás ideal* está ilustrada pelo app de JSPlotly da [Figura 6.4](#).

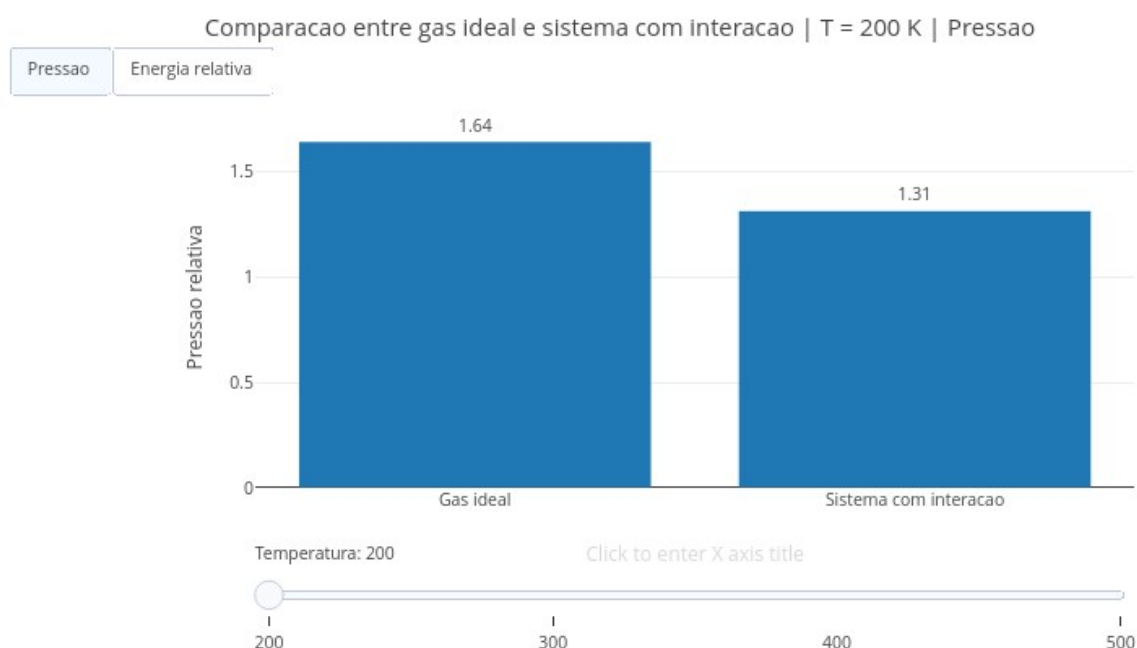


Figura 6.4: Gás real: quando as partículas deixam de ser “perfeitas”. Clique com o botão direito neste [LINK](#) e acesse o app.

Nesse modelo de *gases reais*, partículas podem se atrair, se repelir ou trocar energia de maneiras que o modelo ideal não representa completamente. Também no script você compara um gás ideal com um gás real, esse um sistema com interação entre partículas, e observa o que muda quando as partículas deixam de se comportar como pontos independentes.

Agite antes de usar

Execute o script com os valores originais e depois altere a const interação para valores de 0.05, 0.30, e 0.60.

```
const V = 10;
const n = 1.0;
const interacao = 0.20;
```

Depois use o *slider* para alterar a temperatura absoluta (em graus Kelvin, K). Observe como a diferença entre o gás ideal e o sistema com interação muda em cada cenário. Contrastando com a [Equação 6.3](#) para um gás ideal, no sistema com interação a variação da pressão conta com um fator associado à interação:

$$P_{\text{interação}} = P_{\text{ideal}}(1 - \text{interação})$$

Essa é uma simplificação didática, apenas. Ela não pretende reproduzir todos os detalhes de um gás real, mas ajuda a visualizar que interações entre partículas podem modificar o comportamento previsto pelo modelo ideal.

A função principal do script é:

```
function calcularEstado(T) {
  const P_ideal = (n * R * T) / V;
  const P_real = P_ideal * (1 - interacao);

  const E_ideal = T;
  const E_real = T * (1 - 0.5 * interacao);

  return {
    T: T,
    pressao: [P_ideal, P_real],
    energia: [E_ideal, E_real]
  };
}
```

Mas se você achar interessante conhecer também a Equação de van der Waals que modela os gases reais, segue abaixo:

$$\left(P + \frac{n^2 a}{V^2}\right)(V - nb) = nRT \quad (6.4)$$

Nessa [Equação 6.4](#), V_m é o volume molar, T é a temperatura, R é a constante universal dos gases - $8,31 \text{ J} \cdot \text{mol}^{-1} \text{K}^{-1}$ - a é um parâmetro de atração, e b um parâmetro de volume.

Voltando para o script da [Figura 6.4](#), para cada temperatura o código calcula a pressão do gás ideal, a pressão do sistema com interação, a energia relativa do gás ideal, e a energia relativa do sistema com interação. Depois, esses valores aparecem em barras comparativas. O *slider* muda a temperatura e os botões alternam entre pressão e energia relativa. Se quiser um tutorial rápido para mexer no *app*:

1. Rode o script com os valores originais;
2. Observe as barras de pressão;
3. Use o slider para mudar a temperatura;
4. Veja que a pressão aumenta quando a temperatura aumenta;

5. Clique em *Energia relativa*;
6. Compare o gás ideal com o sistema com interação;
7. Volte para *Pressão*;
8. Altere interação no código;
9. Rode novamente;
10. Observe como o aumento da interação amplia a diferença entre os modelos.

Veja que o modelo ideal deixa de descrever bem o sistema quando as partículas interagem muito. Alguns cenários são também propostos para seu estudo.

1. *Interação quase desprezível*. Quando a interação é muito pequena, o sistema se aproxima do gás ideal ?

```
const interacao = 0.02;
```

2. *Interação moderada*. Mude interação para 0.2. A diferença entre ideal e interativo já é visível ?
3. *Interação intensa*. Agora mude interação para 0.6. O modelo ideal começa a falhar de forma evidente ?
4. *Temperatura baixa*. Use o slider em 200 K. Em temperaturas menores, a energia relativa reduzida fica mais evidente ?
5. *Temperatura alta*. Agora mude para 500 K. A temperatura aumenta os dois modelos da mesma forma ou mantém diferenças proporcionais ?

O gráfico mostra que modelos científicos são aproximações. O gás ideal é útil porque simplifica o sistema. Mas essa simplicidade depende de algumas condições, tais como partículas muito afastadas, baixa interação entre moléculas, pressões moderadas, e temperaturas suficientemente altas. Quando essas condições deixam de valer, interações microscópicas passam a influenciar grandezas macroscópicas, como pressão e energia. A título de curiosidade, uma forma de se enxergar a diferença entre gás, líquido e sólido se dá pela proximidade de suas partículas constituintes.

O script usa dois recursos importantes da biblioteca `Plotly.js` de *JavaScript*: `frames` e `updatemenus`. Os `frames` guardam diferentes estados do gráfico:

```
const framesPressao = estados.map(function(est, i) {
  return {
    name: "P" + i,
    data: [...]
  };
});
```

Cada frame representa uma temperatura diferente. Os botões escolhem o tipo de variável mostrada, e o slider escolhe a temperatura, permitindo uma visão comparativa.

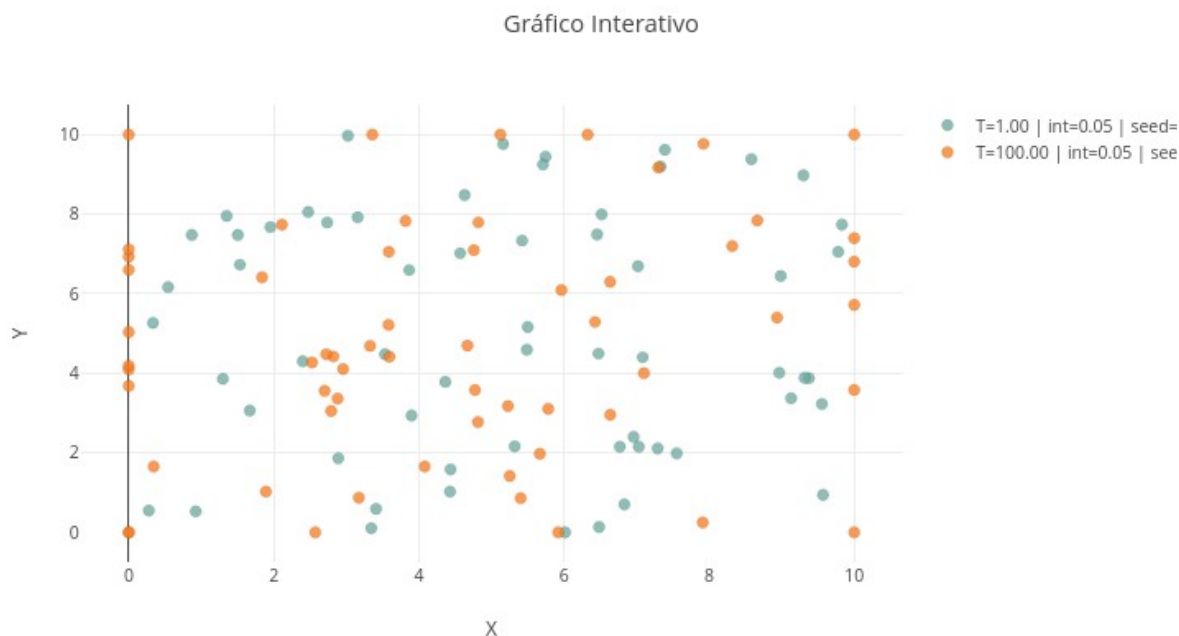
```
label: "Pressao"
label: "Energia relativa"
```

VOLTA PRO MUNDO



Retomando o início desse capítulo, podemos afimar agora que os sprays gelam porque há uma expansão rápida (trabalho com perda de energia), o sal dissolve podendo esfriar ou aquecer, e que o ar esquentar ao ser comprimido. Quando falamos em gases, muitas vezes começamos por variáveis como pressão, volume e temperatura. Quando falamos em termodinâmica, o foco se desloca para calor, trabalho e energia interna. Quando falamos em soluções, surgem concentração, dissolução e interação entre partículas. Apesar de consistirem panoramas bastante distintos, todos possuem características em comum, já que dizem respeito à partículas que se organizam, se movimentam e trocam energia com o ambiente. Assim, de gases para soluções, a Termodinâmica emprega relações comuns para explicar o comportamento da matéria, e distintas apenas em complexidade crescente. E “jamais” abandona a constante universal R dos gases !

Assim como foi feito para a [Figura 6.3](#), também é possível uma visualização mais clara para o comportamento de um gás real, o que é proposto no *app* da [Figura 6.5](#) abaixo. Ele não realiza uma animação como a da [Figura 6.3](#), mas propõe uma visão comparativa sem gráficos pelo editor.



*Figura 6.5: Gás real: quando as partículas deixam de ser “perfeitas”. Clique com o botão direito neste [LINK](#) e acesse o *app*.*

O código agora cria pequenos “mundos microscópicos” contendo

partículas em movimento que dependem de temperatura, intensidade de interação, quantidade de partículas, bem como de condições iniciais aleatórias. Observe então padrões emergindo do movimento coletivo das partículas.

Agite antes de usar

Execute o script original, depois altere os parâmetros e empilhe cenários diferentes pelo botão add do JSPlotly.

```
const T = 1.0;
const interacao = 0.05;
const N = 60;
```

Por exemplo, alterando a temperatura (0.5, 1.5, 2.2) e observando as cores e a dispersão das partículas. Veja que a temperatura está ligada ao movimento das partículas. Quanto maior a temperatura, maiores as velocidades, colisões e a dispersão do sistema. Além disso, o parâmetro `interacao` controla forças simplificadas entre as partículas. Essa representação mostra como um sistema de pontos independentes visto para gases ideais passa a apresentar um comportamento coletivo de um sistema com interação para gases reais.

Para entender um pouco como funciona o script, as partículas começam em posições aleatórias:

```
x.push(10 * rnd());
y.push(10 * rnd());
```

Depois recebem velocidades proporcionais à temperatura:

```
vx.push((rnd() - 0.5) * 1.8 * T);
vy.push((rnd() - 0.5) * 1.8 * T);
```

Em seguida, o sistema evolui em pequenos passos:

```
x[i] = x[i] + 0.08 * vx[i];
y[i] = y[i] + 0.08 * vy[i];
```

Quando partículas ficam muito próximas, surgem interações:

```
const f = interacao / d2;
```

Isso produz pequenas forças entre elas. A cor também muda continuamente com a temperatura, seguindo um padrão comum em Química. Para temperaturas menores os pontos ficam azulados, e para temperaturas maiores, os pontos ficam avermelhados (termocromismo). Essa é uma estratégia interessante para transformar números em comportamento visual. Globalmente, a lógica do algoritmo envolve uma geração pseudoaleatória de valor, atualização iterativa, colisões com paredes, forças simplificadas entre partículas, e o mapeamento contínuo de cor.

Para usar o *app*, experimente esse pequeno tutorial:

1. Rode o script original;

2. Observe a distribuição das partículas;
3. Aumente a temperatura (T);
4. Veja como as partículas ficam mais dispersas;
5. Observe a mudança gradual de cor;
6. Aumente interacao;
7. Veja como o sistema se torna mais “organizado” ou perturbado;
8. Aumente o número de partículas (N);
9. Use o botão *add* para comparar cenários diferentes no mesmo gráfico;
10. Experimente diferentes valores iniciais em seed (semente).

E no fundo, veja como o comportamento coletivo pode surgir apenas de regras simples entre partículas. Seguem alguns cenários para exploração do *app*.

1. *Sistema frio*. Por que partículas frias parecem menos agitadas ?

```
const T = 0.4;
```

2. *Sistema quente*. Mude para T = 2.2. Como o aumento da energia altera a dispersão do sistema ?
3. *Pouca interação*. O sistema se aproxima de partículas independentes ?

```
const interacao = 0.01;
```

4. *Forte interação*. Mude para interacao = 1.5. Interações intensas produzem padrões coletivos mais evidentes ?
5. *Muitas partículas*. O comportamento global muda quando o sistema fica mais “povoado” ?

```
const N = 150;
```

6. *Comparação empilhada*. Use T = 0.5 e sobreponha com T = 2.0 pelo botão *add*.

Como um todo, o script aproxima a ideia de que em sistemas reais podem surgir comportamentos coletivos. Esses comportamentos são evidenciados pelas regiões mais dispersas ou mais concentradas, pelas diferenças associadas à temperatura, e pelos padrões relacionados à interação. Esse tipo de abordagem aparece em vários temas diferentes, também Física estatística, dinâmica molecular, simulação de fluidos, ciência dos materiais, biologia celular e modelagem climática. São os chamados sistemas complexos, e que podem surgir de regras microscópicas muito simples.

Uma animação para gases reais

Vamos lá ! Confesse que o *app* da [Figura 6.3](#) para gases reais estava bem mais legal que o de gases reais da [Figura 6.5](#) acima, não ? Então, por que não considerar uma animação também para o comportamento coletivo de interação desse último ?

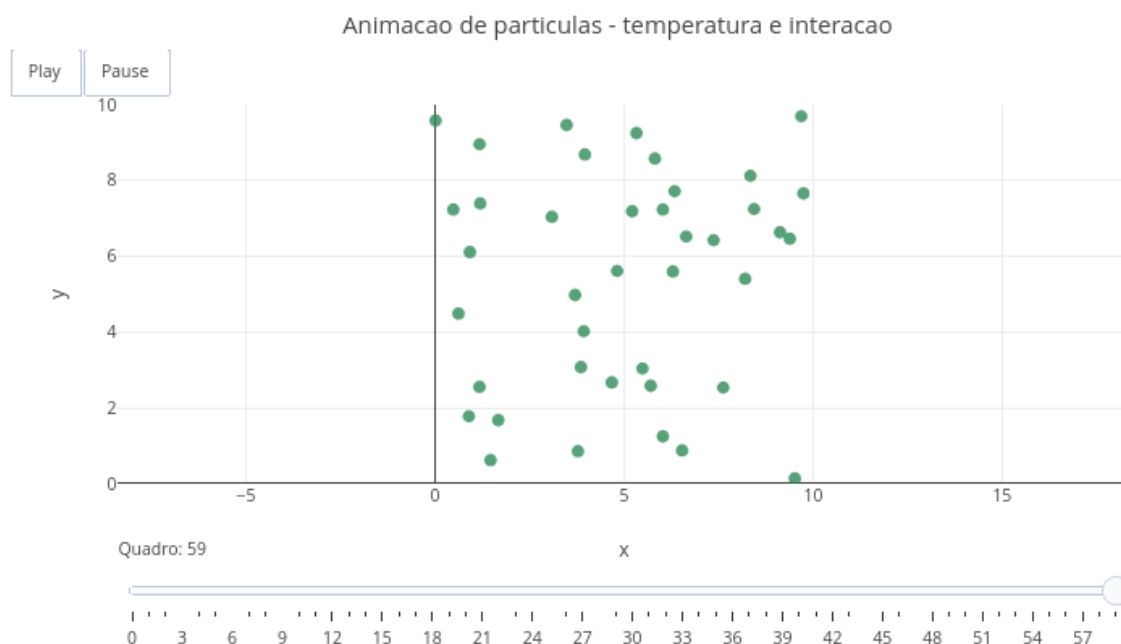


Figura 6.6: Gás real: quando a Termodinâmica ganha movimento. Clique com o botão direito neste [LINK](#) e acesse o app.

O script transforma a Termodinâmica em uma animação dinâmica de partículas interagindo em tempo real. Elas se movem, colidem com as paredes, influenciam umas às outras e mudam de comportamento conforme a temperatura. Essa movimentação sob interação faz com que o comportamento microscópico coletivo produza propriedades macroscópicas observáveis.

Agite antes de usar

Execute o script original e use os botões Play e Pause para observar o movimento das partículas. Assim como para o script da Figura 6.5, altere depois a temperatura e o efeito de interação.

```
const T = 1.2;
const interacao = 0.04;
const N = 40;
```

Observe que o script tenta representar a ideia de que a temperatura está associada à energia cinética média das partículas. Quanto maior a temperatura, maior a velocidade média, as colisões e a agitação do sistema. Além disso, o parâmetro `interacao` introduz forças simplificadas entre partículas, permitindo mostrar o comportamento coletivo mais complexo para um gás real. A visualização ocorre na animação por movimento e cor. A cor muda conforme a temperatura: azul (sistema frio), passando por verde (moderado), laranja (quente), até vermelho (muito energético).

Se quiser saber como o algoritmo produz a animação, as partículas começam em posições aleatórias:


```
x.push(10 * rnd());
y.push(10 * rnd());
```

Depois recebem velocidades relacionadas à temperatura:

```
vx.push((rnd() - 0.5) * 1.6 * T);
vy.push((rnd() - 0.5) * 1.6 * T);
```

O sistema evolui em pequenos passos temporais:

```
x[i] = x[i] + 0.10 * vx[i];
y[i] = y[i] + 0.10 * vy[i];
```

Quando partículas se aproximam, surgem interações:

```
const f = interacao / d2;
```

Cada quadro da animação é armazenado em:

```
frames.push({...})
```

Isso permite ao JSPlotly reproduzir o sistema como uma animação contínua.

Para “brincar” com a animação:

1. Rode o script original;
2. Clique em Play;
3. Observe o movimento coletivo;
4. Use Pause para analisar um quadro específico;
5. Aumente a temperatura T;
6. Veja como as partículas ficam mais rápidas;
7. Observe a mudança de cor associada à temperatura;
8. Aumente interacao;
9. Veja como o comportamento coletivo muda;
10. Aumente o número de partículas N;
11. Observe como o sistema se torna mais “denso”. Perceba que propriedades macroscópicas podem surgir apenas do movimento microscópico de partículas. Cenários ?
12. *Sistema frio*. Como partículas frias se comportam visualmente ?

```
const T = 0.4;
```

1. *Sistema quente*. Mude T para 2.4. O aumento da energia torna o sistema mais caótico ?
2. *Interação quase nula*. O sistema se aproxima do comportamento de um gás ideal ?

```
const interacao = 0.01;
```

4. *Forte interação*. Mude interacao para 1.20. As partículas começam a influenciar fortemente umas às outras ?

5. *Sistema muito “povoado”*. Como o aumento do número de partículas altera colisões e padrões ?

const N = 120;

1. *Comparação visual*. Compare $T = 0.6$ com $T = 2.0$. É possível sentir visualmente a diferença de temperatura apenas observando o movimento ?

O script traz também um panorama interessante para temperatura, já que sua modificação impacta visualmente em velocidade, colisão, dispersão e agitação das partículas no simulador. Da mesma forma, a interação das partículas também gera um efeito coletivo.



E SE...

Para essa seção de divagações e descobertas, segue um elenco mais “real” do que “ideal”.

- E se o gás deixasse de ser ideal justamente quando a pressão se tornasse muito alta ?
- E se partículas comesçassem a interagir tão fortemente que o sistema passasse a se comportar mais como um líquido do que como um gás ?
- E se o aumento da temperatura fosse suficiente para reorganizar completamente o movimento coletivo das partículas ?
- E se pequenas diferenças microscópicas produzissem comportamentos macroscópicos gigantescos ?
- E se pressão, temperatura e volume fossem apenas manifestações visíveis de bilhões de colisões invisíveis acontecendo o tempo todo ?
- E se soluções líquidas escondessem processos semelhantes, com partículas dissolvidas alterando energia, organização e equilíbrio do sistema ?
- E se motores, atmosferas, metabolismo celular, mudanças climáticas e até o funcionamento do próprio corpo fossem consequência das mesmas ideias físicas exploradas nestes scripts ?



MESMO PADRÃO, OUTROS MUNDOS

Os conceitos desse capítulo podem ser revisitados em muitos outros contextos, como numa simples panela de pressão, por exemplo. Ao aumentar a pressão interna, a água pode atingir temperaturas maiores antes de entrar em ebulição intensa. Cilindros de ar comprimido também mostram que reduzir volume e elevar pressão envolve armazenamento de energia e exige cuidados de segurança. Já bolsas térmicas instantâneas funcionam porque certos processos de dissolução ou mistura absorvem energia, enquanto outros a liberam. E o que dizer de calibrar os pneus após rodar um bocadinho de estrada? Com o pneu quente, o ar dentro da câmara está expandido, informando uma pressão errada no calibrador do posto de gasolina, já que essa decairá quando o pneu esfriar.

Quando pensamos em animais e plantas, é possível perceber que gases, pressão parcial e difusão aparecem em processos biológicos fundamentais. Geladeiras e aparelhos de ar-condicionado também são exemplos muito ricos envolvidos nos conceitos de compressão, expansão, troca de calor e trabalho de forma articulada. Um último exemplo dessa seara inesgotável é a própria atmosfera terrestre, a qual oferece um grande laboratório natural com constantes variações de compressão, expansão, temperatura e comportamento de gases em sistemas reais.

POR DENTRO DA FERRAMENTA



Ao longo desta obra, a essa seção tem sido dedicada a espelhar o pensamento computacional e a lógica de programação junto aos diversos scripts de JSPlotly, finalizando com um pseudocódigo voltado a um desses scripts em cada capítulo.

Os scripts de JSPlotly utilizados nesse capítulo, por exemplo, miram numa compreensão progressiva para alguns conceitos da termodinâmica. Brevemente, a relação $P \times V$ de pressão de volume (Script no. 1), a comparação de um gás ideal com um sob interação (Script no. 2), a comparação de cenários microscópicos de partículas (Script no. 3), bem como algumas animações dinâmicas de sistemas coletivos (Script no. 4). Esses scripts estão munidos de alguns comandos e relações para aplicação de funções matemáticas (gases), conservação de energia (termodinâmica) e simulação de partículas.

Mas você já deve ter percebido também que, ao longos dos capítulos deste trabalho, aquelas competências digitais passaram a integrar paulatinamente o próprio capítulo, diminuindo a necessidade de sua exclusividade para o tópico que se apresenta. A partir de agora, então, essa seção irá dedicar-se à **verdadeira ferramenta** por detrás de todos os scripts que podem ser feitos com o JSPlotly: a linguagem **JavaScript**.

O entendimento de como opera essa ferramenta permitirá a você compreender e modificar qualquer código deste livro, ajustá-lo às suas preferências ou necessidades, bem como criar outros tantos que desejar. Além disso, poderá também entender o que há por detrás das páginas de internet e criar as suas próprias, como blogs e sites. Isso porque o JavaScript opera silenciosamente quando você clica num botão da tela de seu celular, preenche um formulário ou acompanha uma animação gráfica na internet. Por óbvio, entretanto, esta nanométrica seção não lhe dará condições de sair por aí programando em JavaScript. Mas lhe dará munção básica para que avance ao próximo nível nessa e em qualquer linguagem de programação.

JavaScript (JS), é uma linguagem de programação moderna, utilizada por grandes empresas e *big techs*, e focada na interatividade entre a página web e o usuário. Diferente de outras linguagens, *JavaScript* não precisa ser compilada separadamente por um programa, pois é interpretada a partir de máquinas homônimas presentes em navegadores comuns (*browser*), como Firefox, Chrome, Safari, Opera, Edge, etc.

Assim, *Javascript (JS)* é uma linguagem para web. Ela completa a tríade essencial, *HTML-CSS-JS*. Enquanto *HTML* preocupa-se com o conteúdo e *CSS* com o estilo, *JavaScript* ocupa-se do comportamento, ou seja, da interatividade do usuário frente à uma página web. Vamos então chamá-la agora carinhosamente por *JS*, somente.

Como ocorre com outras linguagens de programação, a linguagem *JS* possui bibliotecas, que são módulos independentes de códigos para cumprir certas funções. Nesse sentido, *JS* cumpre metade do nome do *JSPlotly*. A outra metade vem da principal biblioteca que é utilizada por

esse para a construção de gráficos, mapas, e alguns outros objetos interativos, [Plotly.js](#).

6.1 Estrutura de JS

Uma linguagem de programação normalmente possui características estruturais comuns. Isso quer dizer que há comandos para declarar uma variável e seu tipo, a entrada e saída de dados (resultados), a criação e uso de funções, e cálculos aritméticos, por exemplo. E tal como outras linguagens de programação, JS opera com uma lógica similar de declarações (palavras-chave, operadores, valores, expressões). Em resumo, a gente dizer as declarações de JS compõem:

1. Palavras-chave: sintaxe da linguagem JS;
2. Operadores: caracteres que realizam operações;
3. Valores: texto, números, verdadeiro/falso (variável "booleana"), "não definido", "nulo";
4. Expressões: trechos de código que produzem um único valor.

Do ponto de vista estrutural, por conseguinte, JS possui características que também são comuns a outras linguagens de programação:

1. Saída de dados;
2. Declaração de variáveis;
3. Tipos de dados;
4. Constantes;
5. Aritmética;
6. Vetores;
7. Funções;
8. Geração de dados aleatórios;
9. Objetos;
10. Operadores;
11. Estruturas de controle;

Por enquanto é só. Se quiser aprender um pouquinho mais de JS, basta vira a página !

7. Radioatividade, Energia Nuclear e Impactos



O MUNDO TE CHAMA

Nem toda energia aparece como calor ou movimento. Algumas das transformações mais intensas do universo acontecem nas escalas invisíveis de dentro do átomo. A radioatividade mostra que a matéria não é completamente estável. Dentro dos átomos existe energia armazenada, capaz de curar, medir, datar, e gerar eletricidade, mas também matar, exigindo responsabilidade ética, ambiental e social. Certos núcleos se transformam espontaneamente, emitindo partículas e energia. E essas transformações permitem datar fósseis e rochas, diagnosticar e tratar doenças, gerar energia elétrica, esterilizar materiais e estudar estruturas invisíveis.

Por outro lado, a radioatividade também levanta questões ambientais, sociais e mesmo da sobrevivência da humanidade, como a gestão de resíduos radioativos e segurança, o impacto ambiental, decisões sociais e políticas, geração alternativa de energia, enriquecimento de isótopos para fins pacíficos e restrição ao emprego de ogivas e armas nucleares.

O capítulo pretende fornecer uma visão panorâmica no tema de energia nuclear e impactos no mundo real, conectando ciência, tecnologia e sociedade, e apresentando alguns dos conceitos no tema. Ilustrando, a meia-vida radioativa, os tipos de radiação, fissão nuclear e energia liberada, e a discussão de riscos, benefícios e impactos da energia nuclear.

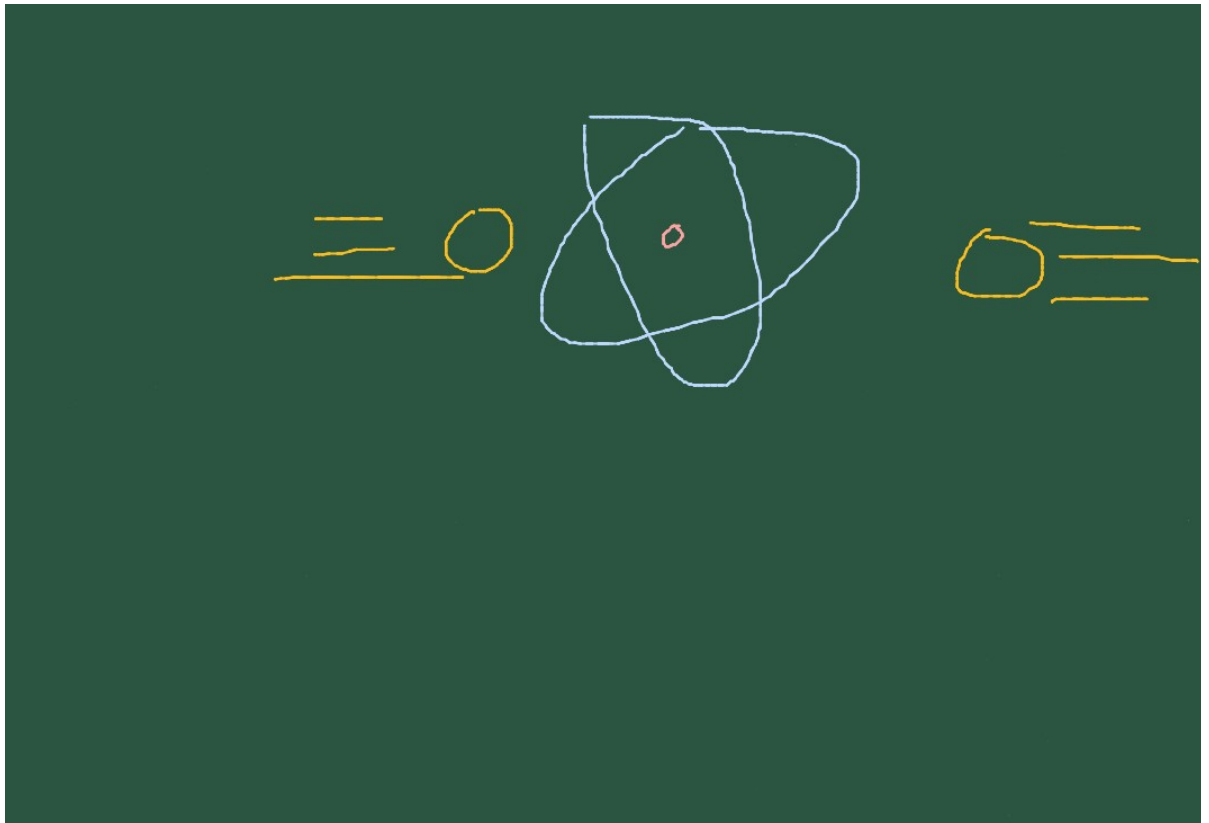


Figura 7.1: O núcleo do átomo transforma matéria, energia e decisões humanas.

Ao final, você será capaz de:

- explicar o que torna um núcleo radioativo;
- diferenciar radiações alfa, beta e gama;
- interpretar curvas de decaimento e meia-vida;
- relacionar fissão nuclear com produção de energia;
- comparar benefícios e riscos da energia nuclear;
- discutir impactos ambientais, sociais e tecnológicos da radioatividade;
- investigar modelos computacionais para visualização interativa do tema;

MEXA ANTES DE ENTENDER



Acesse o JSPlotly da [Figura 7.2](#) abaixo para entender um pouco como uma amostra radioativa diminui ao longo do tempo.

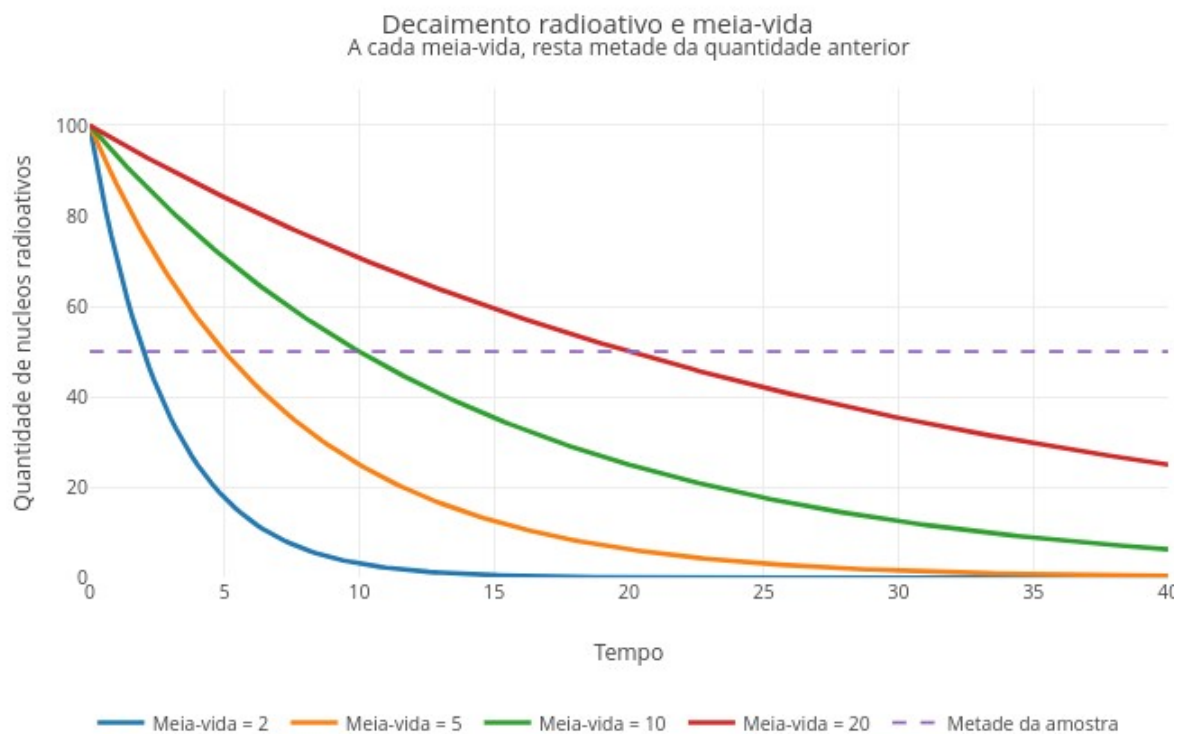


Figura 7.2: Meia-vida: a cada intervalo de tempo resta metade da quantidade anterior. Clique com o botão direito neste [LINK](#) e abra o app em nova aba.

Agite antes de usar

O gráfico compara diferentes valores de meia-vida, mostrando que alguns materiais decaem rapidamente, enquanto outros permanecem ativos por muito mais tempo. Execute o script e observe as curvas para as meias-vidas.

Compare qual curva cai mais rápido.

```
const meiasVidas = [2, 5, 10, 20];
```

Depois altere como abaixo. Observe que a forma do decaimento continua a mesma, mas a escala muda.

```
const quantidadeInicial = 200;  
const tempoMaximo = 80;
```

Verifique que a radioatividade não desaparece toda de uma vez. O decaimento radioativo é probabilístico. Não sabemos exatamente quando um núcleo específico vai decair mas, quando observamos muitos núcleos juntos, aparece um padrão regular. Um tutorial para uso do *app* segue abaixo.

1. Rode o script;
2. Observe as quatro curvas;

3. Veja qual delas cruza primeiro a linha da metade;
4. Compare meia-vida curta e meia-vida longa;
5. Aumente tempoMaximo;
6. Observe o comportamento de longo prazo;
7. Altere quantidadeInicial;
8. Veja que o padrão relativo permanece;
9. Modifique a lista meiasVidas;
10. Compare novos materiais simulados.

Observe que o que muda mais é o tempo necessário para a amostra decair, e não sua quantidade inicial, o que evidencia uma das aplicações mais práticas do cálculo de meia-vida: a datação de fósseis e artefatos antigos. Que tal explorar alguns cenários ?

1. *Dcaimento rápido.* Por que materiais com meia-vida curta desaparecem rapidamente ?

```
const meiasVidas = [1, 2, 3, 4];
```

2. *Dcaimento lento.* Por que materiais com meia-vida longa permanecem ativos por muito tempo?

```
const meiasVidas = [10, 20, 40, 80];
```

3. *Tempo maior.* A quantidade chega exatamente a zero ou apenas se aproxima dele ?

```
const tempoMaximo = 100;
```

4. *Quantidade inicial maior.* A meia-vida muda quando começamos com mais material ?

```
const quantidadeInicial = 1000;
```

5. *Comparação extrema.* Como curvas muito diferentes ajudam a entender risco, persistência e armazenamento ?

```
const meiasVidas = [1, 5, 50, 100];
```

As curvas mostram decaimento exponencial. No início, a queda pode parecer rápida. Depois, a quantidade continua diminuindo, mas cada nova metade é calculada sobre o que ainda resta. Por isso, a curva se aproxima de zero gradualmente. Ou seja, meia-vida é o tempo que leva para restar metade de um material.

Por dentro do script

O script usa uma função para calcular o decaimento:

```
function calculaDecaimento(N0, T12, tempo) {  
  return N0 * Math.pow(0.5, tempo / T12);  
}
```

Onde N0 é a quantidade inicial, T12 é a meia-vida, e tempo é o período transcorrido. Depois, o script calcula várias curvas:

```
const meiasVidas = [2, 5, 10, 20];
```

Cada curva mostra uma amostra com meia-vida diferente. A linha tracejada marca metade da quantidade inicial, e ajuda a visualizar quando cada curva atinge sua primeira meia-vida.



FAZENDO APARECER

A equação de decaimento radioativo mostra quanto de uma substância radioativa ainda resta depois de certo tempo:

$$N(t) = N_0 \cdot \left(\frac{1}{2}\right)^{t/T_{1/2}} \quad (7.1)$$

Na [Equação 7.1](#), $N(t)$ representa a quantidade de material radioativo que resta no tempo t , N_0 a quantidade inicial da substância radioativa, e $T_{1/2}$ a meia-vida.

Assim, a quantidade restante pode ser entendida como a quantidade inicial multiplicada por metade elevada ao número de meias-vidas transcorridas. Chato, não? Então, quem sabe um “corta pela metade, corta pela metade, corta pela metade,...”, e assim por diante. Na prática, um padrão que permite prever quanto material radioativo ainda resta depois de certo tempo.

A radiação é conhecida por apresentar riscos à saúde em função de seu poder de penetração nos tecidos, chegando inclusive aos ossos. Existem alguns tipos de radiação, cada um interagindo com a matéria de forma diferente, em com diferentes níveis de impacto sobre a matéria. O JSPlotly a seguir apresenta uma forma mais descontraída para esse estudo.

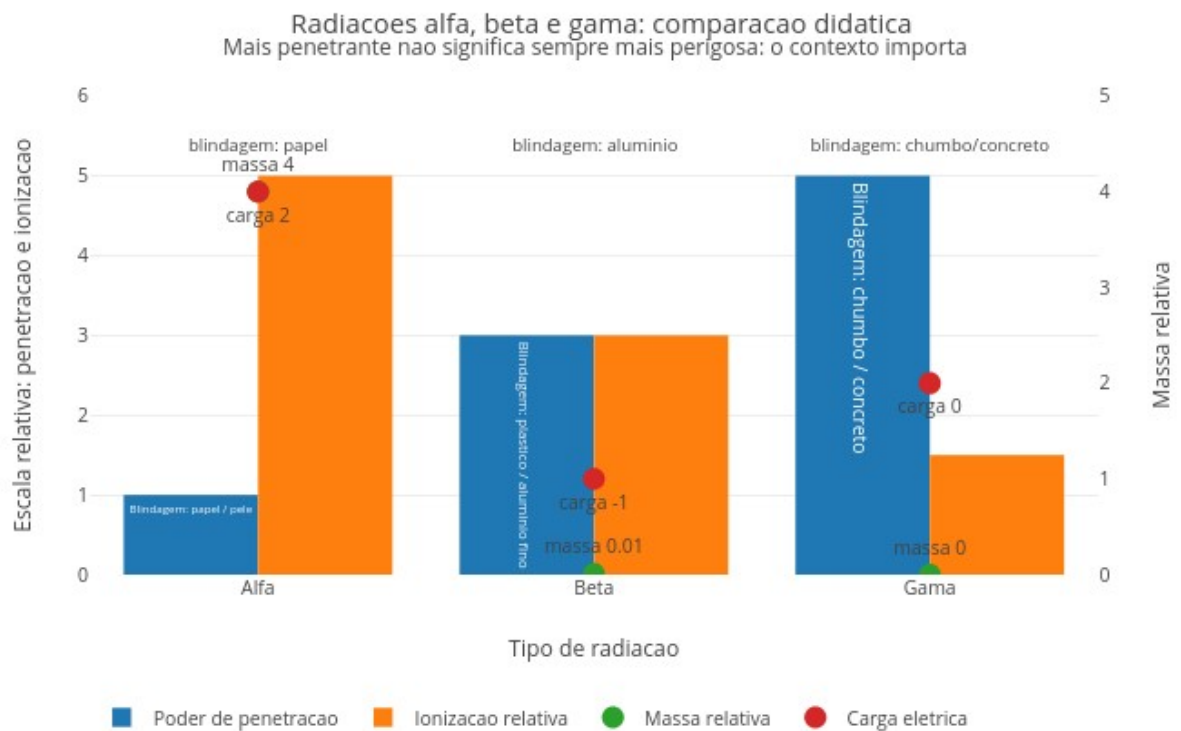


Figura 7.3: Uma radiação mais penetrante não significa necessariamente mais perigosa. Script inicial mais simples. Clique com o botão direito neste [LINK](#) e abra em nova aba.

O script compara três tipos de radiação: alfa (α), beta (β) e gama (γ). Você pode observar, simultaneamente, o poder de penetração, a capacidade de ionização, massa relativa, carga elétrica e tipo de blindagem.

Agite antes de usar

Execute o script. As barras mostram penetração e ionização, enquanto os pontos mostram massa e carga. Observe qual radiação penetra mais, qual ioniza mais, e qual tem maior massa. Passe o mouse sobre as barras para ver a blindagem. E compare os tipos de radiação. Por exemplo, alfa vs beta, beta vs gama, e alfa vs gama.

Apesar de transparecer que um tipo seja mais “forte” e mais perigoso do que outro, no mundo real não funciona assim. O risco depende de alguns fatores, como a penetração (até onde chega), a ionização (quanto dano local causa), e o contexto (externo ou interno ao corpo), além do tempo e da quantidade de exposição.

Por exemplo, a alfa possui baixa penetração mas alta ionização, enquanto que a gama possui alta penetração mas menor ionização local. Isso quebra o mito de que quanto mais penetrante a radiação, mais perigosa ela se apresenta. Vai abaixo um modo de uso do script.

1. Rode o script;
2. Observe as barras de penetração;
3. Compare com as de ionização;
4. Observe os pontos de massa;
5. Veja a carga elétrica;
6. Passe o mouse para ver a blindagem;
7. Compare os três tipos;
8. Relacione massa e penetração;
9. Relacione ionização e dano potencial;
10. Interprete o conjunto como um sistema.

Refleta qual tipo de radiação causa mais dano em diferentes situações. E siga os cenários abaixo para se apropriar do tema e “*brilhar no escuro*”.

1. *Comparação direta.* Observe o gráfico completo. Qual radiação penetra mais ? qual ioniza mais ?
2. *Radiação alfa.* Foque em “Alfa”. Por que ela é facilmente bloqueada, mas ainda perigosa ?
3. *Radiação beta.* Foque em “Beta”. Por que ela representa um intermediário entre alfa e gama ?
4. *Radiação gama.* Agora foque em “Gama”. Por que ela exige materiais densos para blindagem, como chumbo ?
5. *Blindagem.* Observe os textos: * papel, *alumínio*, e *chumbo*. Por que materiais diferentes são necessários para cada tipo ?

6. *Interpretação integrada.* Compare tudo. Qual radiação é mais perigosa fora do corpo ? e dentro do corpo ?

Comparando os tipos de radiação, podemos resumir como:

1. Alfa: alta massa, alta ionização, baixa penetração.
2. Beta: massa pequena, penetração média, ionização intermediária.
3. Gama: sem massa, alta penetração, menor ionização local.

Concluindo, o tipo de interação importa mais do que sua “força” isolada.

Por dentro do script

O script organiza os dados em uma lista:

```
const radiacoes = [...]
```

Cada tipo possui propriedades de massa, carga, penetração, ionização e blindagem. Depois separa os dados para o gráfico:

```
const penetracao = radiacoes.map(r => r.penetracao);  
const ionizacao = radiacoes.map(r => r.ionizacao);
```

No gráfico as barras mostram penetração e ionização e os pontos massa e carga, reunindo várias dimensões em uma única visualização.

VOLTA PRO MUNDO



O átomo possui duas regiões, o núcleo e a eletrosfera. O núcleo é a região central, extremamente compacta e densa, e que concentra praticamente toda a massa atômica. Nele ficam os prótons (com carga elétrica positiva) e os nêutrons (partículas sem carga que ajudam a estabilizar o núcleo). Ao redor desse centro, em um vasto espaço quase totalmente vazio, fica a eletrosfera, onde os elétrons (partículas de carga negativa e massa desprezível) “giram” em alta velocidade, atraídos pela carga positiva do núcleo.

Diversas reações que ocorrem nos átomos podem liberar energia, como a fusão nuclear, que ocorre quando dois núcleos atômicos leves se unem para formar um único núcleo mais pesado, o que libera uma quantidade colossal de energia. E também a `fissão nuclear`, que envolve a divisão de um átomo pesado, também liberando grande quantidade de energia.

Agora que você já aprendeu “na prática” um pouco sobre a meia-vida, o decaimento radioativo e os tipos de radiação, que tal simular um fenômeno do mundo real, a fissão nuclear, base para a produção de energia elétrica em usinas nucleares ? E melhor ainda...com uma animação !

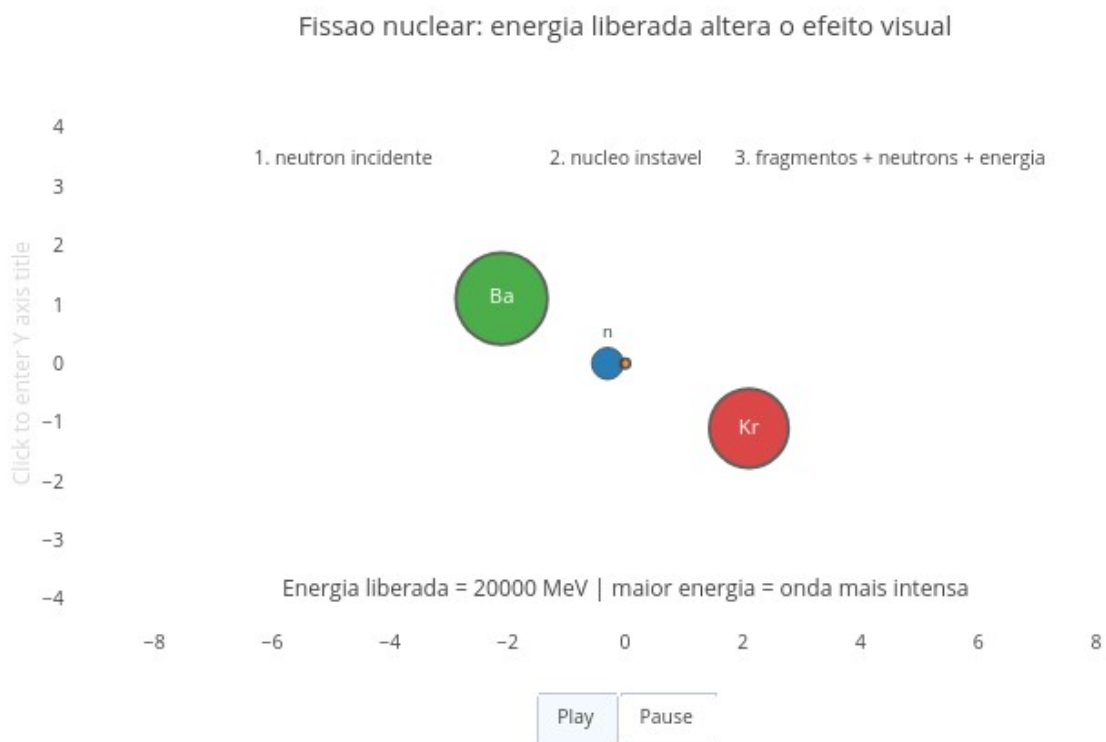


Figura 7.4: A fissão nuclear transforma massa em energia e pode gerar reações em cadeia. Clique com o botão direito neste [LINK](#) e abra o app em nova aba.

A fissão nuclear libera energia quando o núcleo se rompe. Mais detalhadamente, pela divisão de um núcleo atômico pesado e instável (como o Urânio-235) em dois ou mais núcleos menores, liberando uma enorme quantidade de energia, nêutrons e radiação.

Agite antes de usar

O script da [Figura 7.4](#) mostra uma *fissão nuclear animada*. Nela você pode acompanhar um nêutron incidente, o núcleo de U-235, a quebra em fragmentos menores, a emissão de novos nêutrons e a liberação de energia em ondas.

Execute o script e clique em Play. Observe o nêutron se aproximando, o núcleo instável, a ruptura e a explosão de energia. Depois altere o valor de energia liberada, e observe como o efeito visual muda (ex: 100, 200, 400).

```
const energiaLiberada = 50;
```

Pense: o que muda mais, a forma da reação ou a intensidade da energia ?

Quando o núcleo se divide, parte da massa é convertida em energia, novos nêutrons são liberados, e esses nêutrons podem atingir outros núcleos. Isso leva a um fenômeno de reação em cadeia, na qual uma única fissão pode gerar várias outras. Vai agora um modo de usar para o aplicativo.

1. Rode o script;
2. Clique em Play;
3. Observe o nêutron incidente;
4. Veja o núcleo antes da fissão;
5. Observe o momento da ruptura;
6. Veja os fragmentos se afastando;
7. Observe os novos nêutrons;
8. Veja as ondas de energia;
9. Pause em diferentes momentos;
10. Altere a energia liberada;
11. Compare os efeitos.

Perceba que mudanças iniciais podem gerar grandes efeitos. Seguem também alguns cenários para você experimentar.

1. *Baixa energia*. Por que o efeito parece mais “contido” ?

```
const energiaLiberada = 20;
```

2. *Energia intermediária*. Agora mude a energiaLiberada para 100. Quando o processo começa a parecer mais intenso ?
3. *Alta energia*. Suba então para 800. O que muda na propagação da energia ?
4. *Número de frames*. O que você percebe melhor com animação mais lenta ?

```
const numeroFrames = 80;
```

5. *Interpretação física*. Observe os três nêutrons liberados. Quantas novas fissões eles poderiam iniciar ?

A fissão nuclear ocorre em etapas. O nêutron atinge o núcleo (colisão), o núcleo se deforma (instabilidade), ocorrendo a divisão em fragmentos

(fragmentação), com a liberação de novos nêutrons (emissão) e a propagação de ondas (energia). Repare que a energia nuclear não é uma explosão “mágica” que surge de átomo fechado, mas o resultado de reorganização da matéria.

Por dentro do script

O script começa definindo a energia:

```
const energiaLiberada = 20;
```

Essa energia controla o efeito visual:

```
const fatorEnergia = Math.sqrt(energiaLiberada / 200);
```

Depois, a animação é construída frame a frame:

```
for (let i = 0; i < numeroFrames; i++)
```

A simulação tem duas fases. A primeira é a *aproximação*, quando o nêutron se aproxima do núcleo.

```
neutronX = -6 + ...
```

E a segunda fase é a *fissão*, quando o núcleo se divide em Ba (bário), Kr (criptônio) e novos nêutrons.

```
const p = (fase - 0.35) / 0.65;
```

E a energia aparece como ondas, e que representam a energia liberada no processo.

```
ondaEnergia(p, ...)
```

E SE...



Como de praxe neste livro, um pouco de reflexão, ora ligada aos scripts de JSPlotly, ora não. Dividindo a seção pelo script:

Script no.1

- E se você pudesse observar uma amostra radioativa desde o início — veria ela desaparecer de forma contínua ou em pequenos eventos invisíveis ?
- E se cada núcleo tivesse um “relógio próprio” imprevisível — como explicar o comportamento coletivo tão regular ?
- E se a meia-vida fosse diferente — quanto tempo levaria para um material deixar de ser perigoso ?
- E se uma pequena diferença na meia-vida mudasse completamente o destino de um ambiente contaminado ?

Script no.2

- E se você pudesse reduzir seu corpo ao tamanho de um núcleo — como pareceria a colisão de um nêutron ?
- E se a fissão nuclear fosse visível a olho nu — você a veria como uma explosão... ou como uma reorganização da matéria ?
- E se cada nêutron liberado atingisse outro núcleo — quantas gerações de reações aconteceriam ?
- E se essa cadeia fosse controlada — o que mudaria em relação a uma reação descontrolada ?
- E se toda a energia de uma cidade viesse de reações como essa ?

Script no.3

- E se a radiação fosse invisível, mas seus efeitos acumulados fossem inevitáveis — como medir o risco ?
- E se a radiação mais penetrante não fosse a mais perigosa — o que realmente define perigo ?
- E se o mesmo tipo de radiação pudesse curar (na medicina) e também causar danos ?
- E se o perigo dependesse mais da forma de interação do que da intensidade ?

Script no.4

Hahaaa !!! Isso mesmo. Questões certinhas e devaneios sobre um script de JSPlotly que você nem viu ainda ! Mas já vai se preparando, pois ele está na próxima seção do capítulo.

- E se você tivesse que decidir como produzir energia para uma cidade inteira ?
- E se escolher uma fonte significasse abrir mão de outra ?
- E se uma fonte fosse limpa, mas intermitente (irregular na oferta) ?
- E se outra fosse estável, mas gerasse resíduos ?
- E se a melhor decisão não fosse técnica, mas ética ?

Script no. 5

Hahaaa !!! Esse não existe... pegadinha ! Mas é pra você experimentar uma bilocação cerebral...parte dentro da caixa craniana, parte lá longe !

- E se mudássemos os critérios — e não a tecnologia — o resultado da escolha mudaria ?
- E se o “melhor” sistema energético dependesse da sociedade que o utiliza ?
- E se o futuro energético fosse uma combinação, e não uma única solução ?
- E se toda a energia nuclear viesse, no fundo, da reorganização de partículas invisíveis ?
- E se massa e energia fossem apenas duas faces da mesma coisa ?
- E se o que chamamos de “matéria estável” fosse apenas um equilíbrio temporário ?
- E se estivéssemos olhando para um pedaço de urânio...ou para a história energética da humanidade ?

- E se a energia que alimenta cidades fosse a mesma que pode destruir cidades ?
- E se o problema não fosse a física...as como escolhemos usá-la ?
- E se, em outra estrela, reações nucleares ocorressem continuamente em escalas gigantescas ?
- E se o Sol fosse apenas um reator nuclear natural ?
- E se a energia que chega até nós tivesse começado em processos como esses ?
- E se tudo isso — decaimento, radiação, fissão, energia — fosse parte de um mesmo padrão ?
- E se entender esse padrão fosse o primeiro passo para tomar decisões melhores ?
- E se aprender física nuclear não fosse apenas entender o átomo... mas entender responsabilidade ?
- E se o conhecimento não fosse só poder...mas também escolha ?
- E se agora, depois de tudo isso...você começasse a olhar para energia de outro jeito ?



MESMO PADRÃO, OUTROS MUNDOS

A radioatividade permeia nosso cotidiano o tempo todo, mesmo que não a percebamos. A pesquisa de matrizes energéticas, a medicina nuclear, a Arqueologia, as Ciências Naturais e outros tantos vértices da pesquisa e indústria desses tempos estão, de uma forma ou outra, envolvidos com radioatividade. Na medicina nuclear, com uso de radioisótopos em exames de imagem, rastreamento de processos fisiológicos e tratamentos específicos. Na datação por carbono, permitindo estimar a idade de materiais orgânicos, rochas e objetos arqueológicos por determinação da meia-vida. Na inspeção industrial, sendo utilizada para se verificar soldas, medir espessuras, esterilizar materiais e acompanhar processos.

A radioatividade também está presente na pesquisa de exploração espacial, pela maior exposição no vácuo a um fluxo constante de partículas ionizantes de alta energia, como a radiação solar, raios cósmicos e cinturões de radiação em redor dos planetas. Também está presente no meio ambiente, uma vez que rejeitos radioativos exigem isolamento, monitoramento e planejamento. E, como um todo, também na Sociedade, despertando debates intensos sobre o desenvolvimento e uso da energia nuclear, porque envolve ciência, risco, confiança pública, política energética e memória histórica.

Em relação a esse último papel pertinente à decisões políticas e uso consciente de alternativas energéticas, segue um último JSPlotly ([Figura 7.5](#)). O aplicativo agora compara fontes de energia por critérios ponderados. Será que dá pra escolher uma fonte que agregue benefícios e exclua riscos ?

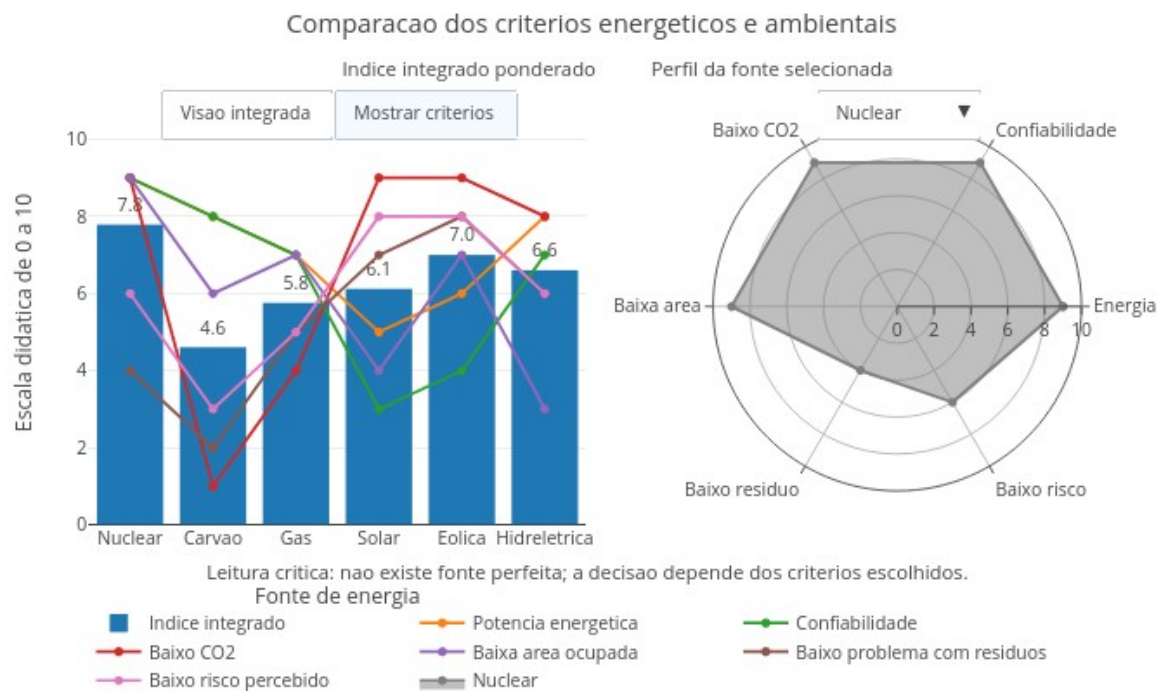


Figura 7.5: Escolher fonte de energia envolve decisão pública, ambiental e tecnológica. Clique com o botão direito neste [LINK](#) e abra em nova aba.

O script compara diferentes fontes de energia: nuclear, carvão, gás, solar, eólica e hidrelétrica. A comparação considera vários critérios ao mesmo tempo, como produção de energia, confiabilidade, emissão de CO₂, área ocupada, resíduos e risco percebido. Assim, decidir sobre energia exige comparar benefícios, riscos e impactos.

Agite antes de usar

O gráfico de barras resume o índice integrado, enquanto que o gráfico radar mostra o perfil de cada fonte. Uma fonte com radar equilibrado tende a apresentar bom desempenho em vários critérios. Mas uma fonte com radar irregular pode ser muito forte em alguns aspectos e fraca em outros.

Execute o script e observe o gráfico de barras. Depois use o menu suspenso para selecionar uma fonte:

Nuclear
Carvão
Gás
Solar
Eólica
Hidrelétrica

Observe o gráfico radar à direita e depois clique em:

Mostrar critérios

Compare quais critérios favorecem ou desfavorecem cada fonte, e

verifique se existe uma fonte de energia perfeita.

A seguir, um pequeno tutorial para uso do *app*.

1. Rode o script;
2. Observe o índice integrado de cada fonte;
3. Veja qual fonte aparece melhor no ranking inicial;
4. Use o menu suspenso à direita;
5. Compare o perfil radar de cada fonte;
6. Clique em Mostrar critérios;
7. Observe quais critérios puxam cada fonte para cima ou para baixo;
8. Compare nuclear com carvão;
9. Compare nuclear com solar e eólica;
10. Compare hidrelétrica com as demais;
11. Reflita sobre o peso dos critérios;
12. Pergunte o que mudaria se outro critério fosse mais importante. O script mostra que cada fonte possui vantagens e limitações. Uma fonte pode ter alta produção, mas gerar mais resíduos. Outra pode emitir pouco CO₂, mas depender do clima. Outra ainda pode ser confiável, mas exigir grande área ou apresentar riscos específicos. Por isso, decisões energéticas não são técnicas apenas, mas envolve farta discussão entre setores da sociedade, ambiente, economia e políticas públicas.

Segue também alguns cenários para exploração.

1. *Prioridade para baixo CO₂*. Quais fontes se destacam quando reduzir emissões é o principal objetivo ?
2. *Prioridade para confiabilidade*. Quais fontes parecem mais adequadas quando a energia precisa estar disponível continuamente ?
3. *Prioridade para baixo risco percebido*. Por que uma fonte tecnicamente eficiente pode enfrentar resistência social ?
4. *Comparação nuclear X carvão*. Como duas fontes de alta produção podem ter impactos ambientais muito diferentes ?
5. *Comparação nuclear X solar/eólica*. Como baixa emissão de CO₂ pode aparecer em fontes com perfis técnicos bem diferentes ?
6. *Alterando pesos*. No código, modifique:

```
const pesoC02 = 2.5;
const pesoRisco = 2.0;
const pesoConfiabilidade = 0.5;
```

O ranking muda quando a sociedade decide valorizar outros critérios ?

Energia nuclear costuma gerar muita eletricidade com baixa emissão direta de CO₂, mas exige cuidado com rejeitos, segurança e percepção pública. Fontes fósseis podem ser confiáveis, mas emitem mais gases associados ao aquecimento global. Energias solar e eólica emitem pouco CO₂, mas dependem de condições ambientais e sistemas de armazenamento. Por sua vez, hidrelétricas podem produzir muito, mas

ocupam grandes áreas e alteram ecossistemas. Por isso, o futuro energético não depende de uma resposta simples.

Decisão energética é uma comparação de compromissos. Questões como impacto, risco, custo, confiabilidade e consequências precisam ser levadas em conta para escolha de uma fonte energética, mesmo que isso não leva àquela que gera mais energia.

Por dentro do script

O script começa definindo as fontes comparadas:

```
const fontes = ["Nuclear", "Carvao", "Gas", "Solar", "Eolica",
"Hidreletrica"];
```

Depois atribui notas “didáticas” de 0 a 10 para cada critério. Aqui são estimativas, mas o barato é que você pode editá-las a seu critério, também.

```
const energia = [9, 8, 7, 5, 6, 8];
const confiabilidade = [9, 8, 7, 3, 4, 7];
const baixoC02 = [9, 1, 4, 9, 9, 8];
```

Em seguida, o script define pesos:

```
const pesoEnergia = 1.2;
const pesoC02 = 1.4;
```

Esses pesos indicam quais critérios importam mais na decisão. O índice integrado é calculado pela média ponderada:

```
return (
  energia[i] * pesoEnergia +
  confiabilidade[i] * pesoConfiabilidade +
  baixoC02[i] * pesoC02 +
  baixaArea[i] * pesoArea +
  baixoResiduo[i] * pesoResiduo +
  baixoRisco[i] * pesoRisco
) / somaPesos;
```

Por fim, o painel mostra as barras com o índice integrado, as curvas dos critérios, e o radar da fonte selecionada.

POR DENTRO DA FERRAMENTA



Separação de comandos, inserção de comentários, e boas práticas

Os comandos em *JS* são separados por *ponto e vírgula*, “;”. Já os *comentários* constituem trechos não interpretados, comumente utilizados em programação. Comentários são tremendamente úteis quando se deseja colocar um título no *script* de código, por exemplo, para explicar quem é quem nas variáveis ou determinada ação no código, e apontar uma futura melhoria, entre muitos (lista *ToDo*, ou “para fazer”). Em *JS*, os comentários são alternativamente inseridos por “//” antes de cada comando, ou por “/.../”, para comentários mais extensos.

Códigos em *JS* podem ser inseridos também junto à páginas web (HTML) com seus comandos intercalados pela notação que segue (etiquetas ou *tags*):

Uma característica interessante no *JS* é que tanto faz colocar ou não espaços entre os comandos, pois a linguagem não interpreta esses espaços. Contudo, dentro das *boas práticas em programação*, é aconselhável separar blocos de comandos por *identação* de 2 espaços, de modo a facilitar a leitura do código. Outro cuidado que deve ser apreciado diz respeito ao uso de fonte maiúscula ou minúscula. *JS* interpreta diferentemente termos em caixa alta ou baixa do teclado, então é bom prestar atenção pra não incorrer em erro na interpretação.

Declaração de variáveis

Entre os diversos tipos de variáveis, duas são frequentemente utilizadas: *const* e *let*. A variável *const* é utilizada para se atribuir um valor fixo a uma variável no código, enquanto a variável *let* é empregada quando se deseja reatribuir um valor a ela, por meio de algum cálculo ou laço iterativo. Exemplificando, quando se deseja variáveis de controle que mudam ao longo do tempo (*sliders*, passos, intervalos).

Obs: em versões antigas de *JS* é também empregado *var*, hoje em desuso.

“Botando a mão na massa”

Como esse treinamento envolve instruir o algoritmo com uma *entrada* para que sua interpretação resulte numa *saída*, aqui propomos essa última pelo comando `print`, como segue.

```
const a = -0.2;
print(a)
```

Para esse treinamento, use o *JSPlotly* personalizado do capítulo anterior. Assim, teste a atribuição da variável *const* e do comando `print` por cópia do trecho acima e cola no campo de texto **ÁREA PARA DIGITAR COMANDOS** do script. Existem outros comandos de saída para *JS*, como `console.log()` e `document.body.appendChild()`, mas não utilizáveis pelo *JSPlotly*.

Pode-se atribuir também um texto para apresentar a constante, veja:

```
const a = -0.2;
print("constante:", a)

const a = 5; // variável constante (não se altera ao longo do código)
const b = 10; // variável que pode alterar seu valor no código
print(a*b)
```

Outro exemplo, com inserção de texto antes do resultado:

```
let a = 5;
const b = 10;

let resultado = a + b;
print("Soma de a + b:", resultado);
```

###const versus let

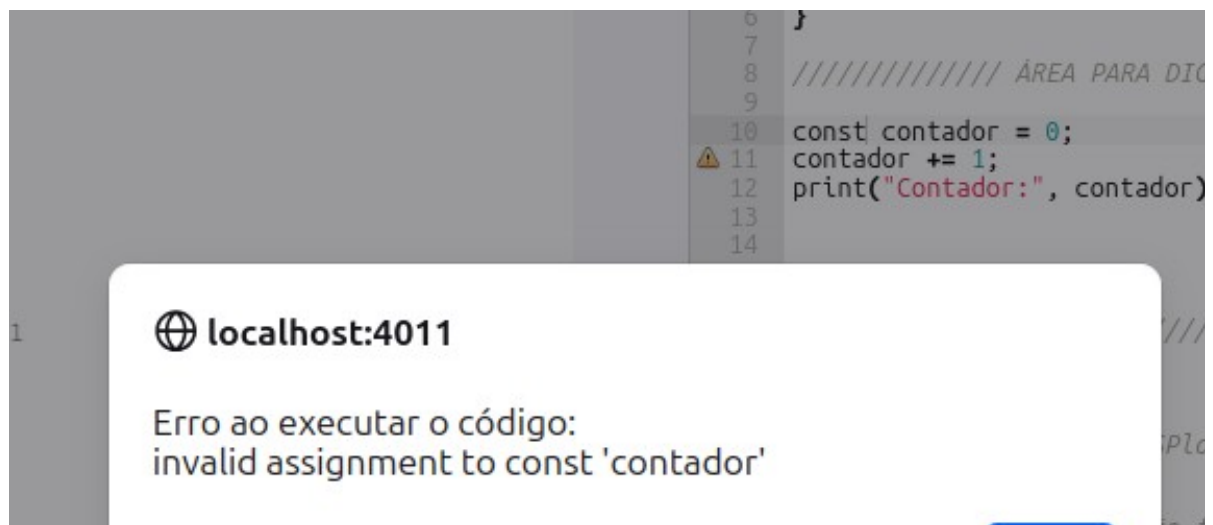
Pode parecer que *const* e *let* são distintas apenas na aparência, já que operam de modo intercambiável. Contudo, para explicitar a diferença entre os dois comandos, apague o ecrã gráfico (*clean*) e substitua a linha de código da seção anterior pela que segue na **ÁREA PARA DIGITAR COMANDOS** do script:

```
let passo = 0.5; // usuário pode alterar
let x0 = 0; // domínios dinâmicos
print(passo, x0)

// ... mais tarde:
passo = 0.2; x0 = 30;
print(passo, x0)
```

Aparentemente, o resultado sugere que na prática nada muda, sendo a diferença mais centrada na clareza e segurança do código. Mas não é bem assim. Experimente agora trocar o campo pra executar o código abaixo:

Agora substitua a variável do “*contador*” para *const* ao invés de *let*. O resultado incorrerá no erro abaixo:



Resumindo: para constantes, use *const* e para variável reatribuível use *let*.

Tipos de dados

Como é comum também em outras linguagens, *JavaScript* possui alguns tipos comuns para definição de dados. Para uso de *Plotly.js*, são bem significativos:

Seguem alguns exemplos práticos. Experimente trocar cada um na saída *print*:

```
// Numbers:
let comprimento = 116;
let peso = 75;

// Strings:
let color = "Yellow";
let lastName = "Johnson";

// Booleans
let x = true;
let y = false;

// Object:
const membro = {firstName:"John", lastName:"Doe"};

// Array object:
const carros = ["Saab", "Volvo", "BMW"];

// Date object:
const datas = new Date("2022-03-25");

// Saídas:
print(comprimento)
```

Constantes

Por vezes, é necessária a inserção de constantes matemáticas em equações para construção de gráficos ou para fazer cálculos. Seguem alguns exemplos. Para a saída no *JSPlotly* customizado abaixo, basta o bom e velho *print* que segue ao também bom e velho *clean*.

```
Math.PI - valor de (pi), aproximadamente 3.14159
Math.E - valor da constante de Euler, aproximadamente 2.71828
Math.LN2 - logaritmo natural de 2, aproximadamente 0.693
Math.LN10 - logaritmo natural de 10, aproximadamente 2.302
Math.LOG2E - logaritmo de e na base 2, aproximadamente 1.442
Math.LOG10E - logaritmo de e na base 10, aproximadamente 0.434
Number.MAX_VALUE - maior número positivo
Number.MIN_VALUE - menor número positivo...
```

Para constantes naturais, contudo, é necessário defini-las antecipadamente, como em:


```
// Definindo constantes físico-químicas comuns
const Avogadro = 6.02214076e23; // Número de Avogadro (mol-1)
const Boltzmann = 1.380649e-23; // Constante de Boltzmann (J/K)
const Planck = 6.62607015e-34; // Constante de Planck (J·s)
const gas = 8.31446; // Constante dos gases ideais (J/(mol·K))
const luz = 299792458; // Velocidade da luz no vácuo (m/s)
```

Funções

Funções constituem um bloco de códigos para efetuar alguma ação. Em *JavaScript* uma função possui a seguinte sintaxe:

```
function nome(parâmetro1, parâmetro2, parâmetro3) {
  // código para execução
}
```

Seguem alguns exemplos de funções para testes.

```
function soma(a) { // define o nome e os "parâmetros" da função
  return a + a // define os argumentos da função (no caso, a*a),
               // pra retorna dos valores
}
```

```
// Teste
let x = soma(7); // aplica parâmetros
print(x) // saída da função
```

Obs: O comando *return*, por vezes omitido, se apresenta útil quando se deseja que o “retorno” seja concluído somente ao final do fluxo do script, sem resultados intermediários.

```
function expoente(a, b) { // define o nome e os "parâmetros" da
  função
  return a ** b; // define os argumentos da função (no caso, a^b),
                // pra retorna dos valores
}
```

```
// Teste da função
let x = expoente(4, 3); // aplica parâmetros
print(x); // saída da função
```

```
function indexa(vetor){
  return vetor + 1
}
```

```
var vetor = [1, 2, 3, 4, 5];
print(indexa(vetor))
```



```
function Celsius(Fahrenheit) {  
  return (5/9) * (Fahrenheit-32);  
}
```

```
let conversao = Celsius(115);  
print(conversao)
```

Funções de seta (arrow functions)

Como visto anteriormente, uma notação alternativa é a função *arrow*, que abrevia a sintaxe de funções. Segue um exemplo:

```
const multiplica = (a, b) => a * b;  
print(multiplica(4, 6));
```

Se gostou até aqui...não perca as **estruturas de controle** do capítulo final desta obra.

8. Relatividade e Quântica — Quando espaço, tempo, energia e matéria deixam de ser intuitivos



O MUNDO TE CHAMA

Você já percebeu que quase tudo ao seu redor depende de uma Física que não aparece no cotidiano ? O GPS do celular precisa corrigir efeitos relativísticos. O laser depende da interação quântica entre luz e matéria. O painel solar funciona porque fótons arrancam elétrons de materiais. O microscópio eletrônico usa comportamento ondulatório de partículas. O computador moderno só existe porque entendemos semicondutores, bandas de energia e elétrons em escala quântica. A Física Moderna entra em cena quando a Física Clássica não consegue explicar alguns aspectos do mundo.

Exemplos não faltam, como o comportamento de um objeto em velocidades próximas à velocidade da luz, os fenômenos que ocorrem nas escalas muito pequenas de átomos e subpartículas, em energias muito altas, e em situações onde medir já interfere no que é medido. A ideia desse capítulo é aproximar dois grandes ramos da Física Moderna, a Relatividade, que muda nossa ideia de espaço, tempo, massa e energia, e a Física Quântica, que muda nossa ideia de matéria, luz, trajetória, medida e probabilidade.

Depois de explorar, você consegue:

- explicar por que espaço e tempo não são absolutos;
- interpretar dilatação do tempo, contração do comprimento e fator de Lorentz;
- relacionar massa e energia pela expressão $E = mc^2$;
- compreender a dualidade onda-partícula;
- relacionar frequência, comprimento de onda e energia de fótons;
- reconhecer aplicações reais da Física Moderna em GPS, lasers, painéis solares, medicina, eletrônica e computação;
- comparar previsões clássicas, relativísticas e quânticas em situações-limite;
- manipular códigos para visualizações interativas no tema.

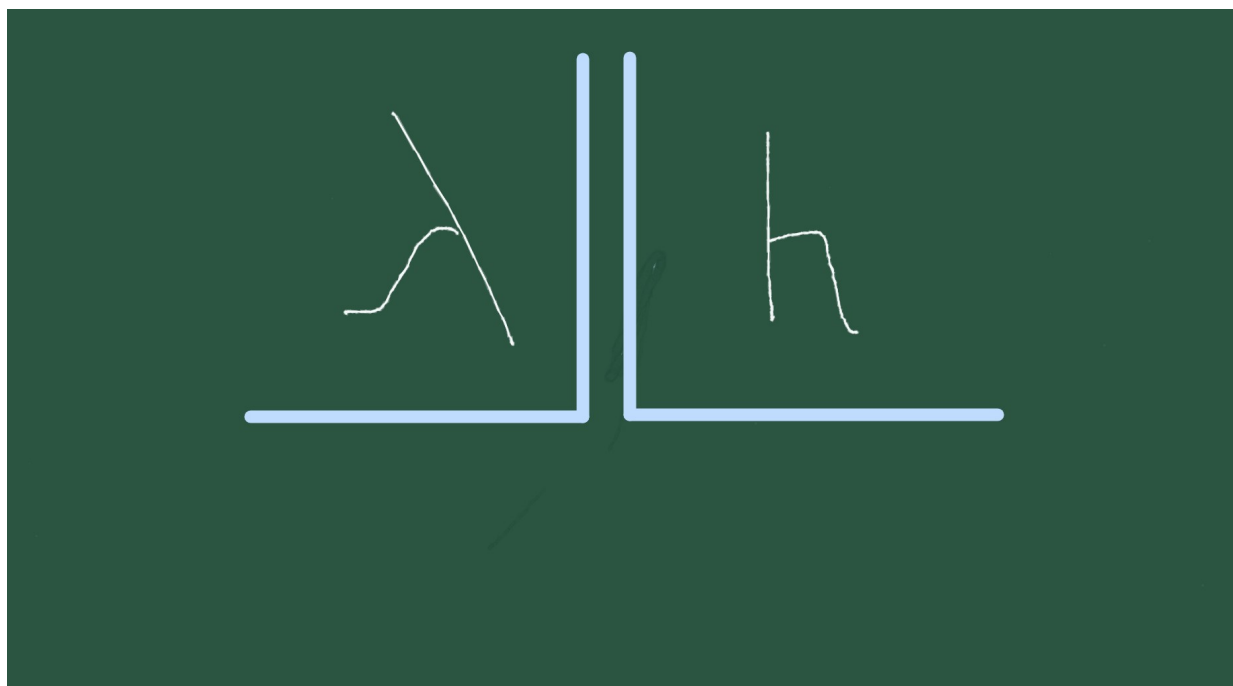


Figura 8.1: Quando visto de muito perto ou rápido demais, o mundo não se comporta como estamos acostumados.

MEXA ANTES DE ENTENDER



Física Moderna *não é para amadores*". Antes de qualquer definição formal, experimente o JSPlotly que abre esse capítulo. Ele trata da Relatividade, quando a velocidade muda tempo e espaço. O script mostra três efeitos centrais da relatividade especial: o aumento do fator de Lorentz, a dilatação do tempo, e a contração do comprimento.

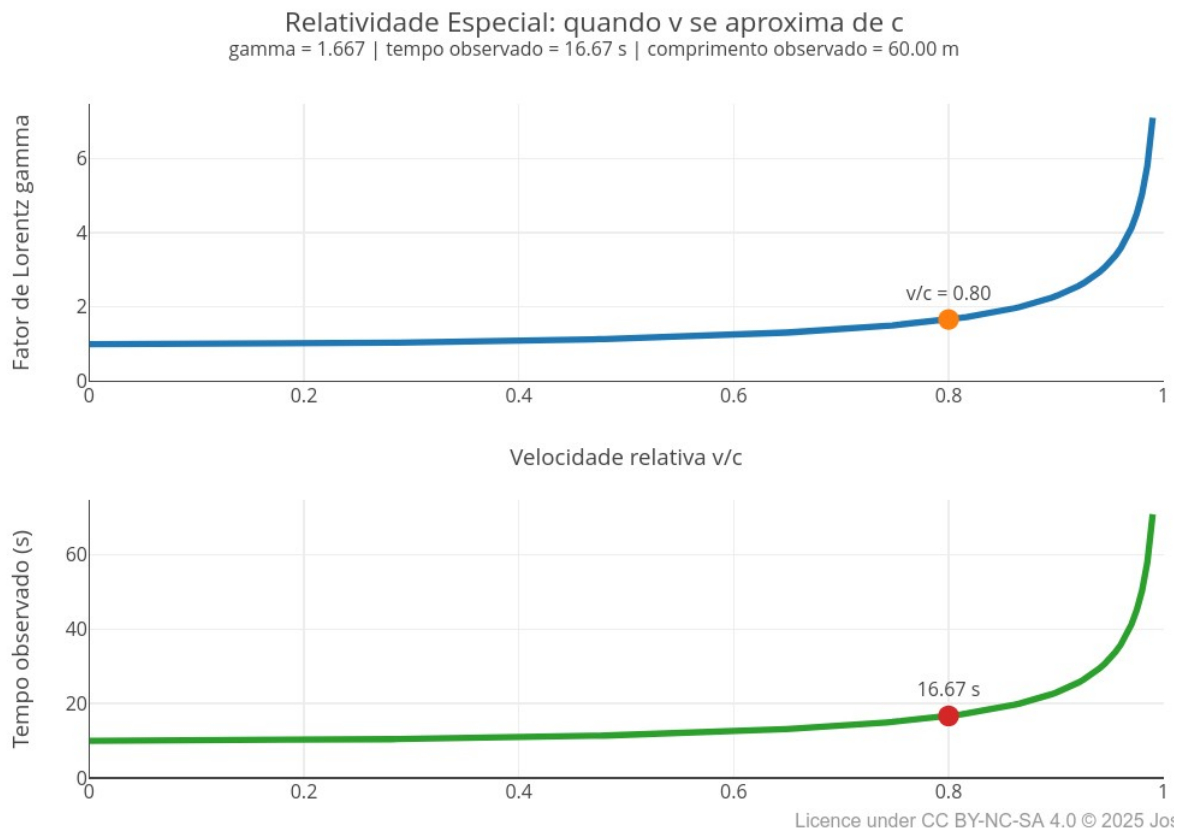


Figura 8.2: Os efeitos relativísticos são pequenos no cotidiano, mas enormes perto da velocidade da luz. Clique com o botão direito do mouse neste [LINK](#) e abra-o numa nova aba.

Os três gráficos do *app* referem-se à relatividade, mas vista por ângulos diferentes. O primeiro mostra que o fator de Lorentz cresce cada vez mais rápido quando a velocidade se aproxima da velocidade da luz. O segundo mostra que o tempo observado aumenta com esse fator. E o terceiro mostra que o comprimento observado diminui.

A variável principal é `betaEscolhido`, que representa a razão entre a velocidade do objeto v e a velocidade da luz c :

$$\beta = \frac{v}{c} \quad (8.1)$$

Quando `betaEscolhido` se aproxima de 1, a velocidade se aproxima da velocidade da luz. Nesse limite, tempo e espaço deixam de se comportar como no cotidiano. Assim, execute o script com:

```
const betaEscolhido = 0.80;
```

Depois teste valores menores e maiores (0.10, 0.50, 0.95, e 0.99).

Observe principalmente o que acontece quando o valor passa de 0.90. Depois altere também:

```
const tempoProprio = 10;
const comprimentoProprio = 100;
```

Veja como os efeitos relativísticos aparecem sobre diferentes tempos e comprimentos. No cotidiano, velocidades pequenas produzem efeitos imperceptíveis. Mas, em velocidades muito altas, surge o *fator de Lorentz* (γ). Ele é um número adimensional (sem unidade de medida) que indica o grau de dilatação do tempo e de contração do espaço, sendo sempre maior ou igual a 1. Dessa forma, à medida que um objeto se aproxima da velocidade da luz, ele quantifica os efeitos relativísticos, como a dilatação do tempo, contração do comprimento e aumento da massa relativística. Uma equação simplificada para o *fator* é dada abaixo:

$$\gamma = \frac{1}{\sqrt{1 - \beta^2}} \quad (8.2)$$

Esse fator modifica as medidas observadas. O tempo observado aumenta:

$$t = \gamma \cdot t_0 \quad (8.3)$$

E o comprimento observado diminui:

$$L = \frac{L_0}{\gamma} \quad (8.4)$$

Por isso, quanto maior a velocidade relativa, mais o tempo se dilata e mais o comprimento se contrai. Segue um breve tutorial de uso do *app*.

1. Rode o script com `betaEscolhido = 0.80`;
2. Observe os três gráficos;
3. Veja o fator de Lorentz no primeiro gráfico;
4. Compare o tempo próprio com o tempo observado;
5. Compare o comprimento próprio com o comprimento observado;
6. Troque para `betaEscolhido = 0.10`;
7. Observe como os efeitos quase desaparecem;
8. Troque para `betaEscolhido = 0.95`;
9. Veja como os efeitos crescem rapidamente;
10. Teste `betaEscolhido = 0.99`;
11. Observe que pequenas mudanças próximas da velocidade da luz produzem grandes diferenças.

Agora alguns cenários para você “relativizar”.

1. *Velocidade cotidiana*. Por que não percebemos dilatação do tempo em situações comuns ?

```
const betaEscolhido = 0.01;
```

2. *Velocidade intermediária*. A metade da velocidade da luz já produz efeitos importantes ?

```
const betaEscolhido = 0.50;
```

3. *Próximo da luz.* Por que o tempo observado aumenta tanto quando a velocidade se aproxima de c ?

```
const betaEscolhido = 0.95;
```

4. *Quase velocidade da luz.* O que acontece com tempo e comprimento quando v/c fica muito próximo de 1?

```
const betaEscolhido = 0.99;
```

5. *Viagem curta no referencial do viajante.* Como uma pequena duração medida pelo viajante pode parecer maior para outro observador ?

```
const betaEscolhido = 0.95;
```

```
const tempoProprio = 5;
```

6. *Nave comprida.* Como o comprimento observado muda quando o objeto se move muito rapidamente?

```
const betaEscolhido = 0.90;
```

```
const comprimentoProprio = 200;
```

No geral, a ideia que fica pode parecer bem estranha na Física Moderna. Quando a velocidade se aproxima da velocidade da luz, o comportamento do tempo e do espaço deixa de parecer familiar. O tempo e o comprimento medidos por observadores diferentes podem não serem o mesmo, o comprimento medido por observadores diferentes pode não ser o mesmo.

A grande “sacada” conceitual é perceber que tempo e espaço não são absolutos, mas dependem do movimento relativo entre o observador e o objeto.

Por dentro do script

O script percorre valores de velocidade relativa entre 0 e 0,99:

```
for (let i = 0; i <= n; i++) {
  let b = (betaMax * i) / n;
  let g = 1 / Math.sqrt(1 - b * b);
```

Para cada valor de beta, calcula o fator de Lorentz:

```
gamma.push(g);
```

Depois calcula o tempo observado:

```
tempoObservado.push(g * tempoProprio);
```

E o comprimento observado:

```
comprimentoObservado.push(comprimentoProprio / g);
```

O ponto destacado no gráfico mostra exatamente o valor escolhido pelo aluno:

```
const betaEscolhido = 0.80;
```

Assim, a equação vira curva, e a curva vira uma comparação visual.



FAZENDO APARECER

A Física Clássica funciona muito bem em nosso cotidiano. Ela descreve quedas, lançamentos, colisões, ondas, máquinas, fluidos e circuitos com enorme sucesso. Mas, no final do século XIX e início do século XX, alguns problemas começaram a desafiar essa visão. A luz se comportava como onda, mas em certos experimentos parecia partícula. Elétrons eram partículas, mas em certas situações pareciam ondas. O tempo parecia não ser absoluto, embora a velocidade da luz fosse a mesma para diferentes observadores. A energia parecia contínua, mas certos fenômenos só podiam ser explicados se ela viesse em “pacotes” de luz, ou fótons.

8.0.1 Relatividade: espaço e tempo em movimento

Na relatividade especial, a velocidade da luz no vácuo é tomada como constante fundamental, mudando tudo. Se a velocidade da luz é a mesma para diferentes observadores, então espaço e tempo precisam se ajustar. O fator que mede esse ajuste é o fator de Lorentz:

$$\gamma = \frac{1}{\sqrt{1 - \frac{v^2}{c^2}}} \quad (8.5)$$

Na [Equação 8.5](#), v é a velocidade relativa entre o observador e o objeto em movimento, e c é a velocidade da luz.

Quando v é muito menor que c , o fator de Lorentz é praticamente 1. Nesse caso, a Física Clássica continua funcionando muito bem. Mas quando v se aproxima de c , γ cresce rapidamente. A partir daí, aparecem efeitos relativísticos importantes:

$$\Delta t = \gamma \Delta t_0 \quad (8.6)$$

O intervalo de tempo medido por um observador pode ser maior que o tempo próprio (Δt_0 ; intervalo de tempo medido por um observador que está em repouso em relação ao evento que está acontecendo). Também ocorre contração do comprimento na direção do movimento:

$$L = \frac{L_0}{\gamma} \quad (8.7)$$

O comprimento medido pode ser menor que o comprimento próprio (L_0 ; distância ou tamanho medido por um observador que está em repouso em relação ao objeto). Esses efeitos não são “defeitos de observação”, mas fazem parte da estrutura do espaço-tempo.

8.0.2 Massa e energia: duas formas de representar a mesma realidade

Uma das consequências mais conhecidas da relatividade é a equivalência entre massa e energia:

$$E = mc^2 \quad (8.8)$$

Essa expressão mostra que massa pode ser entendida como uma forma concentrada de energia. Ela ajuda a compreender fenômenos nucleares, produção de energia em estrelas, decaimentos radioativos e processos de partículas.

8.0.3 Quântica: energia, luz e matéria em pacotes

Na Física Quântica, muitos fenômenos não podem ser explicados quando se supõe que a energia varia continuamente. Para a luz, a energia de um fóton é dada por:

$$E = hf$$

Em que E é a energia do fóton, h é a constante de Planck ($6,626 \times 10^{-34}$ J*s), e f é a frequência da radiação. Como a velocidade da luz também se relaciona com frequência e comprimento de onda:

$$c = \lambda f \quad (8.9)$$

podemos escrever:

$$E = \frac{hc}{\lambda} \quad (8.10)$$

Isso revela que, *quanto menor o comprimento de onda, maior a energia do fóton*. Por isso radiações como ultravioleta, raios X e raios gama são mais energéticas que ondas de rádio, micro-ondas ou luz infravermelha.

8.0.4 Dualidade onda-partícula

A luz pode se comportar como onda em fenômenos como interferência e difração (capacidade de ondas contornar obstáculos ou se espalhar ao passar por fendas). Mas também pode se comportar como partícula em fenômenos como o efeito fotoelétrico (emissão de elétrons por material atingido por onda eletromagnética). A matéria também apresenta dualidade. Partículas como elétrons podem apresentar comportamento ondulatório. A relação de de Broglie associa comprimento de onda ao momento de uma partícula (tendência em manter movimento ou causar rotação):

$$\lambda = \frac{h}{p} \quad (8.11)$$

Foram dessas observações que surgiram tecnologias como microscópios eletrônicos e difração de elétrons ao final da década de 1920 e início da década de 1930, bem como novas e recentes tecnologias baseadas em propriedades quânticas da matéria. Como o computador quântico, por

exemplo, que utiliza esses princípios para um processamento ultra-avancado de dados.

Você observou até agora como espaço e tempo deixam de ser absolutos na relatividade. Na verdade, até a própria luz deixa de se comportar apenas como uma onda contínua na Física Quântica, também. Nesse caso, a energia luminosa aparece em pacotes discretos chamados fótons, partículas fundamentais da radiação eletromagnética. O próximo *app* de JSPlotly explora essa ideia, relacionando comprimento de onda, frequência e energia da luz em diferentes regiões do espectro eletromagnético.

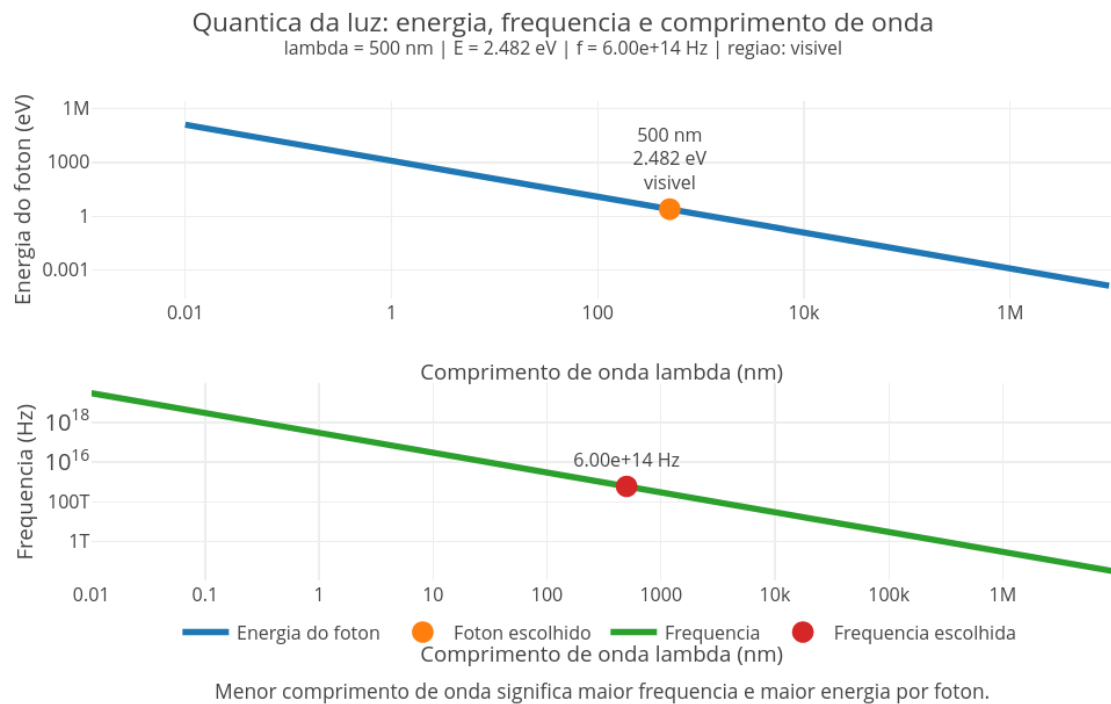


Figura 8.3: Pequenas mudanças no comprimento de onda podem produzir enormes diferenças de energia. Clique com o botão direito do mouse neste [LINK](#) e abra-o numa nova aba.

O script mostra como comprimento de onda, frequência e energia estão conectados na Física Quântica. A ideia central é que a luz transporta energia em pacotes chamados fótons. Cada fóton possui uma energia dada por:

$$E = hf \quad (8.12)$$

Como frequência e comprimento de onda estão relacionados:

$$f = \frac{c}{\lambda} \quad (8.13)$$

Então:

$$E = \frac{hc}{\lambda} \quad (8.14)$$

Isso significa que comprimentos de onda menores correspondem a frequências maiores e fótons mais energéticos.

Agite antes de usar

O primeiro gráfico mostra a energia do fóton, enquanto o segundo mostra sua frequência. Ambos usam escala logarítmica porque o espectro eletromagnético cobre intervalos gigantescos de valores (de 10^{-12} m - ondas de rádio, a 10^{-3} m - raios gama). Execute o script com o seguinte comprimento de onda:

```
const lambdaEscolhido_nm = 500;
```

Depois teste diferentes regiões do espectro (ex: comprimentos de 700, 550, 450, 100, e 0.1 nm). E observe como energia e frequência mudam drasticamente. Compare especialmente luz visível (320-700), ultravioleta (200-320) e raios X (0.01-10). E perceba que diferentes tipos de luz possuem energias bastante diferentes. E se quiser arriscar um pouquinho mais, tente os raios-gama (0.001-0.05).

Uma grande vantagem das simulações digitais é que elas não fazem mal algum no mundo real !

No script, o comprimento de onda controla tudo. Quando λ diminui, a frequência e a energia do fóton aumentam. Por isso que o infravermelho possui fótons menos energéticos, a luz visível possui energias intermediárias, e os raios X e raios gama possuem energias muito maiores. Segue um rápido tutorial de uso.

1. Rode o script com `lambdaEscolhido_nm = 500`;
2. Observe a posição do ponto nos dois gráficos;
3. Veja a energia do fóton;
4. Veja a frequência correspondente;
5. Troque para 700 nm;
6. Compare com a luz vermelha;
7. Troque para 450 nm;
8. Observe o aumento da energia;
9. Teste 100 nm;
10. Veja a região ultravioleta;
11. Teste 0.1 nm.
12. Observe a região de raios X;
13. Compare como energia e frequência crescem quando o comprimento de onda diminui.

Segue também alguns cenários para você explorar:

1. *Luz vermelha*. Por que a luz vermelha possui menos energia por fóton do que a luz azul ?

```
const lambdaEscolhido_nm = 700;
```

2. *Luz verde*. Troque o comprimento de onda para 550. A região central do visível possui energia intermediária ?

3. *Luz azul*. Agora edite para 450. Por que a luz azul possui maior frequência e maior energia ?
4. *Ultravioleta*. Mude para 100. Por que radiações ultravioletas podem causar mais danos biológicos ?
5. *Raios X*. Mude para 0.1. Como fótons tão energéticos conseguem atravessar materiais e tecidos ?
6. *Infravermelho distante*. Mude para 1000000 (pode escrever mais simples no código, como *1e6*). Por que ondas muito longas possuem frequências e energias menores ?

Após experimentar os valores acima, você deve perceber visualmente que, para um comprimento de onda menor, a frequência e a energia são maiores. Ou seja, cor, radiação e energia numa única relação matemática.

Por dentro do script

O script começa definindo as constantes fundamentais, constante de Planck (h) e velocidade da luz:

```
const h = 6.626e-34;
const c = 3.0e8;
```

Depois calcula a energia do fóton:

```
const energia_J = h * c / lambda_m;
```

Em seguida converte para elétron-volts:

```
return energia_J / e;
```

A frequência é calculada por:

```
return c / lambda_m;
```

Depois o script percorre vários comprimentos de onda em escala logarítmica:

```
for (let i = -2; i <= 7; i += 0.04)
```

Isso permite visualizar simultaneamente regiões muito diferentes do espectro eletromagnético.

O ponto destacado mostra o fóton escolhido pelo aluno:

```
const lambdaEscolhido_nm = 500;
```

Assim, o você pode navegar do infravermelho até raios gama alterando apenas um parâmetro...e sem se ferir.



A Física Quântica tem lugar no mundo quando percebemos que a energia nem sempre varia continuamente. Às vezes, ela aparece em pacotes discretos, os fótons, uma das primeiras portas para esse novo modo de enxergar a natureza. No cotidiano, esses efeitos são pequenos demais para serem percebidos. Mas em situações que envolvem altas velocidades ou grandes energias, como satélites, raios cósmicos e tecnologias de alta precisão, a relatividade precisa ser levada em conta, já que o movimento em grandes velocidades também altera as medidas de espaço e tempo.

A quantização da luz está presente também em muitas tecnologias modernas, como painéis solares, lasers, sensores digitais, exames de raios X, telecomunicações, espectroscopia, dispositivos de Astronomia e tantos outros. E também está presente no LED (diodo emissor de luz), aquela luzinha que tem em quase todos os equipamentos eletrônicos, incluindo equipamentos de som e celulares, e que emite luz por um efeito fotoelétrico “ao contrário”, quando a eletricidade vira fótons quantizados.

Para experienciar um pouco mais sobre a relativização do espaço-tempo e a quantização da luz, que tal um último script de JSPlotly ? Este terceiro script une os dois mundos em uma mesma visualização. Dessa vez você compara cenários clássicos, relativísticos e quânticos ao mesmo tempo, percebendo que a Física Moderna não substitui completamente a Física clássica, mas amplia seus limites de velocidade, energia ou escala. O *app* integrado da [Figura 8.4](#) combina efeitos relativísticos e quânticos usando uma animação em diferentes cenários interativos..

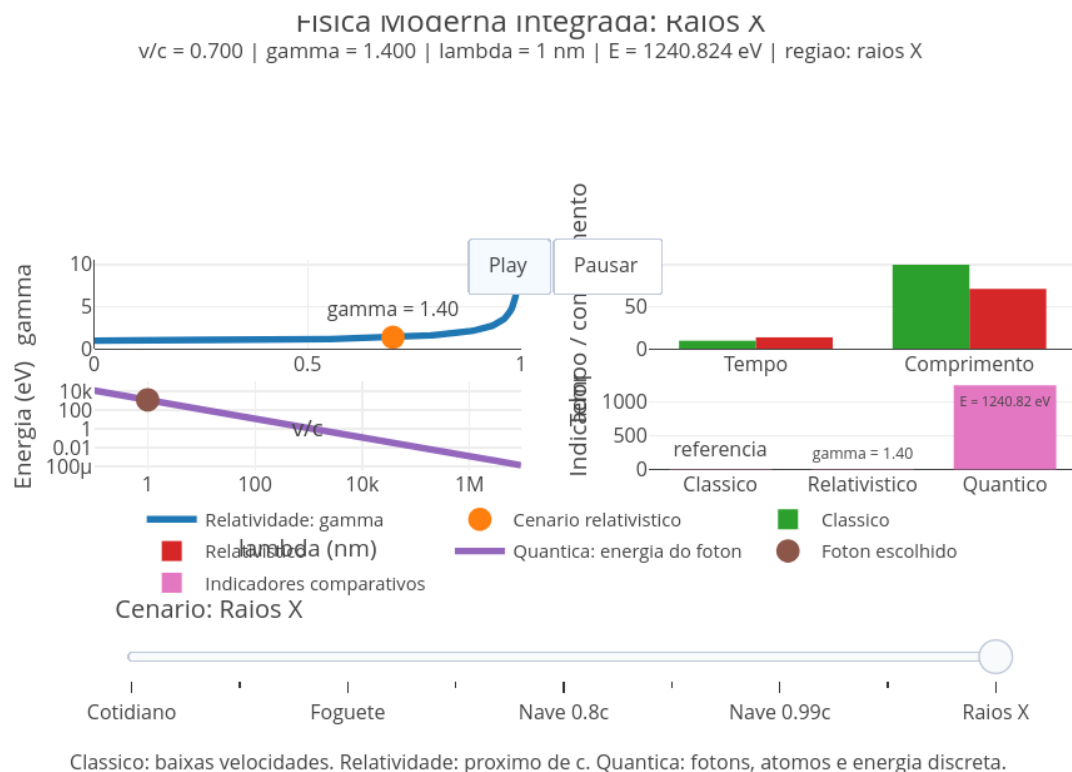


Figura 8.4: O mundo parece clássico no dia-a-dia, mas é relativístico e quântico em condições extremas. Clique com o botão direito do mouse neste [LINK](#) e abra-o numa nova aba.

O script integra dois pilares da Física Moderna, a Relatividade Especial e a Física Quântica. Agora, diferentes cenários podem ser comparados usando um único sistema interativo. O script mostra simultaneamente:

- crescimento do fator relativístico;
- dilatação do tempo;
- contração do comprimento;
- energia dos fótons;
- comparação entre mundo clássico, relativístico e quântico.

A ideia principal é perceber que a Física Moderna surge quando velocidades, energias ou escalas deixam de se comportar como no cotidiano.

Agite antes de usar

O gráfico superior esquerdo mostra a relatividade especial. O gráfico superior direito compara medidas clássicas e relativísticas. O gráfico inferior esquerdo mostra a energia dos fótons. E o gráfico inferior direito resume os indicadores comparativos.

Execute o script e clique em Play, observando como os cenários mudam automaticamente. Depois utilize o *slider* inferior para navegar manualmente entre:

- cotidiano;
- avião rápido;
- foguete;
- nave relativística;
- ultravioleta;
- raios X.

Observe como os gráficos mudam juntos. Compare especialmente o crescimento de γ , a energia dos fótons, e as diferenças entre o comportamento clássico e moderno. No cotidiano as velocidades são pequenas comparadas à velocidade da luz, as energias são relativamente baixas, e os efeitos quânticos quase não aparecem diretamente. Por isso, o mundo parece bastante intuitivo. Mas quando as velocidades se aproximam de c , os comprimentos de onda ficam muito pequenos, ou as energias ficam muito altas, surgem efeitos relativísticos e quânticos, socorridos pela Física Moderna na explicação desses extremos.

Segue um tutorial para uso do *app*.

1. Rode o script;
2. Clique em Play;
3. Observe primeiro o gráfico relativístico superior esquerdo;
4. Veja o crescimento de γ ;
5. Compare os gráficos de tempo e comprimento;
6. Observe o gráfico quântico inferior esquerdo;
7. Veja como a energia cresce quando λ diminui;
8. Observe os indicadores comparativos no canto inferior direito;
9. Use o slider para voltar aos cenários;

10. Compare cotidiano e nave relativística;
11. Compare luz visível e raios X;
12. Observe como diferentes regiões da Física aparecem em diferentes escalas.

Perceba, exercitando-se acima, que Relatividade e Quântica mostram quando a Física clássica deixa de ser suficiente. Seguem alguns cenários para você “brincar”:

1. *Cotidiano*. Por que os efeitos relativísticos e quânticos parecem quase invisíveis no dia a dia ?
2. *Avião rápido*. Mesmo velocidades muito altas para humanos ainda são pequenas comparadas à velocidade da luz ?
3. *Nave 0.5c*. O que começa a mudar quando metade da velocidade da luz é atingida ?
4. *Nave 0.95c*. Por que tempo e comprimento passam a se comportar de modo tão diferente do cotidiano ?
5. *Ultravioleta*. Por que radiações ultravioletas possuem fótons mais energéticos que a luz visível ?
6. *Raios X*. Como comprimentos de onda tão pequenos produzem energias tão grandes ?
7. *Comparação integrada*. O que há em comum entre efeitos relativísticos e efeitos quânticos ?

A proposta visual que fica aqui é a de que Física clássica funciona bem em condições cotidianas. Mas é a Relatividade da Física Moderna quem domina em velocidades extremas, e é a Física Quântica quem domina em escalas microscópicas e energias altas. Em outras palavras, a Moderna atua quando a Clássica é limitante para explicar o mundo.

Por dentro do script

O script começa definindo funções básicas:

```
function gammaLorentz(beta)
```

e:

```
function energiaEV(lambda_nm)
```

Depois cria vários cenários:

```
const cenarios = [...]
```

Cada cenário possui uma velocidade relativa e um comprimento de onda característico.

Por exemplo:

```
{ nome: "Nave 0.95c", beta: 0.95, lambda: 380 }
```

Depois o script calcula o fator relativístico, o tempo observado, o comprimento observado, a energia do fóton, e a frequência da luz. Os frames geram a animação automática e o slider permite comparar

E SE...



Agora o sublime momento para experiências e reflexões do córtex cerebral (pé no chão) e da glândula pineal (pé nas nuvens).

- E se a velocidade da luz fosse menor ?
- E se a constante de Planck fosse maior ?
- E se efeitos quânticos fossem visíveis no cotidiano ?
- E se humanos pudessem viajar a 99 % da velocidade da luz ?
- E se elétrons se comportassem apenas como pequenas bolinhas clássicas ?
- E se o tempo realmente passasse de forma diferente para pessoas em movimentos muito distintos ?
- E se astronautas em viagens extremamente rápidas envelhecessem menos que pessoas na Terra ?
- E se um relógio em uma nave relativística registrasse uma duração diferente daquela observada por alguém parado ?
- E se o espaço não fosse fixo, mas dependesse do movimento do observador ?
- E se objetos muito rápidos realmente parecessem menores na direção do movimento ?
- E se a velocidade da luz fosse diferente? O Universo ainda seria parecido com o que conhecemos ?
- E se fosse possível perceber diretamente os efeitos relativísticos no cotidiano, como em carros ou aviões ?
- E se a luz não viesse em pacotes discretos chamados fótons ?
- E se a cor da luz estivesse ligada não apenas à aparência, mas também à quantidade de energia transportada ?
- E se nossos olhos fossem capazes de enxergar ultravioleta ou infravermelho ?
- E se o céu tivesse outra aparência para seres vivos capazes de perceber diferentes regiões do espectro eletromagnético ?
- E se fótons de raios X fossem visíveis aos nossos olhos ?
- E se o Sol emitisse principalmente raios gama em vez de luz visível ?
- E se tecnologias modernas como celulares, GPS, lasers e painéis solares deixassem de funcionar sem Física Moderna ?
- E se a Física clássica fosse apenas uma aproximação válida para velocidades baixas e energias moderadas ?
- E se o Universo microscópico obedecesse regras completamente diferentes das percebidas no cotidiano ?
- E se partículas pudessem se comportar como ondas ?
- E se observar um fenômeno alterasse o próprio fenômeno observado ?
- E se espaço, tempo, matéria e energia fossem partes de uma mesma estrutura mais profunda ?
- E se o mundo que percebemos fosse apenas uma pequena faixa de tudo o que realmente existe no Universo ?
- E se diferentes observadores pudessem discordar sobre tempo e espaço, mas ambos estivessem corretos ?
- E se viajar perto da velocidade da luz transformasse completamente nossa percepção do passado e do futuro ?
- E se civilizações extremamente avançadas utilizassem efeitos relativísticos e quânticos como algo cotidiano ?

- E se nossa intuição estivesse adaptada apenas a uma faixa muito limitada da realidade física ?
- E se a Física Moderna não fosse “estranha”, mas apenas distante da escala em que nossos sentidos evoluíram ?
- E se compreender o Universo exigisse abandonar parte daquilo que parece intuitivo ?
- E se o mais estranho da Física Moderna fosse justamente o fato de ela funcionar tão bem ?

MESMO PADRÃO, OUTROS MUNDOS



A Física Moderna não substitui a Física Clássica em todas as situações, mas amplia seus limites. A Física Clássica continua excelente para muitas situações cotidianas. Mas, quando entramos no muito rápido, no muito pequeno ou no muito energético, a Física Moderna toma um assento cativo.

Detalhando um pouco, é assim com os sistemas de navegação por satélites, GPS (*Global Positioning System*), por exemplo, que se movem rapidamente e estão em regiões onde a gravidade é diferente da superfície da Terra. Nesse caso, a relatividade especial e relatividade geral precisam ser consideradas para corrigir medidas de tempo, já que, sem essas correções, a localização por GPS acumularia erros.

Lasers dependem de transições eletrônicas entre níveis de energia, e esse níveis são quânticos. Não são quaisquer energias que os elétrons podem assumir em átomos e materiais. Painéis solares, por sua vez, funcionam quando a luz chega em fótons. Quando esses fótons interagem com certos materiais (ex: silício), podem liberar elétrons e gerar corrente elétrica. Esse processo depende diretamente da relação entre energia, frequência e estrutura eletrônica dos materiais.

Em medicina diagnóstica, raios X, tomografia, radioterapia, PET scan e ressonância magnética, envolvem ideias modernas sobre radiação, núcleos, partículas, campos e interação entre energia e matéria. Também para computadores e celulares, diversos componentes eletrônicos, tais como transistores, semicondutores, LEDs, sensores, memórias e processadores, dependem de propriedades quânticas dos materiais.

POR DENTRO DA FERRAMENTA



Nessa seção, damos sequência a um aprendizado panorâmico em *JavaScript*, voltado ou não à edição de objetos com JSPlotly, como realizado no capítulo anterior. Nesse sentido, segue um pouco sobre a estrutura de *JS* para aritmética e *arrays* (vetores).

E agora um truque bem bacana !!! **Você poderá treinar a linguagem JavaScript utilizando um JSPlotly específico para isso !**. Esse JSPlotly personalizado está disponível na [Figura 8.5](#) abaixo. Abra-o num navegador e deixe-o quietinho lá. Quando quiser rodar algum trecho desses treinamentos, basta copiar, colar o trecho no campo “ÁREA PARA DIGITAR COMANDOS”, e rodá-lo. Muito legal, não ?!

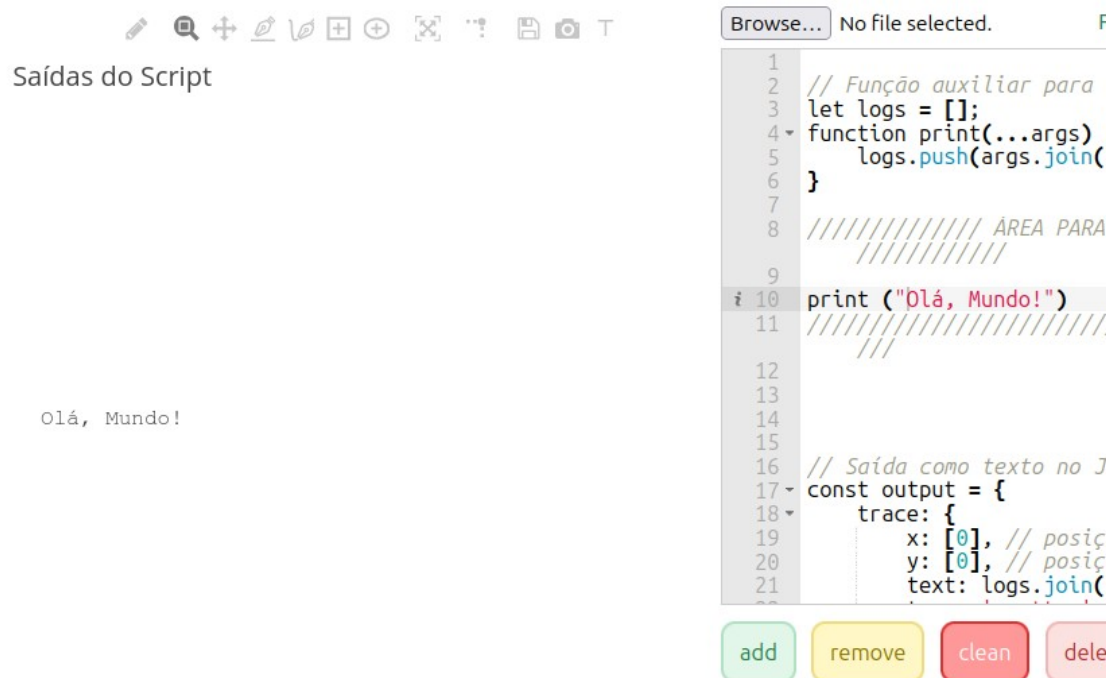


Figura 8.5: JSPlotly personalizado para treinamento da linguagem JavaScript [JSPlotly_treinoJS.html](#). Basta clicar com o botão direito do mouse para abri-lo em outra janela do navegador. Deixe-o quietinho lá. Quando quiser executar algum trecho de código ao longo desse treinamento, basta copiar, colar no campo “ÁREA PARA DIGITAR COMANDOS”, e rodar no JSPlotly personalizado.

Aritmética

Diversas operações matemáticas são possíveis, quer pelo *JS* puro, quer pela instalação de bibliotecas específicas, tais como *Plotly.js* utilizada no JSPlotly, *mathjs* ou *numjs*. Segue abaixo uma lista de operações aritméticas realizadas com a linguagem:

1. + : adição de números ou concatenação de texto (*strings*);
2. - : subtração;
3. * : multiplicação;
4. / : divisão;
5. % : módulo;
6. ++ : incremento de valor;
7. -- : decremento;

8. ****** : exponenciação.

Obs: o operador de módulo % divide o primeiro operando pelo segundo operando e retorna o resto.

Os exemplos a seguir ilustram o uso desses operadores aritméticos, bem como de algumas constantes matemáticas. Para testá-los, cole no *JSPlotly* customizado, como informado acima. Você pode experimentá-los separadamente ou em bloco.

```
let resultado = 0;

resultado = 5 + 3;           print("5 + 3 = ", resultado);
// 8
resultado = 10 - 4;         print("10 - 4 = ", resultado);
// 6
resultado = 6 * 7;          print("6 * 7 = ", resultado);
// 42
resultado = 20 / 5;         print("20 / 5 = ", resultado);
// 4
resultado = 17 % 3;         print("17 % 3 = ", resultado);
// 2
resultado = 2 ** 3;         print("2 ** 3 = ", resultado);
// 8
let contador = 5; contador++; print("contador++ = ", contador);
// 6
let decrementar = 8; decrementar--; print("decrementar-- = ",
decrementar); // 7
let soma = 10; soma += 5;     print("soma += 5 = ", soma);
// 15
let diferenca = 20; diferenca -= 7; print("diferença -= 7 = ",
diferenca); // 13
let produto = 3; produto *= 4; print("produto *= 4 = ",
produto); // 12
let quociente = 24; quociente /= 3; print("quociente /= 3 = ",
quociente); // 8
resultado = Math.sqrt(25);   print("sqrt(25) = ", resultado);
// 5
resultado = Math.abs(-15);   print("abs(-15) = ", resultado);
// 15
resultado = Math.floor(7.8); print("floor(7.8) = ",
resultado); // 7
resultado = Math.ceil(7.2);  print("ceil(7.2) = ", resultado);
// 8
resultado = Math.round(9.4); print("round(9.4) = ",
resultado); // 9
resultado = Math.max(42, 27); print("max(42,27) = ",
resultado); // 42
resultado = Math.min(42, 27); print("min(42,27) = ",
resultado); // 27
resultado = Math.sin(Math.PI/2); print("sin(π/2) = ", resultado);
```

Arrays

Um *array* em *JavaScript* representa uma estrutura de dados para

armazenamento de elementos ordenados. Esses elementos podem envolver qualquer tipo de dado, como números, strings, objetos, etc. Na prática, um *array* pode ser criado, acessado, filtrado, ou mapeado. Seguem exemplos:

Criação

```
const numeros = [1, 2, 3, 4, 5];
const cores = ['vermelho', 'verde', 'azul'];
```

Acesso

```
print(numeros[0]);
print(cores[2]);
```

Ou seja, para você acessar o *array* *numeros*, por exemplo, basta colar e rodar o seguinte trecho no JSPlotly de treinamento:

```
const numeros = [1, 2, 3, 4, 5];
const cores = ['vermelho', 'verde', 'azul'];

print(numeros[0]);
```

Mapeamento (*map*)

```
const numeros = [1, 2, 3, 4, 5];
const quadrados = numeros.map(numero => numero * numero);
print(quadrados);
```

Uma palavrinha sobre métodos

Observe que, diferente do que já foi explicitado, agora aparece um comando múltiplo, como “*numeros.map*”. Existe uma infinidade de comandos com esta sintaxe em *JS*. Ele representa a função que será aplicada a cada elemento do *array* *numeros* pelo método *map*. Além disso, surge a sintaxe “=>” ou sintaxe de *arrow function* (função de seta), uma forma concisa de atribuir um mapeamento. Assim, o comando inteiro:

Em *JavaScript* objetos são tratados como no mundo real, ou seja, possuem *atributos* e *comportamentos*. Atributos descrevem as características que um objeto possui, e os comportamentos as ações que podem realizar. A notação em *JS* para referenciar as propriedades de um objeto podem ser, alternativamente:

```
objeto.propriedade
```

```
...ou...
```

```
objeto[propriedade]
```

Voltando ao exemplo acima e esmiuçando-o um pouquinho mais, o método *map()* (*numeros.map*) é chamado no *array* *numeros* (um *Array.prototype*). O *protótipo* permite transmitir propriedades e métodos para um objeto *JS*. No caso, o método *map()* itera sobre cada elemento do *array* original, aplica uma função a cada um deles e retorna um novo *array*

com os resultados. Assim, *numero => numero * numero* é a *função de seta*. E nesse caso, *numero* é o parâmetro de entrada da função, um atalho para o elemento atual do *array* sendo processado pelo *map*. Em adição, “=>” representa o separador entre o(s) parâmetro(s) e o corpo da função. Finalmente, “*numero * numero*” é a expressão que é executada, com o resultado sendo retornado pela função. Seguem outros exemplos para arrays e mapeamento:

```
const x = [0, 1, 2, 3, 4];
const y = x.map(valor => Math.pow(2, valor))
print(y);

const Vmax = 100;
const Km = 10;
const S_values = Array.from({length: 10}, (_, i) => i * 5);

const V_values = S_values.map(S => (Vmax * S) / (Km + S));

print("Valores de S:", S_values);
print("Valores de V:", V_values);
```

JavaScript, uma linguagem orientada a objetos

Ainda que o esforço para traduzir os comandos acima possa auxiliar como funciona a linguagem, é por óbvio esperar uma certa confusão de termos. Sendo assim, vamos subindo de nível!

Como diz o título acima, *JS* é uma linguagem orientada a objetos. Isso significa que o código é organizado em torno de *objetos*, que combinam dados (*propriedades*) e comportamentos (*métodos*) para modelar entidades do mundo real. Isso permite um código mais modular, reutilizável e fácil de gerenciar, ao organizar a lógica em “*partes*” menores e mais compreensíveis (*pensamento computacional*).

Objetos, propriedades, e métodos

Um *objeto* é uma estrutura que agrupa dados e comportamentos. Os dados ficam guardados em *propriedades*, e os comportamentos são implementados por *métodos* (funções dentro do objeto). Assim, propriedade são pares nome-valor armazenados dentro de um objeto, podendo variar como números, textos, arrays, e outros objetos. Já os métodos são funções associadas a um objeto, e que descrevem ações que ele pode executar.

Objetos em *JS* “*podem ser comparados a objetos da vida real*”, e guardam informações relacionadas entre si. Por exemplo, o objeto “*ponto*” abaixo representa um ponto no plano:

```
const ponto = { x: 2, y: 5 }; // objeto com duas propriedades
```

Assim, as propriedades são as características do objeto:

```
print(ponto.x); // mostra 2
print(ponto.y); // mostra 5
```

“Propriedades de objetos são basicamente as mesmas que variáveis normais em JavaScript, exceto pelo fato de estarem ligadas a objetos”. Propriedades podem ser acessadas por colchetes, [...].

Assim, um objeto pode ser encarado como um “pacote” contendo nome, conteúdo e ferramentas próprias. Virtualmente, tudo em *JavaScript* — arrays, funções, strings, números — é, de alguma forma, um objeto com suas propriedades e métodos. Segue um exemplo:

```
const carro = {
  marca: "Toyota",      // propriedade (dado)
  ano: 2022,            // propriedade
  ligar: function() {   // método (ação)
    console.log("O carro está ligado!");
  }
};

carro.ligar();          // chama o método
console.log(carro.marca); // acessa a propriedade
```

Reforçando (não custa nada...), um objeto *JavaScript* envolve uma coleção de propriedades, onde cada propriedade é definida por um par chave-valor, permitindo uma representação ordenada de elementos. Objetos podem conter diferentes tipos de dados como valores, como números, strings, booleanos, arrays, outras funções e até mesmo outros objetos. Objetos criados são facilmente acessados por seus atributos. Seguem exemplos para testes.

Criação

```
const pessoa = {
  nome: 'João',
  idade: 30,
  cidade: 'São Paulo'
};
```

Acesso

```
print(pessoa.nome);
print(pessoa['idade']);
```

Atributos

```
pessoa.profissao = 'Engenheiro';
print(pessoa);
```

Para um exemplo fechado de objetos:

```
let pessoa = {
  nome: "João",
  idade: 30,
  profissao: "Desenvolvedor",
```

```
hobbies: ["leitura", "música", "jogos"],
endereco: {
  rua: "Rua Principal, 123",
  cidade: "São Paulo",
},
saudacao: function () {
  print("Olá, meu nome é " + this.nome);
},
};

print(pessoa.nome); // Saída: "João"
print(pessoa.hobbies[0]); // Saída: "leitura"
pessoa.saudacao(); // Saída: "Olá, meu nome é João"
```

E por aí vai !! No próximo capítulo essa seção irá explorar um pouco sobre variáveis e funções.